

Vantage: Availability-Graded Broadcast for Signature-Free BFT

NIKITA POLYANSKII, IOTA Foundation, Germany

Digital signatures make blocks and votes transferable evidence: one party can prove to another what a third party said. Authenticated channels convince only the direct receiver: it learns who sent a message but cannot pass that proof on. A proposer that references another party's block therefore cannot prove to the voters who published that block, so the high-throughput signature-free protocols we survey first complete an availability vote or a reliable broadcast for every published data block; only then may a proposer reference it.

We present VANTAGE, a partially synchronous Byzantine fault-tolerant (BFT) protocol for $n \geq 3f+1$ parties, at most f of them Byzantine, that uses only authenticated channels and collision-resistant hashing. Parties publish blocks on hash-linked author lanes; each view's proposer pairs a *core manifest* of lane prefixes that a quorum has acknowledged to it with an optimistic *tip manifest* of blocks it has just received. A new primitive, *Availability-Graded Broadcast* (AGB), makes the core irrevocable on a quorum of directly received acknowledgments while grading, rather than blocking on, the tip. A view that AGB leaves open or silent is decided by its own on-demand instance of information-theoretic agreement. A block may thus be referenced in a proposal one message delay after publication, as in signed optimistic designs. Let δ be the actual message delay after the Global Stabilization Time (GST). When all parties are correct, a proposal commits within 2δ of its send once all n parties acknowledge it and within 3δ on the ordinary quorum path, so a block published in time for the next proposal is sequenced within 3δ to 4δ of publication. Safety holds under asynchrony; liveness holds after GST. On an emulated ten-region wide-area network (WAN) with 100 parties, VANTAGE has the lowest median latency of the compared signed and signature-free protocols at every offered load up to its peak, and it sequences 250k 512-byte transactions per second at a median end-to-end latency below 500 ms.

CCS Concepts: • **Security and privacy** → **Distributed systems security**; • **Theory of computation** → *Distributed algorithms*.

Additional Key Words and Phrases: Byzantine fault tolerance, atomic broadcast, availability-graded broadcast, signature-free protocols, partial synchrony

1 INTRODUCTION

State-machine replication (SMR) lets a group of replicas maintain one transaction log even when some replicas are faulty. Modern high-throughput systems let every replica publish transaction blocks in parallel, rather than routing every transaction through one leader, and consensus orders short references to those blocks [6, 16, 20, 36].

When may a block be referenced? There are two basic choices. With *certified admission*, a proposer waits until a quorum has confirmed that it holds the block. This is robust, but the confirmation round must finish before ordering can begin, and because every reference is already confirmed, voters can treat the proposal as *indivisible*: one value, accepted or not as a whole. With *uncertified admission*, the next proposer references the block as soon as it receives it from the author. This removes that delay when all messages arrive, but if the proposal is still indivisible, a voter that missed one block can only wait for it, fetch it, or reject the proposal and every other block in it. Signed protocols can take the second choice because signatures make fetched data attributable; every high-throughput signature-free protocol we know takes the first. This paper shows that the second choice does not require signatures.

The cost of signatures. Votes, certificates, and blocks in signed BFT protocols carry signatures under quantum-vulnerable schemes such as Ed25519 and BLS, which fit in 48–96 bytes and, for BLS, aggregate to constant size. The post-quantum standards give up both: an ML-DSA-44 signature is 2,420 bytes and an SLH-DSA-128s signature is 7,856

Author's address: Nikita Polyanskii, IOTA Foundation, Germany, nikitapolysky@gmail.com.

bytes — $38\times$ and $123\times$ an Ed25519 signature [29, 30] — and neither standard aggregates, so with n parties of which up to f are Byzantine, a quorum certificate carries $n - f$ signatures: at $n = 100$ and $f = 33$ it alone carries 162–526 kB. Time adds to size: hash-based signing takes 0.3–1 s per SLH-DSA-128s signature on a recent x86 core, and the faster parameter sets pay with 17 kB signatures [28]. Replacing classical signatures with a post-quantum standard on the consensus critical path has been measured to roughly double commit latency at 50 nodes [45]. Both costs are structural and land on every vote, certificate, and block. Authenticated channels avoid them: a symmetric per-message tag is 16 bytes, cheap to compute, and needs no public-key infrastructure (PKI).

What signatures buy. Signatures provide transferable attribution: after receiving signed bytes from any party, a voter can verify who signed them. An authenticated channel tells a receiver who sent a message, but the receiver cannot pass that proof on, and a hash proves that fetched bytes match a reference, not who published them. Repair can therefore restore missing bytes but not authorship. Without transferable attribution, waiting can stall the view, repair supplies data but not evidence, and rejection discards otherwise available blocks, so the signature-free systems we survey certify each orderable block or wait for availability votes before proposing it [37, 45], forgoing the one-delay admission of optimistic signed designs. Those designs have their own bottleneck: in Autobahn’s optimistic-tip path, a replica that lacks a proposed tip’s data requests it from the leader and votes only after it arrives [20, §5.5.2 and Lemma 7], which is also how the Autobahn artifact we build on behaves; and BBChain votes on the leader’s proposal as one indivisible value [26]. In both, a view’s progress rests on one leader’s proposal arriving whole; Section 7.4 isolates what this relay costs under partial dissemination.

Our approach. Vantage admits another author’s block one message delay after publication without signatures and without first completing an availability vote or a broadcast for it — to our knowledge, the first high-throughput protocol to do so. The claim concerns the fault-free data path under the conditions of Section 3, not faulty-view or resolver latency. Each party publishes its blocks as a hash-linked *author lane*, and a view’s proposer names lane prefixes in two parts: a *core manifest* of prefixes that a quorum has already acknowledged to it directly, and an optimistic *tip manifest* of newer blocks received directly from their authors. Voters answer with graded acknowledgments: support for core and tip together, or for the core alone when a tip is still missing at the response deadline, so a block that some voters lack costs at most the optimistic part. The proposer never serves data as a voting precondition. A quorum of responses makes the core irrevocable, and a quorum with one grade decides the tip immediately. No transferable proof is involved: every count is of first-hand (directly received) acknowledgments. We call this two-level primitive *Availability-Graded Broadcast* (AGB). A view left open by mixed grades or silent by a faulty proposer is resolved by its own on-demand Information-Theoretic HotStuff (IT-HS) instance [3], which validates candidate outcomes against each receiver’s own recorded responses for that view. A silent view that no correct party echoed is skipped by a quorum of skip votes without the resolver.

Our contributions are:

- **Availability-Graded Broadcast (Section 5).** We define a broadcast primitive whose outcome has two levels: a quorum of first-hand acknowledgments makes the core irrevocable, and a quorum that grades the tip unanimously decides at once whether the tip is included or dropped. We prove, without timing assumptions, that correct parties agree on the core and on the tip decision, and that a correct proposer’s view terminates after GST. For a faulty proposer the primitive itself guarantees no common completion; the resolver below supplies it.

- **Signature-free SMR at $n \geq 3f + 1$ (Section 6).** We build VANTAGE on AGB over hash-linked author lanes, and resolve each view that AGB leaves open or silent with its own on-demand IT-HS instance [3]. Safety needs no timing assumptions: correct parties *seal* (commit) one common outcome per view, their output sequences are prefixes of one another, and every output block was published by its author and remains retrievable. After GST, every view is eventually sealed at all correct parties and every valid block of a correct author is eventually included.
- **One-delay admission and latency (Sections 3 and 5).** Let δ be the actual message delay after GST, and let all parties be correct. A block that another party published directly to the proposer may be referenced δ after publication; the proposal seals 2δ after its send when all n parties acknowledge both core and tip, or 3δ on the ordinary quorum path. A block published just in time for the next proposal (*aligned*) is therefore sequenced between 3δ and 4δ after publication once earlier views are output; admitting it through the core instead, after its acknowledgment quorum, costs 5δ to 6δ . The two-delay seal cannot be improved: with $f \geq 1$, no broadcast decides one message delay after its send, even in all-correct executions [2]. These bounds hold at any $n \geq 3f+1$; at $n = 3f+1$, the minimum for partially synchronous consensus [17], 3δ equals the lowest publication-to-sequencing latency of any protocol we compare under the same assumptions, signed ones included (Section 3).
- **Implementation and evaluation.** We implement VANTAGE, two Autobahn variants (seamless and optimistic all-to-all), and two Simple-IT variants (Bracha and optimistic reliable broadcast) in one Rust binary derived from the Autobahn artifact [20], so all five share one transport, batching, storage, client, and metrics stack; Bluestreak and Sailfish++ run from their own artifact [31]. With 100 parties on an emulated ten-region WAN, VANTAGE has the lowest median latency at every offered load up to its peak, 0.49 s at 250k offered tx/s against 0.54 s for Bluestreak and 0.64 s for optimistic Autobahn, and peaks above 260k tx/s; at low load its wire bytes per sequenced byte are one fourteenth of optimistic Autobahn’s, since it carries no per-lane availability proofs or quorum certificates (Section 7). When Byzantine authors publish blocks to only a few parties ($n = 20$), a VANTAGE proposer never relays data and its median latency stays at 0.46 s while optimistic Autobahn’s reaches 2.3 s (Section 7.4).

Roadmap. Section 2 walks through one view and its recovery cases; Section 3 compares the closest protocols under one latency model; Sections 4 to 6 give the model, the primitive, and the resolver with their guarantees; Section 7 evaluates the implementation; and Section 8 discusses limitations. Complete specifications, proofs, comparison and evaluation details, and extended related work are in the appendices.

2 OVERVIEW

Data path: author lanes. Every party continuously publishes its data blocks on its own *author lane*, a hash chain sent block by block to all parties (Figure 1). For each author, a party records the longest valid lane prefix it has received directly from that author. When a proposal later names a lane prefix, every party whose directly received prefix covers it *acknowledges* it in its response to the proposal. Because channels are authenticated, direct receipt tells the receiver who published the block, and $f+1$ matching acknowledgments tell any party that counts them that at least one correct party received the prefix from its author and retains it; a quorum of $n-f$ acknowledgments guarantees $f+1$ correct holders. No block waits for a broadcast or a certification round before it may be proposed.

One view. Views have round-robin proposers. The proposer of view v broadcasts one *view proposal* $B_v = (C_v, T_v)$ made of two *manifests*, each naming at most one lane prefix per author. The *core manifest* C_v names prefixes that the proposer holds directly from their authors and for which it has itself counted a quorum ($n-f$) of acknowledgments,

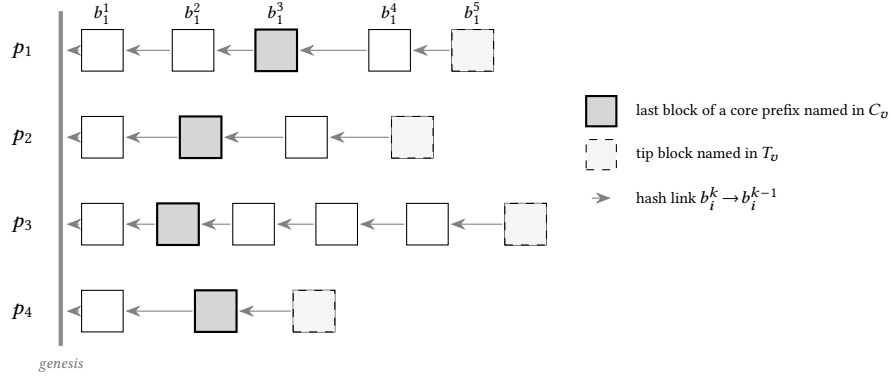


Fig. 1. Each party p_i publishes an author lane $b_i^1, b_i^2, \dots$ from a shared genesis; lanes advance asynchronously to a ragged frontier. The snapshot is from the point of view of Proposer(v), whose view proposal $B_v = (C_v, T_v)$ names at most one prefix per author in each manifest: the core manifest C_v (gray), whose entries a quorum has acknowledged, and the optimistic tip manifest T_v (dashed), newer blocks received directly from their authors.

given in the responses to earlier proposals that named those prefixes. The optimistic *tip manifest* T_v names newer blocks that the proposer received directly from their authors, each extending its author's core entry (or genesis when the author has none).

This per-view broadcast and its graded responses are VANTAGE's instance of *Availability-Graded Broadcast* (AGB), the primitive of Section 5. Every party answers in Bracha's two-round ECHO/READY pattern [9] (Figure 3):

- an *ECHO* of B_v , with grade 1 if the party can verify core and tip — each named prefix was received directly from its author or acknowledged by $f+1$ parties, and every tip block is held — and grade 0 if it can verify the core but not the tip by a local deadline, because the tip is missing, conflicts with what the party received from that author, or does not extend the core. A failed tip lowers the grade instead of blocking the view. The ECHO also carries the party's acknowledgments of the named prefixes it holds directly from their authors; these feed later core manifests;
- a *READY* for B_v once a quorum of ECHOs names B_v , with grade 1 or 0 if a quorum of those ECHOs share that grade, and the mixed grade otherwise. A mixed READY is provisional: if one grade reaches a quorum before the party's READY deadline, the party sends one single-grade refinement for the same B_v .

A quorum of READYS for B_v , grades ignored, *completes* the view: by quorum intersection, B_v is the only proposal that can complete in view v , and its core becomes an irrevocable part of the view's output. A single-grade READY quorum also *seals* the view (a *direct seal*): grade 1 selects the full outcome commit-full, core and tip; grade 0 selects the core-only outcome commit-core. We call the primitive alone — proposal, ECHO, READY, completion, and direct seals — *Direct AGB*; the rest of this section is Vantage's composition around it. The first addition is the *fast seal*: grade-1 ECHOs for B_v from all n parties seal commit-full one message delay before any READY quorum can form. When all parties are correct and the network is timely, a proposal therefore seals within 2δ of its send, where δ is the actual message delay, and within 3δ on the READY path (Figure 2). Views are pipelined: the proposer of view $v+1$ sends B_{v+1} as soon as it has processed B_v , without waiting for view v 's responses.

Recovery: open and silent views. Direct AGB leaves two cases unsealed. If READY grades stay mixed, a completed view is *open*: its core is fixed, but its tip is neither included nor discarded. If the proposer is faulty, a view may be *silent*:

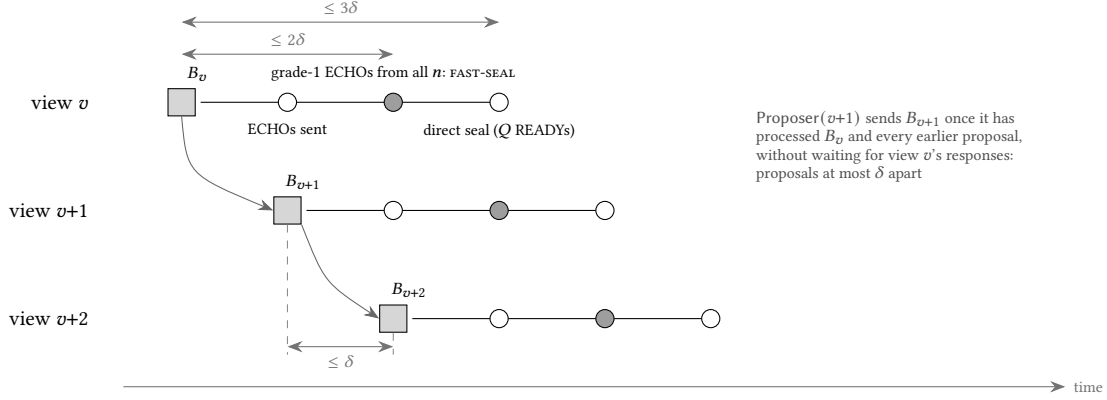
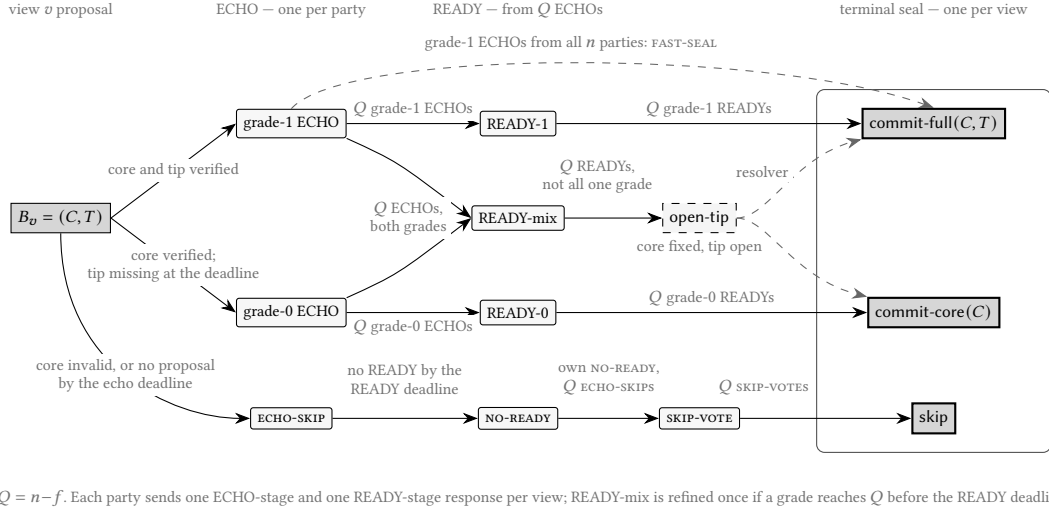


Fig. 2. Pipelined Vantage views in the all-correct case. Grade-1 ECHOs from all n parties arrive everywhere within two delays of the proposal's send and fast-seal commit-full; the direct seal from Q READYs follows within three. The proposer of $v+1$ sends B_{v+1} once it has processed B_v and every earlier proposal, without waiting for view v 's ECHOs or READYs, so consecutive proposals are at most one delay apart; formal view entry through the WISH pacemaker (Section 5) arms the fallback timers and triggers the send only when an earlier proposal has not arrived.



$Q = n - f$. Each party sends one ECHO-stage and one READY-stage response per view; READY-mix is refined once if a grade reaches Q before the READY deadline.

Fig. 3. One Vantage view v . Left to right: the proposal, each party's ECHO-stage response, its READY-stage response formed from Q ECHOs, and the terminal seal, at most one per view. Any READY quorum completes the view and fixes the core; a single-grade quorum seals it commit-full or commit-core, and a quorum without a single grade leaves the tip open (open-tip) for the resolver (Figure 4). Two branches are Vantage shortcuts around Direct AGB: grade-1 ECHOs from all n parties fast-seal commit-full one message delay before READYs, and a quorum of grounded skip votes seals a crash-only silent view as skip. Appendix B gives the complete rules.

parties time out and send refusals, ECHO-SKIP in place of an ECHO and NO-READY in place of a READY. Vantage seals both cases with two mechanisms, again without transferable evidence.

The first is a shortcut for a view whose proposer merely crashed. A party that has itself sent NO-READY and itself counted a quorum of ECHO-SKIPS broadcasts a SKIP-VOTE; we call such a vote *grounded*, since it rests only on the party’s own evidence. A quorum of grounded votes *skip-seals* the view as skip (Figure 3). A skip vote is a persistent stance: a party that has voted to skip never endorses a non-skip outcome for that view in the resolver below, and vice versa, so a skip quorum and a conflicting resolver quorum cannot both form.

Every other unsealed view u — a resolver *target* — is decided by its own on-demand instance of Information-Theoretic HotStuff (IT-HS) [3], started once the data path has advanced three views past u (Figure 4). Candidates are commit-full(C, T), commit-core(C), and skip, each proposed only by a party whose own recorded responses for u justify it. A party endorses (ECHOes) a candidate only if it is consistent with what it itself sent in view u : a party that sent a grade-1 READY never endorses commit-core, one that sent grade 0 never endorses commit-full, and only a party that sent NO-READY endorses skip. Any READY quorum that could have direct-sealed u shares a correct party with any resolver ECHO quorum, and that party refuses one of the two; a candidate that conflicts with a possible direct seal therefore never gathers a quorum. This local check stands in for the certificate a signed protocol would attach. Because it is local, a party that joins the resolver late cannot repeat it, so before IT-HS locks on a candidate or carries it into a later resolver view, a quorum of parties must have witnessed its endorsement quorum first-hand (*Backed*, Section 6). Targets run concurrently with one another and with the data path, and later proposals carry no resolution state.

The fast seal needs the same kind of guard. A party that counts grade-1 ECHOes from all n parties seals commit-full without waiting for READYS, so every party that sends a grade-1 ECHO installs an *optimistic lock*: it refuses every conflicting resolver candidate until it has counted $f+1$ echo-stage responses that are not grade-1 ECHOes for B_v . One such response would not do, because a Byzantine party may report different grades to different parties; among $f+1$ there is a correct party, whose single ECHO is the same everywhere, so no party can have counted all n grade-1 ECHOes.

Sequencing and output. A party *seals* each view at most once, with whichever result arrives first — a direct, fast, or skip seal or a resolver decision; Section 6 shows that they never disagree. Sealed views are *sequenced* in view order: once all earlier views are sealed, a completed view’s core blocks are appended — even while its own tip is still open — and its tip blocks follow if the view seals commit-full. *Output* is local delivery after retrieval: a party that lacks a sequenced block requests it from the other parties, and every sealed manifest is backed by first-hand acknowledgments from a correct party that retains its blocks (Section 4). We say that a resolver *decides* a target and that a party *seals* a view.

3 CLOSEST COMPARISONS

Section 2 gave Vantage’s fault-free data-path latencies: a tip is admitted one message delay after publication, and the proposal seals 2δ after its send on the fast path or 3δ on the READY path. This section places those numbers beside the closest protocols under one set of assumptions. The outcome is that Vantage’s tip path reaches the lowest publication-to-sequencing latency in the comparison, signed protocols included, while the signature-free baselines spend two delays before a block can be proposed at all.

We measure from a correct nonleader publishing a block to a different correct party proposing the object that orders it — a consensus proposal or a directed-acyclic-graph (DAG) vertex — which we call the block’s *carrier*; in VANTAGE, the carrier names a lane prefix that contains the block. We write A for publication-to-carrier latency (*admission*) and L for publication-to-sequencing latency. Every row uses the same favorable setting: all parties are correct after GST, there is no queueing, earlier output is complete, and publication is *aligned* with the next carrier opportunity, so A and L exclude any wait for the protocol’s cadence. For Vantage this is the steady state of Corollary B.9: every earlier view is output

Table 1. Aligned cross-author data latency in the common setting of Section 3. A is publication-to-carrier (admission) latency and L is publication-to-sequencing latency; lower is better. Starred values need responses from all n parties.

publish $\xrightarrow{A}$ first legal carrier $\rightarrow$ sequenced at L				
Protocol / path	Signature-free	Required before carrier	Admission A	Aligned total L
Mempool-Simple-IT (Opt-RBC) [45]	Yes	Availability-vote quorum	2δ	5δ
Sailfish++ (Opt-RBC) [37, 45]	Yes	Opt-RBC for every vertex	2δ	5δ for $n-f-1$ vertices 7δ for the remaining f
VANTAGE (optimistic lane tip)	Yes	Directly published lane tip; no PoA	δ	$3\delta^*$ fast (all-to-all) 4δ quorum (READY)
FinWhale (Mysticeti + fast path) [6, 23]	No	Direct receipt of signed DAG block	δ	3δ fast 4δ quorum
Autobahn (optimistic lane tip) [20]	No	Current lane tip; parent has PoA	δ	$3\delta^*$ fast (all-to-all) $4\delta^*$ fast (linear) 6δ quorum (slow path)
BBCA-Chain [26]	No	One-trip BEB block	δ	4δ

and each proposal is sent as soon as its tip is eligible. Here δ is the actual post-GST message delay. “Signature-free” means no digital signatures or transferable signature certificates; every row still assumes authenticated channels.

Table 1 has two groups. Mempool-Simple-IT, in its faster Opt-RBC variant, and Sailfish++ are the signature-free protocols whose latencies [45] derives in the same message-delay terms; both certify a block, by an availability-vote quorum or by optimistic reliable broadcast (Opt-RBC), before a carrier may reference it. FinWhale (Mysticeti with a fast path), Autobahn, and BBCA-Chain are signed protocols whose carrier may reference a block one delay after publication — a directly received DAG block, an optimistic lane tip, or a one-trip best-effort-broadcast (BEB) block — which is the admission latency Vantage claims without signatures. Autobahn requires a proof of availability (PoA) for the tip’s parent; BBCA denotes Byzantine Broadcast with Complete–Adopt.

Starred values are zero-fault fast paths that need responses from all n parties; one non-voter disables them. FinWhale’s fast path is not starred: it commits on $n - p$ references with $n = 3f + 2p - 1$, so at the displayed $p = 1$ it runs at $n = 3f + 1$ and survives one non-voter [23]. Sailfish++ sequences $n - f - 1$ of each round’s nonleader vertices in 5δ and the remaining f two delays later [37, 45]. Derivations and protocol-by-protocol qualifications appear in Appendix A; the table compares latency under these assumptions, not fault tolerance or overall dominance.

The highlighted row is the lane-tip path, the one-delay admission of Section 1. The core is the other manifest in the same proposal: a core entry first waits for its acknowledgment quorum, so it sequences at 5δ aligned through the fast seal (Corollary B.9). The tip row’s $3\delta^*$ assumes that the carrier is sent as soon as the tip is eligible and that all n parties respond; the Direct AGB READY path gives 4δ . The two paths race at run time and a party seals with whichever completes first: the starred value waits for the slowest of all n responses, the READY path for two successive quorum steps, so under heterogeneous delays the nominally slower path can finish first (Section 7.5 measures the split on the

emulated WAN). Without alignment, a block that misses the current proposal waits at most one delay for the next, so the tip bound is 4δ through the fast seal (Corollary B.9); on the READY path each of these figures is one delay larger.

The signature-free baselines spend two delays before a carrier: an availability-vote quorum in Mempool-Simple-IT and Opt-RBC for each Sailfish++ vertex. Vantage instead admits a directly published lane tip after one delay, and a receiver that lacks the tip lowers its grade instead of withholding its ECHO. The certified baselines buy stronger pre-carrier availability with those delays: a Vantage tip manifest may remain open and use the resolver (Section 6).

The three signed rows pay differently. Autobahn also admits a current lane tip after one delay but requires a PoA for its parent and signed votes; its displayed 3δ endpoint, like Vantage’s, needs all n participants, and its two-phase slow path takes 6δ . FinWhale’s $p = 1$ fast path reaches the same 3δ and survives one non-voter, using signed DAG blocks. BBKA-Chain’s carrier is a one-trip BEB block followed by a three-trip BBKA decision. At $n = 3f + 1$, Vantage therefore *matches* the lowest aligned endpoint in the comparison without signatures; it does not strictly dominate these protocols or prove a lower bound.

The admission rule also shapes leader load. A VANTAGE proposal names lane prefixes by reference; an under-disseminated tip costs its grade while the core is retained, and missing bytes are repaired from lane authors and holders, so no party’s response waits on data supplied by the proposer. When a leader instead admits data that lagging replicas must fetch from it before voting, the missing-data fanout concentrates on that leader (optimistic all-to-all Autobahn); when the proposal is voted as one indivisible value, a single under-disseminated block stalls or forfeits the whole view (BBKA-Chain). Section 7.4 measures the relay cost under a partial-dissemination stress.

4 MODEL AND PRELIMINARIES

The bounds above are proved in the model fixed here. Section 5 then specifies Direct AGB and its guarantees, and Section 6 the resolver and output rules that complete Vantage; the proofs are in Appendices B and C.

Parties and faults. The protocol runs among a fixed set $\Pi = \{p_1, \dots, p_n\}$ of $n \geq 3f+1$ parties. A party is *correct* if it follows the protocol and *Byzantine* otherwise. A static, computationally bounded adversary chooses before the execution a fixed set of $f_{\text{act}} \leq f$ parties, which may deviate arbitrarily under its coordination. Thus a correct party is one that is never corrupted during the execution. As $n \geq 3f+1$, at least $n - f \geq 2f+1$ parties are correct. Agreement uses only that at most f parties are ever faulty. The static choice is used where $f+1$ matching first-hand statements certify that one correct party holds a block — availability witnesses and resolver origin bits — because retrievability and repair need that party to remain correct and keep serving the block. Core entries, backed by $n - f$ acknowledgments, keep $n - 2f \geq f+1$ correct holders under any f corruptions; the $f+1$ threshold does not, so an adversary that corrupts a holder after it acknowledged could remove the only correct copy. We do not treat that case.

Communication. Every pair of parties shares a reliable, authenticated point-to-point channel: the recipient of a message learns the identity of its sender, the adversary can neither forge nor tamper with messages between correct parties, and every message sent between correct parties is eventually delivered. Channels are not assumed to be first-in, first-out (FIFO). Authentication is *not transferable*: that q received m on its channel from p is not evidence q can relay to a third party. This is the relevant difference from the signature-based setting, and the reason non-equivocation must be re-established through quorums. A concrete link may use a pairwise message-authentication code (MAC), but protocol messages contain no transferable vector of recipient-specific MACs: forwarded authentication tags are never accepted or counted as evidence. A broadcast includes its sender: a party delivers its own message to itself immediately and counts it as a first-hand message from itself.

Timing. We assume partial synchrony [17]: there is an unknown Global Stabilization Time (GST) and a known bound Δ such that every message between correct parties sent at time t arrives by $\max(t, \text{GST}) + \Delta$. Safety never rests on timing; liveness is guaranteed after GST. For a particular execution, let $\delta \leq \Delta$ denote its actual upper bound after GST: every correct-to-correct message sent at time t arrives by $\max(t, \text{GST}) + \delta$. The protocol knows Δ but not δ ; latency statements use δ to state the execution’s actual delay. This is an explicit *in-flight-message convention*: even a correct-to-correct message sent before GST is delivered by $\text{GST} + \delta$ in the execution-specific analysis. Local clocks are monotone throughout the execution and measure elapsed real time without drift after GST; they need not have a common origin. Thus a timer armed for duration θ before GST has at most θ local time remaining at GST and expires within at most θ real time thereafter. At a local deadline, a party processes every message delivered by that deadline, and evaluates the positive rules those messages enable, before taking the timeout rule. The protocol uses the constants

$$\theta_E = 3\Delta, \quad \theta_R = 4\Delta$$

for a missing echo-stage response and a missing initial ready-stage response — the smallest values for which the derivations in Theorem B.8 and lemma B.6 hold as written, tight when $\delta = \Delta$; they are not claimed to be protocol lower bounds. These constants affect only timer-triggered responses; positive evidence fires a rule as soon as its local condition holds.

Execution fairness. Liveness is stated for fair, non-Zeno executions. Every finite real-time interval contains only finitely many protocol events; local time and armed timers at correct parties advance; every action that remains enabled at a correct party is eventually taken; and a correct holder eventually serves a request issued by a correct party. These assumptions rule out an execution that performs infinitely many view transitions in finite time or permanently starves an enabled local action. They impose no delivery deadline before GST.

Cryptography. Beyond the ideal authenticated-channel abstraction, the protocol’s sole cryptographic primitive is a collision-resistant hash function $\mathcal{H}: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ with security parameter λ . For a post-quantum instantiation, collision resistance is required against quantum polynomial-time adversaries. The protocol uses no digital signatures, public-key infrastructure, or threshold cryptography. Relative to the ideal channel abstraction, all guarantees hold except with negligible probability in λ , with a hash collision as the only protocol-level computational failure event. A concrete MAC-based channel implementation additionally relies on pairwise-MAC unforgeability; a post-quantum implementation requires that property against quantum polynomial-time adversaries as well. We treat $\mathcal{H}(x)$ as a succinct, collision-free reference to x .

Instance and wire encoding. An execution fixes a session identifier sid , which binds the membership, fault threshold, and application epoch; a maximum data payload length $\ell_{\max}$; and a deterministic, state-independent validity predicate BlockOK for data blocks (state-dependent transaction checks are performed later by deterministic execution). Every protocol message carries sid ; a party rejects a message from another session before storing or counting it. All messages use one canonical, prefix-free byte encoding enc with author-indexed vectors in increasing author order, and every hash input begins with a distinct fixed domain tag and sid (for example, data-block or view-proposal), so objects of different types cannot share an encoding. A message with a noncanonical encoding, more entries than its type’s cap (Appendix B), or a data payload longer than $\ell_{\max}$ is malformed and is never counted.

Quorums. A *quorum* is any set of $n-f$ parties. We use repeatedly that, for $n \geq 3f+1$:

- (i) every quorum contains at least $n - 2f \geq f+1$ correct parties, and the correct parties always form a quorum; and

- (ii) any two quorums Q_1, Q_2 satisfy $|Q_1 \cap Q_2| \geq 2(n-f) - n = n - 2f \geq f+1$, hence share at least one correct party.

5 AVAILABILITY-GRADED BROADCAST

Section 2 introduced the view proposal $B_v = (C_v, T_v)$ and its graded responses (Figure 3). This section states the rules of Direct AGB precisely and separates them from Vantage’s proposer policy and shortcuts. For a correct proposer, each core entry carries $n-f$ exact-position acknowledgments (R2 below); the tip manifest is not certified before proposal. Every manifest is sent by value in the proved protocol. Full lane syntax, publication, acknowledgment, repair, and the value-domain hooks appear in Appendix B.

Direct AGB separates *completion* from *sealing*. Completion fixes one proposal and makes its core irrevocable. A homogeneous grade-1 READY quorum seals $\text{commit-full}(C_v, T_v)$, a homogeneous grade-0 quorum seals $\text{commit-core}(C_v)$, and a residual mixed quorum leaves $\text{open-tip}(C_v, T_v)$ for the Vantage composition. Direct AGB never outputs skip and does not promise totality for a Byzantine proposer. It is not Feldman–Micali gradecast [18]: gradecast grades a receiver’s confidence in one value, whereas AGB fixes a common core and grades only the inclusion of an optional suffix.

5.1 Compact protocol

All statements are broadcast over authenticated channels and counted only when received directly from distinct authors. Let $Q = n - f$. Each party sends one immutable echo-stage response and one initial ready-stage response per view; an initial READY-mix may refine once to a homogeneous READY for the same proposal. The complete event and direct result follow these four rules.

- R1. Propose.** The designated proposer sends one well-formed (C, T) . A correct Vantage proposer forms C from directly published lane frontiers with Q exact-position ECHO acknowledgments and forms T from directly published extensions of those frontiers.
- R2. Echo.** A receiver fixes the first proposal received directly from the proposer. After the proposal becomes active through formal entry or the responsive frontier, the receiver immediately ECHOs grade 1 if core and tip pass their local checks (Figure 3, left). If it has not sent that ECHO by its bounded echo fallback deadline, it sends grade 0 when the core passes, or `ECHO-SKIP` otherwise. If no active well-formed proposal exists by the absolute deadline $e_i(v) + 3\Delta$, it sends `ECHO-SKIP`. A proposal ECHO also carries availability claims outside its immutable identity. An exact-position bit acknowledges the named direct prefix; after chain verification the claim also covers its ancestors. A sparse integer $d > 0$ acknowledges the directly published prefix ending d heights behind, after the anchored ancestry is verified. Repair can provide bytes but cannot create direct provenance or acknowledgments.
- R3. Ready.** On Q ECHOs for one proposal, a party broadcasts READY-1 or READY-0 if that grade already has Q ECHOs, and READY-mix otherwise. A provisional mix refines once if one grade reaches Q before all ECHO responses or the deadline; otherwise the refinement window closes. If the party has sent no READY by $e_i(v) + 4\Delta$, it emits `NO-READY`.
- R4. Complete and direct-seal.** Any Q READYs naming one proposal, grades ignored and each author counted once, complete it. A homogeneous Q additionally emits the corresponding full or core direct result (Figure 3, right). If the completing state has neither homogeneous quorum, the Vantage caller quarantines its carried non-exact-quorum tip witnesses. Late READYs continue to be counted after completion.

Two further rules of a correct Vantage proposer are policy rather than part of the primitive, and no receiver checks them. If a direct extension has a general but not exact acknowledgment quorum, T names the least-height such extension first, so that a stable exact-position census forms. Completed-open quarantine drops an author whose carried tip never gathered Q exact acknowledgments from ordinary later tip manifests, never from the core, and retries at most one quarantined author per proposal through T on an exponential proposer-turn schedule (Section 8).

The WISH pacemaker is the only formal entry mechanism. A WISH message carries a view high-watermark. A party relays that high-watermark after first-hand support from $f + 1$ authors and, after support from Q authors, enters each missing view up to that high-watermark. Immediately before sending either its echo-stage response for view $v - 1$ or its initial ready-stage response for view $v - 2$, a party checks whether that send completes the pair — its echo-stage response for view $v - 1$ together with its initial ready-stage response for view $v - 2$ — and if so raises its own WISH high-watermark through $v + 1$. A separate responsive frontier activates consecutive proposals as soon as they arrive; it is scheduling state, never counted evidence. After GST, correct entries to a view lie within 2δ , within δ when all parties are correct, and one view can delay advancement by at most $4\Delta + \delta$. The complete pacemaker and timer proof are in Appendix B.3.

Vantage adds one shortcut outside Direct AGB. Before sending a grade-1 ECHO for a proposal $B = (C, T)$, a party installs an optimistic lock on that full payload. The lock releases permanently after $f + 1$ first-hand nonmatching echo-stage responses. Counting grade-1 ECHOs from all n parties submits $\text{commit-full}(C, T)$ to Vantage’s TRY-SEAL arbiter without creating completion, a direct result, or a resolution witness. The ordinary Direct AGB READY path continues in parallel (Figure 2); Section 7.5 measures which path finishes first on the emulated WAN.

5.2 Guarantees

The *pacemaker cutoff* below is the finite view after which the pacemaker supplies the entry and proposal-timing premises of conditional termination.

THEOREM 5.1 (DIRECT AGB AND FAST-SEAL SUMMARY). *Without timing assumptions, no two correct parties complete different proposals in one view; every direct result is compatible with the unique completion, and a completed core is valid, author-backed, and permanently retrievable. Every correct-proposer view past the pacemaker cutoff completes at all correct parties when its persistent core checks hold. In an all-correct responsive suffix, a prefix-extending proposal fast-seals full within 2δ of its send; the Direct AGB READY path seals full within 3δ .*

The safety proof is quorum intersection over first-hand, one-shot ECHOs and READYs. A READY quorum contains a correct sender backed by an ECHO quorum; two such quorums intersect in a correct immutable response. For the Vantage-only shortcut, the all- n grade-1 ECHO set fixes every correct ECHO, while the optimistic locks exclude a conflicting resolution-bearing proposal’s ECHO quorum in either temporal order. The full safety, conditional-termination, and lock-compatibility proofs are in Appendix B.5 and lemma D.2.

For a block aligned with the next proposal opportunity, tip admission costs δ and the fast seal costs another 2δ , giving the 3δ endpoint in Table 1; the READY endpoint is 4δ . A core entry needs an anchoring proposal and its ECHO census before a later proposal can carry that prefix in C . Its aligned endpoint is therefore 5δ through fast seal, or 6δ when that later proposal follows the READY path. With arbitrary arrival phase, the proved tip endpoints are 4δ in the worst phase and 3.5δ in expectation under the uniform-phase convention; the corresponding core bounds are 7δ and 6.5δ . These latencies do not bound mixed or Byzantine resolution. Direct AGB uses $O(n^2)$ logical messages per view

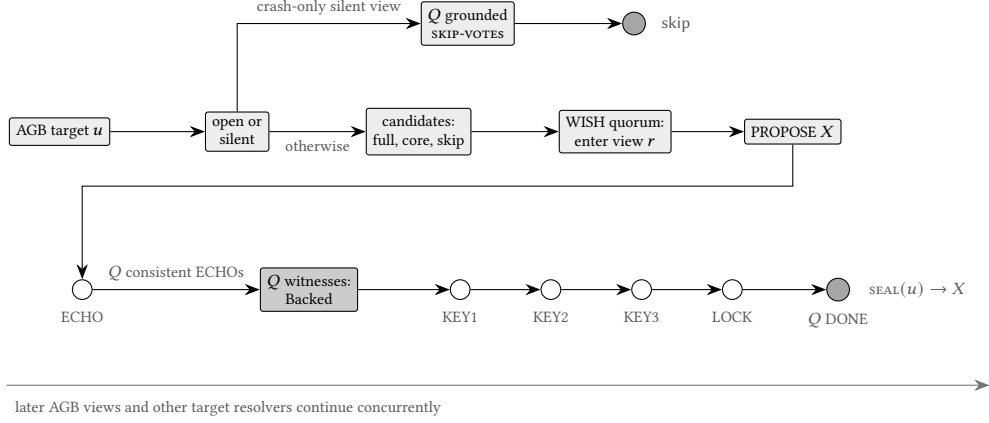


Fig. 4. Resolution mechanisms for a silent or open target. A clean crash-only view can skip after grounded post-ready votes. Otherwise the target’s own WISH/IT-HS instance turns a locally checked fresh value into a stable backed value before KEY1 and decides it after the remaining IT-HS phases. Other target resolvers and later data-plane views continue in parallel, but ordered output cannot cross an unresolved cursor position.

— at most $3n^2 + O(n)$ with READY refinements — and the proved by-value protocol uses $O(n^3\lambda)$ bits, excluding lane publication, repair, and the resolution plane.

6 VANTAGE RESOLUTION AND OUTPUT

Direct AGB promises no common outcome for a faulty proposer (Section 5): a completed view can remain open when its READY grades do not become homogeneous, and a silent or partially disseminated view may not complete at all. This section makes the recovery mechanisms sketched in Section 2 precise (Figure 4). The first-result-wins rule of Section 2 is the TRY-SEAL arbiter, which combines four compatible submissions: a Direct AGB full/core result, the all- n fast-seal full result, a grounded skip result, or a target-local resolver decision. The arbiter stores the first submission and treats later compatible submissions as idempotent. It is Vantage composition state, not a second Direct AGB input. Different unresolved views run independent resolver instances, so no global fallback height or later AGB proposal serializes them.

The reusable resolver contract is per target: all correct calls choose one exact outcome; a full, core, or skip call permanently excludes every future incompatible READY quorum; a non-skip call supplies author-backed manifests; and every correct party eventually obtains the chosen result. Direct AGB composed with any service satisfying these properties has one compatible terminal outcome, and a completed view can never resolve as skip. Vantage realizes the contract with the guards and agreement instance below; the formal resolver properties and generic composition theorem are in Appendix C.1.

The local data-plane proposal horizon is the larger of the next responsive proposal opportunity and the WISH high-watermark. Once it reaches $u + 3$, a party scans its first-hand target state. A candidate needs $Q = n - f$ ready-stage responses. A full candidate additionally needs $f + 1$ matching grade-1 ECHOs; a core candidate needs Q initial ready-stage responses whose first counted form is not grade 1 and $f + 1$ matching ECHOs of either grade; skip needs Q NO-READY responses. These finite candidate sets only grow. A nonempty set at an unsealed party starts a typed per-target RESOLUTION-WISH exchange: $f + 1$ high-watermarks amplify the own WISH and Q enter the resolver view; its round-robin primary proposes on Q accepted IT-HS key suggestions (Section 6.1). An initial WISH is retransmitted

until entry. Without local evidence, remote traffic from fewer than $f + 1$ parties cannot allocate a target instance. A party that already holds a compatible terminal seal nevertheless remains a participant: one peer WISH activates it and makes it broadcast its own initial WISH. It fresh-ECHOs only a resolver value whose submitted outcome equals that seal. This prevents a selectively delivered local seal from removing a correct party needed by the others.

Every target has a persistent stance $z_i(u) \in \{\text{free, non-skip, skip-voted}\}$. Before a fresh non-skip resolver ECHO, a correct party checks its final target echo- and ready-stage history, required data, and active optimistic lock, then atomically claims the non-skip stance. Before sending SKIP-VOTE, it instead claims skip-voted; the sends are mutually exclusive in either order. The fresh ECHO carries origin 1 for full only after a matching grade-1 target ECHO, and for core after a matching target ECHO of either grade. A fresh non-skip ECHO quorum is usable only with at least $f + 1$ origin-1 senders, preserving publication provenance. Every resolver ECHO also binds the exact proposal key q : $q = 0$ denotes fresh validation and $q > 0$ denotes a Backed value carried from an earlier resolver view. ECHO quorums match (u, r, q, x) , so fresh and carried ECHOs for the same value cannot be merged.

After a usable fresh ECHO quorum, a correct party broadcasts one first-hand witness for that target, resolver view, and value. It relays its own witness after $f + 1$ matching witnesses; Q make the value *Backed*. The causally first correct witness must follow a genuine fresh ECHO quorum, while the relay makes Backed eventually common. A WISH sender also receives the responder's current own WISH and, once per coordinate, its applicable retained resolver sends through the requested view plus one, together with its DONE. This send-upon-join rule closes races in which phase traffic arrived before target activation. Only a Backed value may enter KEY1 or be re-proposed after a resolver view change (Figure 4). Full guards, the IT-HS ACCEPTKEY/OPENLOCK rules, and the proof are in Appendices C and D; Algorithm 1 lists the rules in event form.

6.1 Concurrent target-local resolver

Resolver views are separate from data-plane views. For target u , view 1's primary is the committee member immediately after $\text{Proposer}(u)$; later views continue round robin. A party acts on a resolver view's proposal, ECHO, KEY, and LOCK quorums only while that view is current (a quorum counted before entry fires on entry); witness, Backed, WISH, and DONE relay and decision are view-independent. On entry, parties send the primary their IT-HS KEY3 suggestions and broadcast KEY1 lock-opening proofs. A positive suggestion attaches its bounded value and must satisfy ACCEPTKEY and BACKED. The primary re-proposes a maximum positive key; when all accepted keys are zero, it advances a persistent target-local cursor and proposes the next locally justified candidate. A 5Δ timer changes view only if no proposal arrived, avoiding the full timeout for a silent primary; after a proposal, the 11Δ IT-HS view timer remains.

The BACKED step makes external validity stable. Checking only a local $f + 1$ subset would find a correct origin but would not make the value stable: the Byzantine senders among those witnesses may never repeat them, so a party that joins later could not reproduce the check. In contrast, a correct party that counts Q witnesses counted at least $n - 2f \geq f + 1$ correct senders. Their broadcasts make every correct party relay and eventually count Q . A carried value can therefore be validated from BACKED, rather than by rerunning a target-local predicate that may differ across parties. The IT-HS key and lock argument then gives agreement and view-change safety. Appendix D makes that argument self-contained: same-view uniqueness and persistent-lock safety prove agreement, while persistent KEY2 and KEY1 histories prove positive-key acceptance and lock opening.

Across R entered resolver views for one target, the protocol and at-most-once catch-up use $O(Rn^2)$ distinct logical messages and $O(Rn^3\lambda)$ bits in the proved by-value encoding; retransmitting the same initial WISH before entry adds one all-to-all send per timer expiration. The costs of k overlapping targets add. There is no global resolution queue, but

Algorithm 1: Compact Vantage composition at party p_i ; $Q = n - f$

```
/*  $v$ : data-plane view;  $u$ : unresolved target view;  $r$ : target-local resolver view;  $\mathcal{H}(X)$ : digest of a resolver value  $X$ ;  $Y$ : a submitted outcome;  $\text{BACKED}(u, X)$  abbreviates  $\text{BACKED}_i(u, \mathcal{H}(X))$ . */
upon genesis do
  | enter view 1 and broadcast  $\langle \text{WISH}, 2 \rangle$ ; every view then runs Direct AGB (R1–R4, Section 5) with the wrappers below
upon about to send the echo-stage response for view  $v - 1$  or the initial ready-stage response for view  $v - 2$ , whichever completes that pair do
  | raise the own WISH high-watermark through  $v + 1$  and piggyback it
upon  $f + 1$  direct WISH high-watermarks support  $x$  above the own wish do
  | persist and broadcast the own WISH high-watermark through  $x$ 
upon  $Q$  direct WISH high-watermarks support  $x$  do
  | invoke  $\text{ENTER}(v)$  for every locally missing  $v \leq x$ , in order
upon about to send a grade-1 ECHO for proposal  $B$  in view  $v$  do
  | install the optimistic lock on  $B$ ; it releases once  $f + 1$  counted echo-stage responses for  $v$  are not grade-1 ECHOs for  $B$ 
upon Direct AGB emits  $\text{COMPLETE}(v) \rightarrow (C_v, T_v)$  do
  | store the irrevocable core; run  $\text{OUTPUTCURSOR}$ 
upon Direct AGB emits  $\text{DIRECT-SEAL}(v) \rightarrow Y$ , grade-1 ECHOs for one proposal  $B$  are counted from all  $n$  parties, or  $Q \langle \text{SKIP-VOTE}, v \rangle$  statements are counted do
  | submit  $Y$ , commit-full( $B$ ), or skip, respectively, to  $\text{TRY-SEAL}(v, \cdot)$ 
upon own NO-READY and  $Q$  direct ECHO-SKIP for target  $u$ , with  $z_i(u) = \text{free}$  and no non-skip seal do
  | atomically persist  $z_i(u) \leftarrow \text{skip-voted}$  and broadcast  $\langle \text{SKIP-VOTE}, u \rangle$ 
upon the proposal horizon reaches  $u + 3$  and local AGB evidence justifies a first candidate for unresolved  $u$  do
  | allocate the target instance; broadcast  $\langle \text{RESOLUTION-WISH}, u, 1 \rangle$  and retransmit it every  $5\Delta$  until entry
upon resolver WISHes for an unallocated target  $u$  arrive from  $f + 1$  parties, or one arrives while  $p_i$  holds a terminal seal for  $u$  do
  | allocate the instance; in the second case also broadcast  $\langle \text{RESOLUTION-WISH}, u, 1 \rangle$  /* the resolver rules below act only on allocated targets */
upon  $f + 1$  resolver WISHes support  $r$  above the own WISH do
  | raise and broadcast the own WISH through  $r$ 
upon a direct resolver WISH  $(u, r)$  from  $p_j$  do
  | send  $p_j$  the current own WISH; replay once each applicable own SUGGEST, PROOF, PROPOSE, phase statement, and witness through  $r + 1$ , plus the own DONE, if any
upon  $Q$  resolver WISHes support  $r$  do
  | enter each missing target-local view through  $r$ ; send the IT-HS suggestion/proof; arm the  $5\Delta$  no-proposal and  $11\Delta$  full-view timers
upon the no-proposal timer expires without a primary proposal, or the full-view timer expires before a decision do
  | raise and broadcast the own WISH through  $r + 1$ 
/* Resolver statements are counted through one view above the own WISH. The ECHO send and the ECHO, KEY, and LOCK quorum rules act only while their view is current; a quorum already counted at entry fires on entry. Witness, Backed, WISH, and DONE relay and decision are view-independent. */
upon the primary has  $Q$  accepted suggestions do
  | re-propose the value of the maximum positive key, or advance its persistent cursor and propose the next locally justified fresh candidate
upon the first proposal of the primary of  $(u, r)$ , for  $X$  with key  $q$  do
  | store it and stop the no-proposal timer; once  $r$  is current and the proposal is lock-compatible (rechecked as proofs, backing, target data, or the current view change):
  | if fresh ( $q = 0$ ), require the target-history/data/stance checks and attach the origin bit; if carried ( $q > 0$ ), require  $\text{BACKED}(u, X)$ ; broadcast the resolver ECHO for  $(u, r, q, \mathcal{H}(X))$ 
upon  $Q$  ECHOs matching  $(u, r, 0, \mathcal{H}(X))$ , and  $f + 1$  origin-1 ECHOs if  $X$  is non-skip do
  | broadcast  $\langle \text{RESOLUTION-WITNESS}, u, r, \mathcal{H}(X), X \rangle$ 
upon  $f + 1$  matching witnesses do
  | relay the own witness once
upon  $Q$  matching witnesses do
  | latch  $\text{BACKED}(u, X)$ 
upon  $Q$  ECHOs matching the stored proposal  $(u, r, q, \mathcal{H}(X))$ , and  $\text{BACKED}(u, X)$  if  $q = 0$  do
  | update the KEY1 state and broadcast KEY1
upon successive  $Q$  KEY1, KEY2, KEY3, and LOCK statements do
  | update the IT-HS keys/lock and emit the next phase, ending with DONE
upon  $f + 1$  matching DONEs do
  | relay the own DONE once
upon  $Q$  matching DONEs for  $X$  do
  | decide  $X$  for  $u$  and submit its outcome to  $\text{TRY-SEAL}(u, \cdot)$ 
function  $\text{TRY-SEAL}(u, Y)$ :
  | if  $u$  has no terminal outcome then
  | | persist  $Y$ , authorize its named lane roots, fire  $\text{SEAL}(u) \rightarrow Y$ , and run  $\text{OUTPUTCURSOR}$ 
procedure  $\text{OUTPUTCURSOR}()$ 
  | in view order: stop at a view that is neither completed nor sealed; at a completed or non-skip-sealed view, retrieve its lane prefixes and emit the core once; stop while the tip is open; after a full, core, or skip seal append the tip, omit it, or emit nothing, then advance; rerun when named lane prefixes arrive
```

we prove neither a bound on simultaneously active targets nor a constant general target latency; Section 7.6 measures the resolver under a sustained mixed-open attack, and Section 8 states what that leaves open.

THEOREM 6.1 (DIRECT-RESOLVER SUMMARY). *For each target, no two correct parties decide different resolver outcomes. If one correct party decides X , every correct party eventually either decides X or already holds its compatible terminal seal.*

Under partial synchrony, every unresolved target eventually obtains one common valid full, core, or skip outcome at all correct parties. Every non-skip decision is author-backed and future-compatible with Direct AGB.

6.2 End-to-end guarantees

THEOREM 6.2 (VANTAGE CORRECTNESS). *Safety holds without timing assumptions: every correct party seals each view at most once, no two correct parties seal different outcomes, and their output sequences are always prefix-comparable. Every output block is valid, published with its lane prefix by its encoded author to a correct party, permanently retrievable, and obtained before local output. Under partial synchrony, the fixed static fault set, fair repair, and the non-Zeno execution assumption, every view is eventually sealed, every finite common output prefix is eventually produced everywhere, and every valid block of a correct author is eventually included.*

The proof separates compatibility across sealing mechanisms from totality. Quorum intersection makes direct results agree; optimistic locks exclude conflicts between fast-seal and resolver submissions; persistent stances exclude a skip-vote quorum from any fresh ECHO quorum for a non-skip resolver value. Stable backing and IT-HS make each target’s resolver choice identical at all correct parties; targets need no relative order because the prefix cursor already processes data-plane views in order. For a clean crash-only silent view whose first correct entry is $s \geq GST$ and whose correct entries finish by $s + 2\delta$, all correct parties skip-seal by $s + 4\Delta + 3\delta$; aligned entry gives $4\Delta + \delta$. This is a per-view bound, not a bound for a burst of failures. Section 7.6 measures both a transient crash and the mixed-open case. Full statements and proofs appear in Appendix E.

7 IMPLEMENTATION AND EVALUATION

7.1 Implementation and methodology

We implement Vantage in Rust. The implementation is public at <https://github.com/polinikita/vantage-bft>, and the benchmark harness with its configurations and recorded campaigns at <https://github.com/polinikita/wan-bench>. The binary is derived from the Autobahn artifact [20]. The two Autobahn variants (optimistic all-to-all and seamless) extend that artifact’s own protocol implementation, and Vantage and the two Simple-IT variants (Bracha and optimistic reliable broadcast) are added as protocol modules, so all five share one Transmission Control Protocol (TCP) transport, batching, worker, storage, client, and metrics stack. A deterministic task handles the protocol while networking and storage run on separate tasks. Bluestreak and Sailfish++ run from the Starfish artifact [31] at <https://github.com/iotaedger/starfish>. All campaigns use canonical one-byte committee identifiers (compact IDs). A matched check at $n = 30$ finds that the inactive target-local resolver leaves throughput, latency, and CPU unchanged (Appendix F.4).

The shared transport realizes the model’s authenticated channels with a pairwise keyed-hash tag on each frame. Tags are verified on arrival and never become protocol evidence. The current implementation also checks that counted sender and proposer fields match the authenticated connection identity. Every evaluated workload emits identity-consistent fields, so this validation does not change the protocol path measured by the retained campaigns. The results include channel-authentication cost; Section 8 reports a separate cost check.

Unless stated otherwise, cloud runs place one c5d.2xlarge validator in the Amazon Web Services (AWS) eu-west-1a availability zone and inject the ten-region round-trip time (RTT) matrix of Table 2 over private addresses. The netem rules add delay only and cap no bandwidth, so the fleet keeps its single-zone datacenter bandwidth: the throughput results combine WAN latency with datacenter bandwidth. All shared-binary variants use the same fleet. Sailfish++ also

runs there; Bluestreak uses m5d.2xlarge because it exhausts memory on the shared instance type before reaching its highest accepted load (Appendix F.2). Q4 and Q5 are controlled native-Linux studies.

Transactions are random 512-byte values submitted open loop. Each cloud run warms up for 30 seconds and measures for 120 seconds. A transaction is *materialized* at a validator when its block is sequenced in that validator’s output and the block’s bytes are held locally. We report the median validator’s submission-to-materialization latency (end-to-end latency), central processing unit (CPU) use, and outbound framed bytes per sequenced byte. Cloud cells are medians across the available repetitions of up to three fresh-fleet campaigns. When measured throughput is within roughly one percent of offered load, we say that the protocol sustains the load: small over- and undershoots reflect work crossing the measurement-window boundary. The benchmarking harness, exact hardware, repetition counts, and metric definitions appear in Appendix F; the harness repository contains the corresponding configurations and records.

The implementation uses digest naming, positional acknowledgment bitmaps, and transport batching. These encoding choices do not change the protocol rules. The common encoding uses $O(n^2\lambda + n^3)$ control bits; sparse claims and by-value recovery remain within the proved $O(n^3\lambda)$ worst case (Appendix F.3).

Q1 (Section 7.2) and Q2 (Section 7.3) test the end-to-end claim of Section 1 — lowest median latency at every accepted load — across committee sizes at low load and under increasing load at $n = 100$; in both, no view uses a fallback decision. Q3 (Section 7.4) measures the leader-relay cost that Section 3 attributes to admitting data a leader must serve. Q4 (Section 7.5) follows one fault-free view against the $3\delta/4\delta$ endpoints of Table 1. Q5 (Section 7.6) asks whether the resolver of Section 6 keeps pace with crash and mixed-open faults.

7.2 Q1: Fault-free committee scaling

We run seven protocol variants at $n \in \{10, 20, 50, 100\}$ under a fixed aggregate 100 tx/s. All variants sustain approximately the offered load.

Vantage has the lowest p50 and p99 latency at every committee size (Table 3); its p50 stays near 0.45 s as the committee grows from 10 to 100. At $n = 100$ it is about 30 ms faster than Bluestreak at both p50 and p99, while using roughly three times its CPU and 1.4 times its wire cost. Against optimistic all-to-all Autobahn, Vantage is about 0.1 s faster, uses about half the CPU, and sends roughly one fourteenth as many bytes. The wire difference comes mainly from the per-lane availability proofs in each Autobahn consensus cut.

Progress cadence helps explain the latency ordering. At $n = 100$, Vantage advances through about 14 proposal views per second, compared with about 10 DAG rounds for Bluestreak, about four for Sailfish++, and about eight consensus slots for each Autobahn variant (Appendix F.4). These units are not equivalent work; they identify only the next opportunity for a fresh data reference to enter consensus. More frequent opportunities shorten that initial wait, while admission rules and the later commit path account for the remaining difference. The pipelined cadence is the run-time form of the one-delay proposal spacing in Figure 2. Summed over validators and repetitions, the all- n fast seal supplies 67–70% of Vantage’s local seals at $n \in \{10, 20, 50\}$ and 72–73% at $n = 100$; the rest are Direct AGB READY seals (Section 5).

7.3 Q2: Throughput scaling at $n = 100$

We increase offered load from 100 to 300,000 tx/s at $n = 100$. A point is accepted when median sequenced throughput reaches at least 95% of offered load; the first lower point is the overload endpoint, reported in the text and not plotted.

Vantage has the lowest p50 latency at every accepted offered rate, remaining between about 0.45 and 0.51 s. It sustains 250k offered tx/s at about 0.49 s p50; at its highest accepted point it sequences about 268k tx/s at 0.51 s before overloading

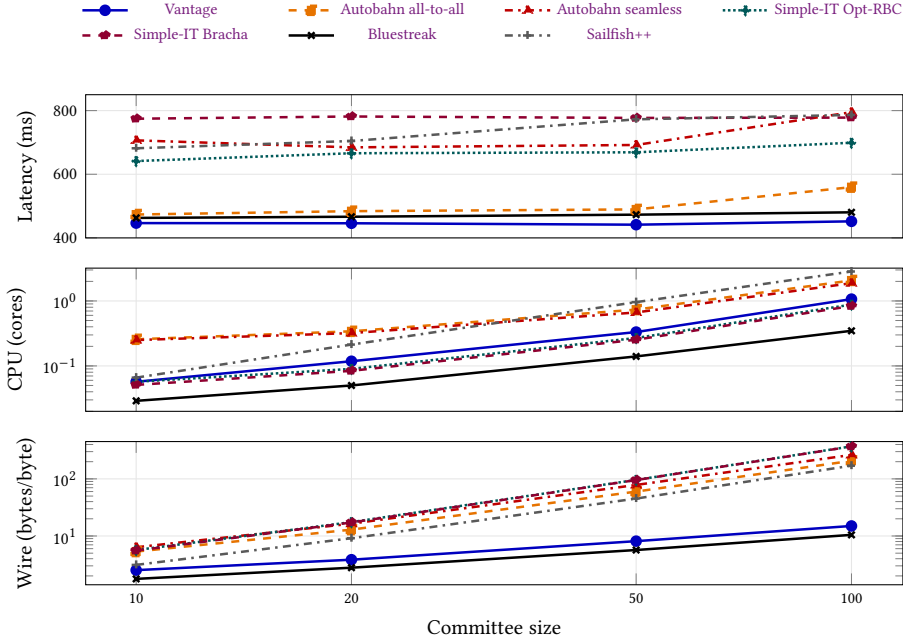


Fig. 5. Fault-free committee scaling at 100 transactions per second. Most cells are medians across three fresh-fleet repetitions; a few baseline cells use fewer. The $n=100$ cell joins from the throughput campaign’s 100 tx/s point, and instance types are listed in Appendix F.2. Wire cost is bytes sent per committed byte. The committee axis and the processor and wire panels are logarithmic.

at 300k. Optimistic Autobahn reaches a similar throughput, about 272k tx/s, but its p50 exceeds 1.1 s. Bluestreak reaches about 240k tx/s at 0.54 s before overloading, and Sailfish++ overloads at a lower offered load. Seamless Autobahn and both Simple-IT variants also accept the 275k point and overload at 300k. The shared-binary variants thus reach nearly the same accepted peak, 263–272k, and fail at the same offered load despite different consensus logic; we attribute this endpoint to the shared transport path on this instance type rather than to the protocols, so throughput parity among them is expected and latency is the discriminating measurement.

Resource cost separates the designs. Bluestreak uses the least CPU at every accepted point. It also sends the fewest bytes at low load; from 150k upward, the accepted variants converge toward one wire byte per committed byte, with only small differences among them.

The tail follows the same ordering. Vantage stays near 0.65 s p99 from 150k through 250k. Over the same interval Bluestreak rises from about 0.8 to 1.5 s, while optimistic Autobahn reaches about 3.3 s at its highest accepted point (Table 3). These are deployment measurements, not protocol capacity bounds.

Across the three repetitions, the all- n fast seal supplies 72–76% of Vantage’s local seals at every accepted load through 250k and 41–45% at the 275k point; the remainder are Direct AGB READY seals, and no view in these runs used a skip or resolver seal.

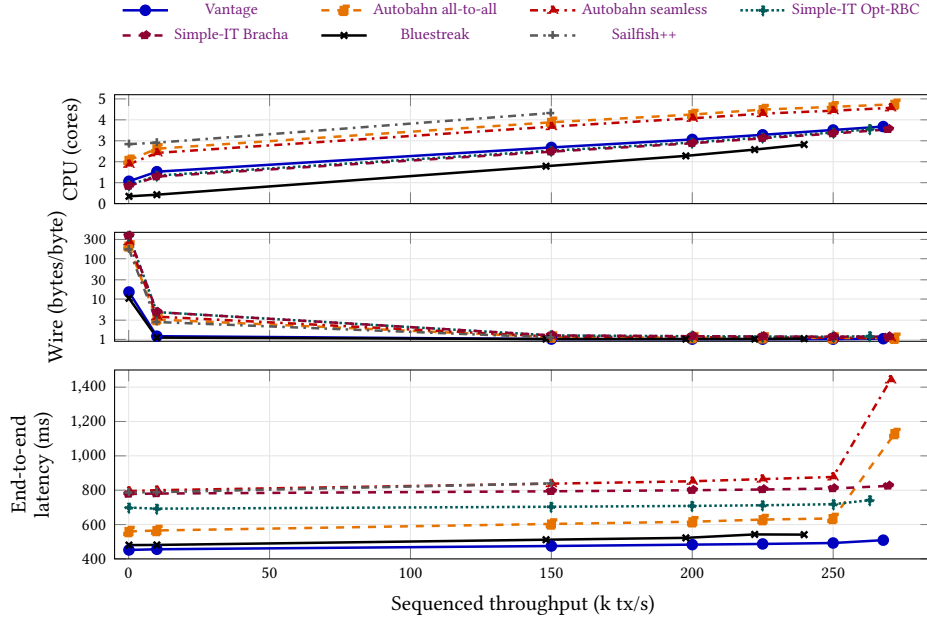


Fig. 6. Throughput sweep at $n = 100$. Cells are medians across campaign repetitions on fresh fleets, on the fleets of Appendix F.2. The x-axis is measured sequenced throughput; end-to-end latency is submission-to-materialization p50. The last point of each series is its highest accepted load; the next offered rung overloads and is not plotted.

7.4 Q3: What does optimistic leader relay cost?

We isolate the work created when a leader admits data before it is widely disseminated, the leader-load effect of Section 3. At $n = 20$ and $f = 6$, every author receives the same share of a uniform offered load R . Each of six Byzantine publishers creates one batch per $\Delta = 200$ ms, publishes it to five correct holders, keeps its header visible, and refuses repair. Including the publisher, each attacked lane has six direct holders, one short of the $f + 1 = 7$ availability threshold. Correct holders serve normally, and the groups cover every correct consensus leader. There is no equivocation, invalid proposal, or extra load; correct authors therefore supply the reachable $0.7R$ workload.

We compare optimistic all-to-all Autobahn, Vantage, and Simple-IT Opt-RBC. Seamless Autobahn is outside this test because sub-threshold tips never enter its certified path. Optimistic Autobahn admits a locally held tip and makes the leader serve missing bytes before replicas vote [20]. Vantage can reject that tip while retaining its proposal core, whereas Simple-IT waits for its availability path before admission. The figure therefore separates correct-author goodput from the attacked-lane data that Autobahn additionally orders.

All three protocols sustain the full correct-author workload at every offered rate. At 200k tx/s this is 140k correct-author tx/s. Optimistic Autobahn also orders nearly all attacked-lane data because a leader that admitted those tips must relay them; Vantage and Simple-IT order none of it. Autobahn’s p50 reaches about 2.3 s, compared with 0.46 s for Vantage and 0.67 s for Simple-IT, and it uses roughly twice their CPU. The result exposes an admission-policy tradeoff rather than a loss of useful goodput.

The tail shows where each policy pays (Table 3). Simple-IT remains near 0.86 s p99 because unavailable data never enters consensus. Vantage’s median changes little, but its p99 rises to roughly 1.2–1.4 s as a rejected tip returns through

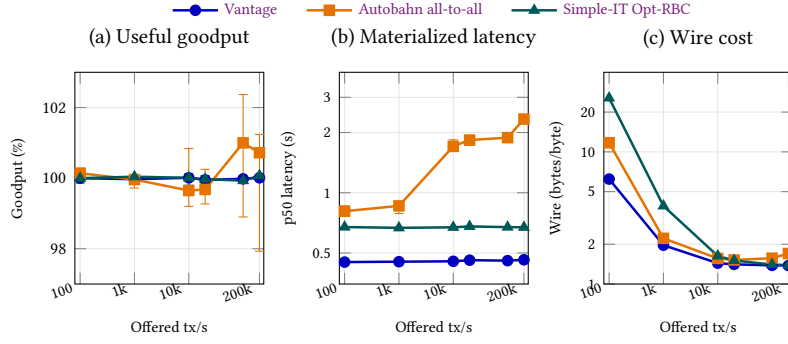


Fig. 7. Leader-relay stress at $n = 20$, $f = 6$; each point is the median of three campaign repetitions, and whiskers span the repetitions. Panel (a) reports correct-author throughput as a fraction of the reachable $0.7R$ load; points scatter within about a percent of 100% because work in flight at the window boundary is counted on one side of it or the other. Panel (b) shows materialization p50 in seconds; panel (c) shows wire bytes per committed byte.

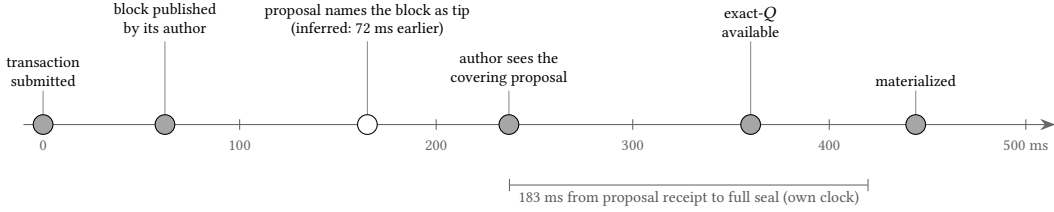


Fig. 8. One transaction's path in the local $n = 20$ study at 100 tx/s: p50 stage latencies chained on one axis from submission. Publication follows submission by about 62 ms (blocks are published every 100 ms); first coverage by a proposal and exact- Q availability are measured from publication; materialization is the direct submission-to-materialization median, 444 ms. p99 values are in Appendix F.4.

a later core — the tip-spoiling cost discussed in Section 8. Optimistic Autobahn's median and tail both grow because the admitted tips must be relayed; at 200k its p99 is about 5.7 s. These are measurements of this withholding profile, not protocol capacity bounds.

7.5 Q4: Inside one Vantage view

To check directly that a proposal can cover a block before the block is exact- Q -available, we run a local $n = 20$ study at 100 tx/s with the same RTT matrix injected in process (Appendix F.4). Each of three 60-second runs materializes all 6,000 submitted transactions after drain. Every data-plane outcome is full, and no resolver message or resolver seal appears.

Figure 8 chains the stage medians on one timeline. A transaction's block is published about 62 ms after submission; the author sees its block named by a proposal 175 ms after publication, before the block becomes exact- Q -available at 298 ms; submission to materialization measures 444 ms directly. The 175 ms is stamped at the author when it fixes the covering proposal, so it spans one RTT plus the wait for the next proposal opportunity; the proposal itself was sent one one-way delay earlier, about 100 ms after publication under this matrix (median one-way delay 72 ms). Both waits follow from two cadences. Authors publish a lane block every 100 ms (the header delay, Appendix F.2), so a transaction's batch waits about 50 ms on average for the next block — the measured 51 ms from first digest to publication. Proposals

follow one another about every 70 ms, one one-way delay apart (Figure 2; about 14 views per second in the $n = 100$ runs, Appendix F.4), so a block that has reached the proposer waits about 35 ms on average for the next proposal — the measured 175 ms less one 143 ms RTT. Proposal receipt to local full seal takes 183 ms.

The proved aligned endpoints — 3δ fast or 4δ READY for a tip, 5δ or 6δ for a core entry (Table 1, Section 5) — are not converted into measured milliseconds: under the RTT matrix δ differs per pair, and the measured intervals average over arrival phase.

In the representative matrix run (Figure 12), payload and lane traffic account for about half of typed bytes, while AGB contributes about 40% of bytes and 60% of logical messages. Sequence state and primary-worker traffic make up nearly all of the remainder. The inactive target-local resolver contributes no bytes or messages.

In a matched network-delay control (Figure 11), the all- n fast-seal shortcut supplies about 70% of local seals under the heterogeneous matrix and every observed seal under a uniform RTT; all outcomes in both experiments are full. This is the run-time race behind the two VANTAGE lines in Table 1: fast-seal waits for the slowest of all n ECHOs while the 4δ READY path waits only for quorums. Heterogeneous delays therefore route the remaining 30% through READY, while uniform delays let all-to-all finish first.

7.6 Q5: Can recovery keep up?

We first isolate a pure transient crash at $n = 10$, $f = 3$. The counted 1,000 tx/s load is placed on the seven non-victims; three otherwise-correct validators are killed for about 30 seconds and then restarted with their local state intact. The native-Linux Docker deployment uses the same ten-region netem matrix and $\Delta = 200$ ms as the cloud campaigns.

As Figure 9 shows, the seven non-victims sustain the offered 1,000 tx/s before, during, and after the outage. Their latest-one-second p50 is about 0.46 s before the crash, peaks near 1.4 s shortly after restart, and returns to about 0.46 s. Transactions delayed during the outage enter the histogram only when they materialize, which is why the peak can follow restart; the curve is a one-second measurement, not a cumulative or one-minute value. Views whose proposer is a crashed victim are silent and take the skip shortcut of Section 6.

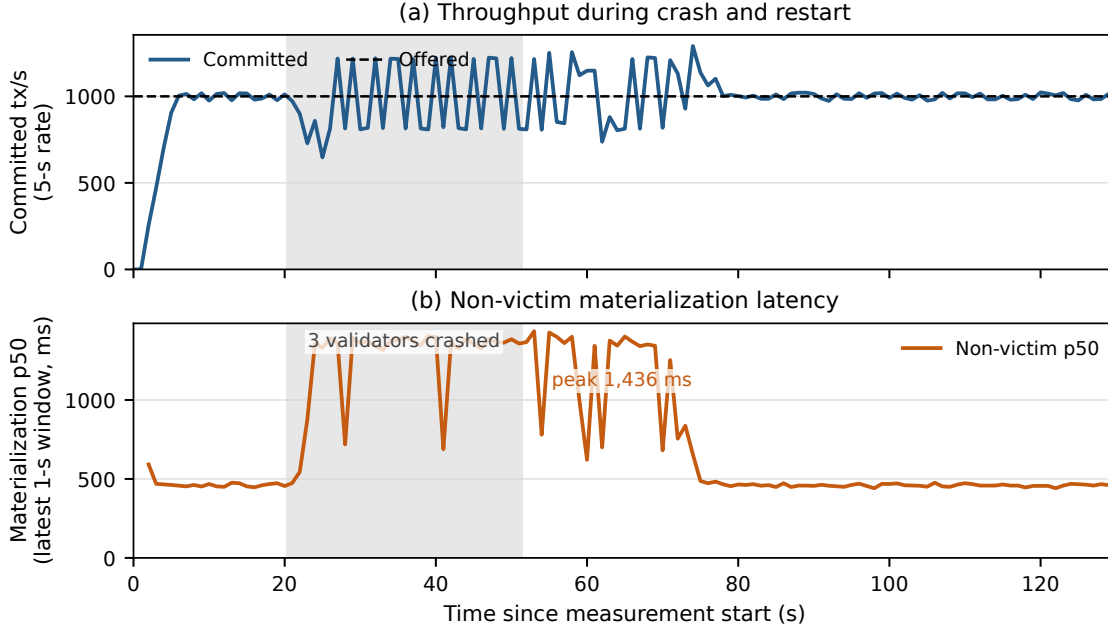


Fig. 9. Pure transient crash at $n = 10$, $f = 3$ and 1,000 offered tx/s. The gray interval is the roughly 30-second outage. Both curves are evaluated every second over the seven non-victims: panel (a) is their median trailing-5-second committed rate, and panel (b) is their median latest-1-second materialization p50. This is one diagnostic run, not a confidence interval.

The harder test creates mixed-open targets continuously. At $n = 10$, $f = 3$, each of three Byzantine publishers proposes its own partially disseminated tip and suppresses all of its protocol responses from seconds 20–110. Correct publishers offer the counted 1,000 tx/s; the faulty publishers add 600 uncounted tx/s. We repeat the 240-second run on three fresh committees.

In Figure 10, every injected mixed-open target seals at every correct validator. Targets arrive and seal at about four per second; the unresolved backlog stays around ten rather than growing, then drains in about two seconds after injection stops because target instances progress concurrently. Correct-load throughput sustains the offered 1,000 tx/s. Targets seal everywhere in about 2.8 s at the median and about 4 s at the slow end. The latest-one-second materialization p50 rises from about 0.44 s to 4.3 s while the attack is active and returns afterward.

The timing follows the resolver path. A target activates when the proposal frontier reaches $u + 3$ (Section 6; Figure 4). Under this fault pattern it may then encounter up to two silent resolver primaries, each consuming the $5\Delta = 1$ s no-proposal timeout, before a correct-primary agreement completes. The 11Δ full-view timer bounds an unsuccessful proposed resolver view; a correct primary can finish without waiting for that deadline, so it is not the main delay in this experiment. Because output remains in view order, a slow open target also delays later transactions at the output cursor. Faster activation, faulty-primary turnover, or correct-primary agreement could reduce this latency, but the present traces do not separate their individual costs.

These measurements answer the mechanism question for this attack: the resolver keeps pace with the observed mixed-open arrival rate and drains the remaining targets after the fault stops. They do not bound an arbitrary Byzantine

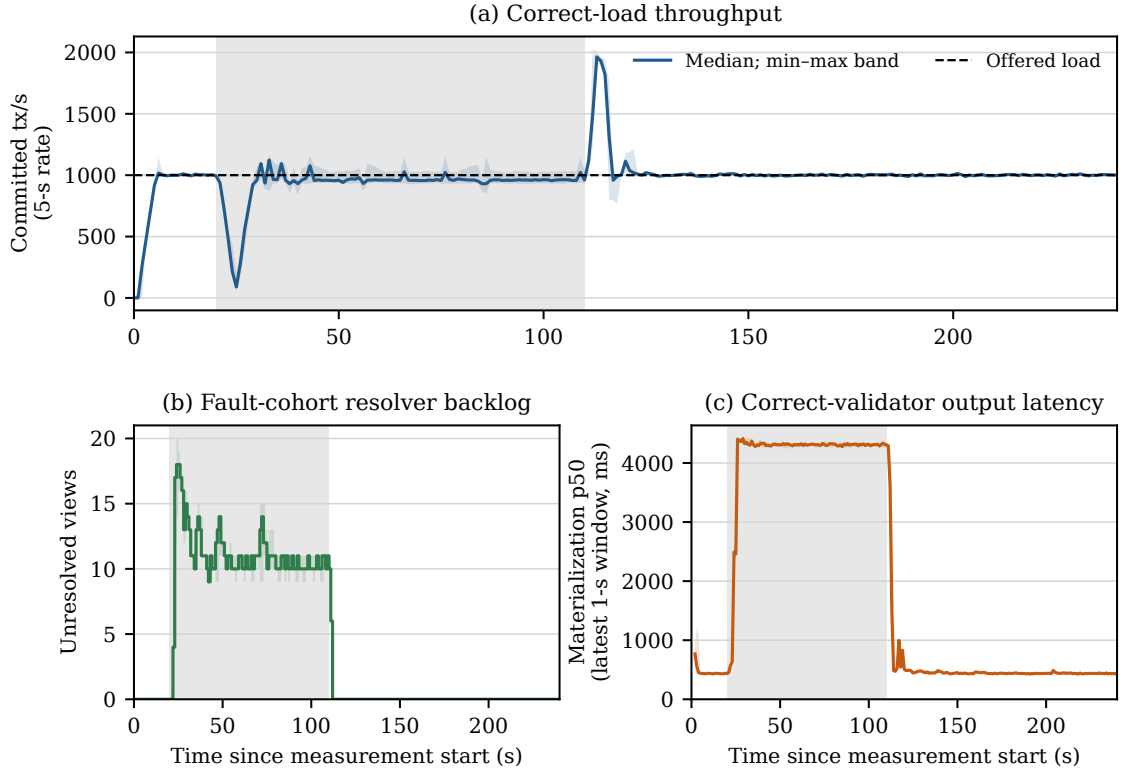


Fig. 10. Sustained mixed-open recovery at $n = 10$, $f = 3$ over three fresh committees. Each run first takes the median over the seven correct validators; curves then show the repetition median and min-max band. The gray interval is Byzantine injection. Panel (a) is the trailing-5-second correct-load committed rate, evaluated every second; panel (b) is the unresolved injected-target count; and panel (c) is the latest-1-second materialization p50.

arrival process, the number of simultaneously active targets, or a target whose resolver repeatedly selects faulty primaries.

8 DISCUSSION AND LIMITATIONS

Experimental scope. The cloud series report medians across up to three fresh-fleet repetitions; a few baseline cells rest on fewer, and the transient-crash trace is one run. The controlled deployment places validators in one AWS availability zone and uses netem to reproduce a ten-region RTT matrix. It therefore isolates latency shape without modeling Internet loss or jitter. The fault experiments (Sections 7.4 and 7.6) cover partial publication with repair refusal, transient crash/restart, and sustained mixed-open views.

The cost of channel authentication. Our model assumes a reliable authenticated link for every pair, concretely a pairwise MAC (Section 4); pairwise MACs in place of signatures go back to Practical Byzantine Fault Tolerance (PBFT) [13]. The transport implements it and our runs keep it enabled (Section 7.1), so the reported numbers include its cost. That cost is small and bytes-proportional rather than round-trip-bound. In a single-host A/B — ten validators in one process, 1000 tx/s, 512 B transactions, the ten-region RTT matrix, 180 s, two runs per arm — both configurations sustain the offered load at about 0.44 s p50, while authentication adds roughly 9% CPU. Its wire share depends on frame size because batching amortizes one 16-byte tag: tags account for about 10% of the small frames at 100 tx/s and below 0.1% of framed bytes at saturation. The authenticated $n = 100$ campaigns report no verification failure.

Resource contention and active targets. The Direct AGB homogeneous path does not logically wait for a resolver. Each unresolved target runs an independent instance, so different targets can advance at network pace rather than waiting for one global service slot. They still share processor and network capacity with the data path, and an open tip stops ordered output at its cursor position. With k active targets, the resolver’s message, wire, timer, and retained-state costs multiply by k . Completed-open quarantine limits retries by correct proposers; it constrains neither Byzantine proposers nor the rate at which they create target evidence. The Q5 stress (Section 7.6) shows matching arrival and seal rates for one $n = 10$ attack, not a bound under arbitrary arrivals or a saturation result. A fixed target is eventually selected and decided, but Byzantine primaries can consume resolver views, so no constant general failed-view latency is claimed; only the clean crash-only case of Corollary D.13 has a direct bound.

Formal retention. The protocol model retains the lane, target, and resolver evidence used by its rules indefinitely. Bounded-state recovery and reclamation are implementation concerns outside the protocol and theorem statements.

Tip spoiling. One unverifiable entry makes the entire optional tip manifest fail a receiver’s local check, so a faulty author can prevent other authors’ fresh blocks in that manifest from being included. The faulty author publishes its lane-tip block only to the correct proposer and withholds matching acknowledgments from every other correct party. Repair may supply the bytes; it cannot recreate direct proof of authorship. Multimmit calls the same withholding easy spoiling in Raptr’s prefix voting [24, 40].

A correct proposer’s completion and core survive it. For every $n \geq 3f + 1$, the proposer is the only correct party that can send grade 1 for the resulting tip manifest, so even if all f Byzantine parties also send grade 1, at most $1 + f < 2f + 1 \leq n - f$ grade-1 ECHOs exist: neither the all- n fast-seal shortcut nor a READY quorum of $n - f$ grade-1 statements can form. A party whose ECHO quorum mixes grades sends READY-mix immediately, and ECHO and READY quorums count both grades, so ordinary completion remains available. If grade 0 later reaches $n - f$, each provisional sender refines to READY-0 and the Direct AGB READY path seals commit-core. Neither case treats repaired possession as proof of authorship. Section 7.4 measures the cost: Vantage’s median changes little while its p99 rises to 1.2–1.4 s as the rejected tip returns through a later core.

A fully Byzantine schedule can still withhold enough ECHOs that neither grade reaches $n-f$ before the refinement window closes. The residual READY-mix then leaves the view open-tip: completed, core fixed, tip undecided. Completed-open quarantine stops a correct proposer from reopening the same lane on every turn — it drops from ordinary later tip manifests any author whose carried tip entry never gathered $n-f$ acknowledgments naming that exact prefix, then re-probes that author through T after 1, 2, 4, . . . of its own proposer opportunities. Appendix B.2 gives the selection rule, the fabricated-witness case, and the per-author state cost. The discipline is caller-side and no receiver can check it. Byzantine proposers ignore it and keep creating residual open views on their own turns.

Reuse boundary. Direct AGB applies when one committee handles a value with two parts: a stable prefix and an optional suffix that can be omitted safely. Two candidates elsewhere are a shared mempool and a checkpoint with a speculative log tail. Each such use has to define its own eligibility rules and prove availability for them; this paper proves only the author-lane instance. AGB alone is not reliable broadcast, not a cross-committee availability certificate, and not consensus. Vantage supplies the resolver, deterministic order, repair, and input fairness around it (Corollary C.3).

Why the skip shortcut is narrow. Relaying a skip vote after $f + 1$ votes is unsafe once a party has echoed a non-skip proposal for that target. Requiring a free stance removes the conflict but can leave too few parties to complete the relay. So the grounded shortcut — one vote per party, sent only after that party’s own NO-READY and a first-hand quorum of ECHO-SKIP — covers a clean crash-only silent view and nothing else. Every other failed view goes through its target-local resolver.

Provenance boundary. The provenance guarantee concerns validator-authored blocks: an output block under index i was published by p_i to a correct party. A Byzantine author may still invent payload bytes or publish forks. Authenticating external clients is an application concern, and state-dependent invalid or replayed transactions must be handled deterministically during execution. None of that weakens common block ordering. None of it is a client-level no-creation guarantee either.

Per-coordinate grades. One grade bit per tip entry would let quorum intersection select an outcome for each two-valued coordinate, produce (C, S) for some $S \subseteq T$, and confine simple withholding to one author’s lane. The resolver stays: Byzantine ECHOs can still make a coordinate mixed, and the optimistic lock and resolution checks would need a new per-coordinate proof, which this paper does not have. The multi-height version is a monotone bitmap; unlike Multimmit’s rank rule, the individual questions remain two-valued at $n = 3f + 1$.

9 CONCLUSION

A signature-free data path can carry another author’s block one message delay after receiving it directly, with no certificate for that block and no availability vote completed beforehand. Vantage splits each proposal into a quorum-available core manifest and an optimistic tip manifest. Completion fixes the core; the first-hand responses decide whether the tip travels with it. When all n parties return a grade-1 ECHO, a proposal seals two message delays after its send, which puts a block published in time for that proposal into the ordered log three delays after publication. Only two paths in our survey reach that endpoint — Autobahn’s all-to-all fast path and FinWhale’s $p = 1$ path (Table 1) — and both buy it with signed votes and a PKI where Vantage uses authenticated channels and hashes.

Safety holds without timing assumptions: correct outputs never diverge, and every output block is valid, author-backed, and retrievable. After GST every valid block of a correct author is eventually included, and a clean crash-only silent view is skipped without depending on the resolver. At $n = 100$ on the emulated ten-region WAN, Vantage holds

the lowest median latency at every accepted load among the compared protocol variants and sustains 250k tx/s at about 0.5 s p50, with a highest accepted throughput of about 268k tx/s (Section 7.3). In the mixed-open stress (Section 7.6), every injected mixed-open target seals, the resolver keeps pace with this attack, and the backlog does not grow, although interval p50 rises to about 4.3 s while the attack is active. This is not a general rate or latency bound: active targets share finite resources and faulty primaries can consume resolver views (Section 8).

ACKNOWLEDGMENTS

Large language models from Anthropic and OpenAI were used under the author’s direction for code development, for adversarial auditing of the proofs, and for editing and restructuring the manuscript’s prose. The protocol design, the proofs, and all claims are the author’s, and the author is responsible for all content.

REFERENCES

- [1] Ittai Abraham, Sourav Das, Yuval Efron, and Jovan Komatovic. 2026. Forget-IT: Optimal Good-Case Latency for Information-Theoretic BFT. Cryptology ePrint Archive, Paper 2026/355.
- [2] Ittai Abraham, Ling Ren, and Zhuolun Xiang. 2022. Good-Case and Bad-Case Latency of Unauthenticated Byzantine Broadcast: A Complete Categorization. In *International Conference on Principles of Distributed Systems (OPODIS) (Leibniz International Proceedings in Informatics, Vol. 217)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 5:1–5:20. <https://doi.org/10.4230/LIPIcs.OPODIS.2021.5>
- [3] Ittai Abraham and Gilad Stern. 2021. Information Theoretic HotStuff. In *International Conference on Principles of Distributed Systems (OPODIS) (Leibniz International Proceedings in Informatics, Vol. 184)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 11:1–11:16. <https://doi.org/10.4230/LIPIcs.OPODIS.2020.11>
- [4] Michael Anoprenko, Andrei Tonkikh, Alexander Spiegelman, Petr Kuznetsov, Anatoliy Zinovyev, and Konstantin Shprenger. 2025. DAGs for the Masses. <https://doi.org/10.48550/arXiv.2506.13998> arXiv:2506.13998v4, revised 1 March 2026.
- [5] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. 2025. Shoal++: High Throughput DAG BFT Can Be Fast!. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Philadelphia, PA, USA, 813–826. arXiv:2405.20488.
- [6] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. 2025. Mysticeti: Reaching the Latency Limits with Uncertified DAGs. In *Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 18 pages. <https://doi.org/10.14722/ndss.2025.240929>
- [7] Kushal Babel, Fatima Elsheimy, Lioba Heimbach, Mohammad Mussadiq Jalalzai, Tobias Klenze, Jovan Komatovic, Jason Milonis, Mike Setrin, and Victor Shoup. 2026. Cadence: Extreme Pipelining with Multiple Concurrent Proposers. arXiv:2607.02275 [cs.DC] arXiv:2607.02275v2.
- [8] Leemon Baird and Atul Luykx. 2020. The Hashgraph Protocol: Efficient Asynchronous BFT for High-Throughput Distributed Ledgers. In *2020 International Conference on Omni-layer Intelligent Systems (COINS)*. IEEE, Piscataway, NJ, USA, 1–7. <https://doi.org/10.1109/COINS49042.2020.9191430>
- [9] Gabriel Bracha. 1987. Asynchronous Byzantine Agreement Protocols. *Information and Computation* 75, 2 (1987), 130–143.
- [10] Manuel Bravo, Gregory Chockler, and Alexey Gotsman. 2022. Liveness and Latency of Byzantine State-Machine Replication. In *36th International Symposium on Distributed Computing (DISC) (Leibniz International Proceedings in Informatics, Vol. 246)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Wadern, Germany, 12:1–12:19. <https://doi.org/10.4230/LIPIcs.DISC.2022.12>
- [11] Christian Cachin and Stefano Tessaro. 2005. Asynchronous Verifiable Information Dispersal. In *IEEE Symposium on Reliable Distributed Systems (SRDS)*. IEEE Computer Society, Los Alamitos, CA, USA, 191–202.
- [12] Miguel Castro. 2001. *Practical Byzantine Fault Tolerance*. Ph.D. Dissertation. Massachusetts Institute of Technology. Technical Report MIT-LCS-TR-817; the MAC-authenticated variant.
- [13] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, New Orleans, LA, USA, 173–186.
- [14] Benjamin Y. Chan and Rafael Pass. 2023. Simplex Consensus: A Simple and Fast Consensus Protocol. In *Theory of Cryptography Conference (TCC) (Lecture Notes in Computer Science, Vol. 14372)*. Springer, Cham, Switzerland, 452–479. https://doi.org/10.1007/978-3-031-48624-1_17
- [15] Xiaohai Dai, Zhaonan Zhang, Jiang Xiao, Jingtao Yue, Xia Xie, and Hai Jin. 2023. GradedDAG: An Asynchronous DAG-Based BFT Consensus with Lower Latency. In *IEEE International Symposium on Reliable Distributed Systems (SRDS)*. IEEE Computer Society, Los Alamitos, CA, USA, 107–117. <https://doi.org/10.1109/SRDS60354.2023.00020>
- [16] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: A DAG-Based Mempool and Efficient BFT Consensus. In *European Conference on Computer Systems (EuroSys)*. Association for Computing Machinery, New York, NY, USA, 34–50. <https://doi.org/10.1145/3492321.3519594>
- [17] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2 (1988), 288–323. <https://doi.org/10.1145/42282.42283>

- [18] Paul Feldman and Silvio Micali. 1988. Optimal Algorithms for Byzantine Agreement. In *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC)*. Association for Computing Machinery, New York, NY, USA, 148–161.
- [19] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. 2022. Dumbo-NG: Fast Asynchronous BFT Consensus with Throughput-Oblivious Latency. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 1187–1201. <https://doi.org/10.1145/3548606.3559379>
- [20] Neil Girdharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn: Seamless High Speed BFT. In *ACM Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 1–23. <https://doi.org/10.1145/3694715.3695942>
- [21] Idit Keidar, Lefteris Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All You Need Is DAG. In *ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 165–175. <https://doi.org/10.1145/3465084.3467905>
- [22] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. 2023. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. In *International Symposium on Distributed Computing (DISC) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 281)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:22. <https://doi.org/10.4230/LIPIcs.DISC.2023.26>
- [23] Razya Ladelsky and Roy Friedman. 2026. FinWhale: An Optimally Resilient Two-Round Terminating DAG Protocol. arXiv:2606.26292 [cs.DC]
- [24] Andrew Lewis-Pye and Patrick O’Grady. 2026. Multimitt: Extending Blocks for Faster Finality. arXiv:2607.21021 [cs.DC] arXiv:2607.21021.
- [25] Andrew Lewis-Pye and Ehud Shapiro. 2026. Morpheus Consensus: Excelling on Trails and Autobahns. In *29th International Conference on Principles of Distributed Systems (OPDIS 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 361)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 35:1–35:20. <https://doi.org/10.4230/LIPIcs.OPDIS.2025.35>
- [26] Dahlia Malkhi, Chrysoula Stathakopoulou, and Maofan Yin. 2025. BBKA-CHAIN: Low Latency, High Throughput BFT Consensus on a DAG. In *Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 14744)*. Springer, Cham, Switzerland, 51–73. https://doi.org/10.1007/978-3-031-78676-1_4 28th International Conference, FC 2024; arXiv:2310.06335.
- [27] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. 2014. Signature-Free Asynchronous Byzantine Consensus with $t < n/3$ and $O(n^2)$ Messages. In *ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 2–9. <https://doi.org/10.1145/2611462.2611468>
- [28] MystenLabs. 2026. Post-Quantum Signatures Zoo. <https://mahdi-mysten.github.io/nist-sigs-zoo/>. Accessed 2026-08-17.
- [29] National Institute of Standards and Technology. 2024. Module-Lattice-Based Digital Signature Standard. FIPS 204. <https://doi.org/10.6028/NIST.FIPS.204>
- [30] National Institute of Standards and Technology. 2024. Stateless Hash-Based Digital Signature Standard. FIPS 205. <https://doi.org/10.6028/NIST.FIPS.205>
- [31] Nikita Polyanskii, Sebastian Müller, and Ilya Vorobyev. 2025. Making Uncertified DAG BFT Provably Live with Linear Payload and Quadratic Metadata Communication. Cryptology ePrint Archive, Paper 2025/567. <https://eprint.iacr.org/2025/567>
- [32] Nikita Polyanskii, Ilya Vorobyev, and Sebastian Müller. 2026. Bluestreak: Scaling DAG BFT by Sparsifying Metadata. Cryptology ePrint Archive, Paper 2026/898.
- [33] Nikita Polyanskii, Ilya Vorobyev, and Sebastian Müller. 2026. Starfish-S: Optimistic Payload Sequencing at Leader-Commit Latency. Manuscript.
- [34] Mayank Raikwar, Nikita Polyanskii, and Sebastian Müller. 2024. SoK: DAG-based Consensus Protocols. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, Piscataway, NJ, USA, 1–18. <https://doi.org/10.1109/ICBC59979.2024.10634358>
- [35] Victor Shoup. 2024. Sing a Song of Simplex. In *International Symposium on Distributed Computing (DISC) (Leibniz International Proceedings in Informatics, Vol. 319)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 37:1–37:22. <https://doi.org/10.4230/LIPIcs.DISC.2024.37> Introduces DispersedSimplex; full version: Cryptology ePrint 2023/1916.
- [36] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. 2025. Sailfish: Towards Improving the Latency of DAG-based BFT. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE Computer Society, Los Alamitos, CA, USA, 1928–1946. <https://doi.org/10.1109/SP61157.2025.00021> Full version: Cryptology ePrint 2024/472.
- [37] Nibesh Shrestha, Qianyu Yu, Aniket Kate, Giuliano Losa, Kartik Nayak, and Xuechao Wang. 2025. Optimistic, Signature-Free Reliable Broadcast and Its Applications. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 3780–3794. <https://doi.org/10.1145/3719027.3765220> Introduces SAILFISH++, a signature-free DAG BFT protocol; use arXiv:2505.02761 v4 or later.
- [38] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. 2025. Shoal: Improving DAG-BFT Latency and Robustness. In *Financial Cryptography and Data Security (FC) (Lecture Notes in Computer Science, Vol. 14744)*. Springer, Cham, Switzerland, 92–109. 28th International Conference, FC 2024; arXiv:2306.03058.
- [39] Alexander Spiegelman, Neil Girdharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 2705–2718. <https://doi.org/10.1145/3548606.3559361>
- [40] Andrei Tonkikh, Balaji Arun, Zhuolun Xiang, Zekun Li, and Alexander Spiegelman. 2025. Raptr: Prefix Consensus for Robust High-Performance BFT. arXiv:2504.18649 [cs.DC] arXiv:2504.18649.
- [41] Yann Vonlanthen, Jakub Sliwinski, Massimo Albarello, and Roger Wattenhofer. 2024. Banyan: Fast Rotating Leader BFT. In *Proceedings of the 25th International Middleware Conference*. ACM, New York, NY, USA, 494–507. <https://doi.org/10.1145/3652892.3700788>

- [42] Feifan Wang, Jingfan Yu, Zixi Cai, and Zhixuan Fang. 2026. Clownfish: Scaling DAG-based BFT Consensus via Sparse Edges. arXiv:2606.04687. <https://doi.org/10.48550/arXiv.2606.04687>
- [43] Qianyu Yu, Giuliano Losa, Nibesh Shrestha, and Xuechao Wang. 2026. Angelfish: Leader, DAG, or Anywhere in Between. <https://arxiv.org/abs/2509.15847> To appear at ACM CCS 2026; full version, arXiv:2509.15847v3.
- [44] Qianyu Yu, Giuliano Losa, and Xuechao Wang. 2024. TetraBFT: Reducing Latency of Unauthenticated, Responsive BFT Consensus. In *ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 257–267. <https://doi.org/10.1145/3662158.3662783> Full version: arXiv:2405.02615.
- [45] Qianyu Yu, Juan Villacis, Giuliano Losa, Zhuolun Xiang, and Xuechao Wang. 2026. Simple-IT: Practical Low-Latency Signature-Free BFT Consensus. arXiv:2606.14404 [cs.DC] arXiv:2606.14404.
- [46] Lior Zeno, Naama Ben-David, and Mark Silberstein. 2026. In Link We Trust: BFT at the Speed of CFT using Switches. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association.

A COMPARISON DETAILS

This appendix expands the normalized comparison of Table 1. The calculations use the same aligned cross-author setting and do not turn favorable endpoints into general fault-path bounds.

Mempool-Simple-IT. A leader names a batch only after collecting $2f + 1$ availability votes, and a receiver checks $f + 1$ locally [45]. This two-delay gate, two-delay Opt-RBC, and one commit-vote delay give 5δ under the roughly $5n/6$ optimistic threshold. It certifies availability before proposal; Vantage admits a directly published tip after one delay but may leave it open.

Sailfish++. Opt-RBC makes each nonleader vertex referenceable after two delays. Then $n - f - 1$ ordinary correct nonleaders sequence in 5δ and the remaining f late ones in 7δ —two classes, not a range [37, 45]. This supplies per-vertex RBC; Vantage’s graded tip instead gives the 3δ all-to-all or 4δ READY endpoint and may need target-local resolution.

Autobahn. The leader may reference a directly received current lane tip after one delay while its parent remains PoA-certified [20]. Linear fast consensus adds three delays, giving 4δ ; the two-phase slow path adds five, giving 6δ [20]. On the sketched all-to-all path, Prepare is the carrier; one delay spreads every vote and another lets each party form the all- n fast certificate locally. Thus the derived total is $\delta + 2\delta = 3\delta$, without a final certificate broadcast. The paper does not state this endpoint; like Vantage’s all- n fast-seal shortcut, one non-voter disables it. The optimistic tip is outside Autobahn’s seamlessness guarantee: a missing receiver fetches it and withholds its vote, while weak/strong refinement still needs $f + 1$ strong assertions in the PrepareQC. Vantage lowers the grade without suppressing ECHO or READY, but an open tip has no constant resolver bound.

FinWhale, Mysticeti, and compressed variants. Cordial Miners begins this best-effort-DAG line [22]; Mysticeti pipelines leaders [6]; Starfish supplies the live Push pacemaker and multisigned Mysticeti-L [31]; Bluestreak compresses with ordinary signatures [32]; and FinWhale adds a two-round fast path [23]. Cross-author admission costs one delay, after which Mysticeti’s three-message decision gives 4δ . FinWhale commits a round- r leader after $n - p$ round- $(r + 1)$ references, where $n = 3f + 2p - 1$ and $1 \leq p \leq f$; creating and receiving them gives 3δ . At $p = 1$, $n = 3f + 1$ and one non-voter is tolerated. Its adopted Push pacemaker bounds a round by $2\Delta + 2\delta$ after GST.

Bluestreak sends one advance and block per party per round; Vantage sends one ECHO and initial READY per party per view plus a proposal. Both favorable patterns are $2n^2 + O(n)$; READY refinement raises Vantage’s general bound to $3n^2 + O(n)$. Bluestreak’s happy case is $O(n^2\lambda + nP)$; Vantage’s positional bitmaps give $O(n^2\lambda + n^3)$ aligned, while its proved by-value and ragged/batch worst case remains $O(n^3\lambda)$.

BBCA-Chain. One BEB trip plus the referencing leader’s three-trip BBCA gives the 4δ favorable nonleader path [26]. Timely correct-leader BBCA tolerates f faults; Vantage’s two-delay carrier-to-seal shortcut assumes all parties correct, and its READY path takes three delays. BBCA-Chain waits for the preceding BBCA and recovers through signed complete, adopt, or no-adopt evidence. Vantage overlaps views with an at-most- δ fresh-opportunity gap but has no general recovery bound. The 4δ excludes BBCA-Chain’s optional blob-availability layer.

BBCA-Chain’s view-advancement value below is likewise derived rather than stated. The general bound is $\Delta_\Gamma + \delta$: by Δ_Γ after synchronized entry, every lagging correct party has completed or adopted, or has probed and sent a new-view block; one actual delay then delivers either a complete/adopt certificate or $2f + 1$ correct NOADOPT blocks everywhere. The paper requires $\Delta_\Gamma \geq \Delta_{\text{sync}} + 3\Delta$ and proves $\Delta_{\text{sync}} = \Delta$, so the minimum admissible timer gives the reported $4\Delta + \delta$.

View advancement. Table 1 omits per-view advancement, recorded here. In a view-based protocol, once every correct party has entered view v or higher, the bound is the additional time until every correct party has entered a view strictly higher than v , following the definition of [45]; for a round-based DAG it is the maximum time a correct party can remain in one round. One failed view or round therefore delays progression by at most the bound once it applies, each view or round in a run of failures incurs its own delay, and the bound says nothing about resolving the failed value or draining a recovery queue. The values are $8\Delta + 4\delta$ for Mempool-Simple-IT, $8\Delta + 5\delta$ for Sailfish++ after a correct leader, and $10\Delta + \delta$ for Autobahn, all from [45]; $4\Delta + \delta$ for Vantage (Lemma B.5); $2\Delta + 2\delta$ for FinWhale, the worst-case round duration proved for the Push pacemaker with leader timeout 2Δ [31], which FinWhale adopts, while Mysticeti alone states no such bound; and $4\Delta + \delta$ for BBKA-Chain, derived above. Vantage’s target-local resolver may take longer than view advancement and has no fixed general bound (Section 8).

B DIRECT AGB: FULL SPECIFICATION AND PROOFS

VANTAGE runs one pipelined AGB instance per view. Its round-robin proposer $\text{Proposer}(v) = p_{1+((v-1) \bmod n)}$ broadcasts $B_v = (C_v, T_v)$. The core C_v and tip T_v are author-indexed lane-frontier manifests, encoded in increasing author order with at most one reference (k, h) per author in each (Figure 1); we omit the coordinate and height when clear. These fields are self-asserted and do not alone prove authorship. A correct proposer places an entry in its exact-quorum-available core only after receiving its valid lane prefix directly from the encoded author and counting $n-f$ exact-position author-bound acknowledgments. For a faulty proposer, “quorum-available” is not certified; R2 instead checks that every output-relevant entry is author-backed, which suffices for the retrievability theorem below. Hashes are distinct within the proposal and disjoint across C_v, T_v ; ordered output removes any cross-view repetition. The proposal identifier $\mathcal{H}(B_v)$ abbreviates $\mathcal{H}(\text{view-proposal} \parallel \text{enc}(\text{sid}, v, C_v, T_v))$ and therefore binds the view and the manifests.

At its abstraction boundary, Direct AGB is parameterized by a deterministic syntactic predicate $\text{FORMED}_v(C, T)$ and local predicates $\text{COREOK}_i(C)$ and $\text{TIPOK}_i(C, T)$. The value domain also supplies a stable item identity and a finite, deterministic item sequence $\text{Items}(X)$ for each formed manifest. For every prior-item set D , $\text{Core}_D(C)$ is $\text{Items}(C)$ after removing identities in D and repeated identities, and $\text{Full}_D(C, T)$ appends the likewise deduplicated items of $\text{Items}(T)$. Thus the core expansion is a prefix of the full expansion, and every full item comes from C or T . Write $\text{Witnessed}(X)$ for the value-domain guarantee that every item in $\text{Items}(X)$ is externally valid and permanently retrievable. A conforming eligibility instantiation must prove

$$\text{COREOK}_i(C) \Rightarrow \text{Witnessed}(C), \quad \text{TIPOK}_i(C, T) \Rightarrow \text{Witnessed}(T)$$

whenever p_i is correct. Agreement uses only locality and determinism of these predicates; availability inherits the displayed witness implications.

For conditional termination, COREOK and TIPOK are persistent for a fixed proposal: once true at a correct party, they remain true. A conforming value domain also provides authorization and fair repair: once a correct party authorizes a witnessed manifest or its root, it eventually authorizes and obtains every item in that manifest’s deterministic expansion in every fair execution.

Vantage instantiates $\text{COREOK}_i(C)$ as $\text{AUTHOROK}_i(C)$ and $\text{TIPOK}_i(C, T)$ as $\text{AUTHOROK}_i(T) \wedge \text{TIPANCHORED}_i(C, T)$. Under this instantiation, $\text{Witnessed}(X)$ means that every named valid lane prefix was published by its encoded author to a correct permanent holder; Lemmas B.2 and B.3 prove the witness implications. The author-indexed syntax in Appendix B.2 instantiates FORMED , and the lane-chain traversal in Section 6 instantiates the expansion contract. The standalone safety arguments use only this contract and the initial-response and same-proposal refinement rules.

B.1 Data Dissemination

Payloads are disseminated in data blocks below AGB. An *author lane* is the author-indexed publication structure formed by one party's data blocks. A height- k block of party p_i has the canonical form $b_i^k = (\text{sid}, i, k, h_i^{k-1}, x)$, where $|x| \leq \ell_{\max}$ and h_i^{k-1} is the predecessor hash (the session fixes the shared height-0 genesis). Its identifier is $h_i^k = \mathcal{H}(\text{data-block} \parallel \text{enc}(b_i^k))$, abbreviated $\mathcal{H}(b_i^k)$. A *valid lane prefix* is a linear hash chain whose blocks share one author index, have consecutive heights and matching predecessor hashes back to genesis, and satisfy BLOCKOK. A correct author's valid lane prefixes are mutually comparable; a Byzantine author may publish forked lane prefixes. A correct party creates only valid extensions of its current lane frontier. To publish a new block it sends $\langle \text{PUBLISH}, b_i^k \rangle$ to all parties. A recipient treats this as author publication only when the authenticated channel sender and the block's encoded author are both p_i .

A hash $h = \mathcal{H}(b_i^k)$ names both the block and, through the predecessor-hash chain, its valid lane prefix $b_i^1, \dots, b_i^k$. A reference (i, k, h) in a well-formed fixed proposal, a local completion or seal, or a well-formed resolver value under local validation authorizes repair of its root block, as does the anchor of a counted availability claim (below). After obtaining a block whose hash, session, encoded author, height, and BLOCKOK check match an authorized reference, the requester also authorizes its predecessor reference; this process continues recursively to genesis. For each authorized missing hash, the party requests the block from all parties. A holder answers with a distinctly typed $\langle \text{SERVE}, h, b \rangle$ repair message. A canonical, correct-session, size-bounded, BLOCKOK body whose hash is h is cached even if its encoded author or height does not match the currently authorized coordinate; only an exact coordinate match advances the predecessor walk. This prevents a false coordinate from consuming the sole useful response for a later genuine reference. A reply that fails the hash or body checks is ignored and never marks the hash as obtained. No repair message establishes publication provenance, even when its sender equals the encoded author; if the corresponding PUBLISH message arrives later, it may upgrade the cached bytes.

A correct party records the first pending request for each requester-hash pair even when it does not yet hold the block, and answers once if it later retains that block. A correct requester sends at most one request for a given authorized hash to each peer, and a correct holder answers at most once for each requester-hash pair. Reliable delivery and retained pending requests make retransmission unnecessary; the rule also bounds correct-party traffic induced by repeated Byzantine requests.

For $h = h_a^k$, let $\text{DIRECTPUB}_j(a, k, h)$ mean that p_j holds the valid lane prefix through h and received every block in it through PUBLISH messages directly from p_a . The party retains this prefix and advances its local direct frontier for a , but this event sends no periodic protocol message.

For a formed proposal B , let $R(B) = (r_1, \dots, r_m)$ be the canonical ordered concatenation of its core and tip lane references. A proposal ECHO from p_i carries an authenticated availability claim $A_i(B)$ outside the immutable ECHO identity. For $r_j = (a, k, h)$, its exact-position bit may be set only when $\text{DIRECTPUB}_i(a, k, h)$ holds. Otherwise it may contain one integer (j, d) with $0 < d < k$ only when the predecessor walk from h identifies the unique ancestor $r' = (a, k - d, h')$ and $\text{DIRECTPUB}_i(r')$ holds. Thus the integer means “I directly received this exact ancestor of the named tip,” not merely “I hold some block at height $k - d$.” A correct sender is silent at that position if it cannot verify the ancestry or its direct provenance.

After verifying the canonical proposal body, a receiver treats an exact bit immediately as the logical first-hand acknowledgment $\langle \text{ACK}, a, k, h \rangle_i$. Since DIRECTPUB is a prefix predicate, once the predecessor chain is verified the same claim also acknowledges every derived ancestor in that prefix. For (j, d) the receiver first obtains and verifies the

named tip's predecessor chain, derives the exact h' , and only then counts the acknowledgments for $(a, k - d, h')$ and its derived ancestors; until that walk succeeds, the claim remains pending. Repeated ECHOs or repeated coordinates never count the same sender twice for one exact tuple. A receiver processes the availability field only on that sender's first accepted echo-stage response for the view; later envelope variants do not create additional claims. A Byzantine ECHO author may fabricate its own fields, but contributes at most one counted sender to any exact tuple. Hence a correct acknowledgment still asserts validity, possession, and direct receipt from the encoded author, and every counting party attributes it only to the authenticated ECHO sender.

Definition B.1 (Availability). A lane reference (a, k, h) is q -available at a party if it has received $\langle \text{ACK}, a, k, h \rangle$ first-hand from at least q distinct parties.

Call a reference *exact- n - f -available* at a party when its n - f counted acknowledgments all came from exact-position bits for that reference. Unlike an integer claim, an exact bit needs no predecessor walk at another receiver. Correct Vantage proposers use this stronger, locally checkable condition for core promotion; ordinary availability and AUTHOROK count either encoding.

A valid lane prefix obtained only through repair is held and served but never acknowledged. Hash links prove content, not authorship: anyone can construct a well-linked block carrying another party's index. Authenticated PUBLISH receipt establishes authorship locally, while $f+1$ author-bound acknowledgments establish that some correct recipient observed such a publication. Consequently, a block that p_a never publishes to a correct party gathers at most f acknowledgments for (a, k, h) .

A reference is *locally available* at a party when its valid lane prefix is held by that party or it is $(f+1)$ -available there; a manifest is locally available when every entry in it is. Define the separate provenance gate

$$\begin{aligned} & \text{AUTHOROK}_j(a, k, h) \\ \iff & \text{DIRECTPUB}_j(a, k, h) \\ \vee & ((a, k, h) \text{ is } (f+1)\text{-available at } p_j). \end{aligned}$$

For a manifest, AUTHOROK_j means that the predicate holds for every entry at its encoded author coordinate. AUTHOROK implies local availability; the converse is false for a fetched lane prefix. *Retention rule:* a correct party retains every valid lane prefix it acknowledges; every valid lane prefix whose local holding it relies on when a protocol rule checks local availability, AUTHOROK , or tip anchoring; and every valid lane prefix it obtains by request for such a check — even when the prompting window has closed by the time the lane prefix arrives. It also retains an anchoring chain whose verified ancestry it uses to emit or resolve an integer ACK claim. Hash-correct bodies received during an authorized repair are also cached so that a later exact coordinate can consume them. Retained lane prefixes are held and served indefinitely; local sealing or output of the naming views is no discard signal, since another correct party's seal and output may lag arbitrarily. Bounded-state reclamation is outside the model. We call a reference *retrievable* when some correct party permanently retains its valid lane prefix. In a fair execution, the repair rule below makes every correct party that authorizes the exact reference eventually obtain it.

Algorithm 2 gives the corresponding event handlers; the prose above is normative.

LEMMA B.2 (AVAILABILITY WITNESSES). *For every reference (a, k, h) :*

- (i) *if a correct party relies on it in a local-availability, AUTHOROK , or tip-anchoring check, then the reference is retrievable;*
- (ii) *in every fair execution, every correct party that authorizes a retrievable reference eventually obtains its valid lane prefix;*

Algorithm 2: Authenticated data publication and repair at party p_i

```

/* A stored block may carry a direct-publication mark, a repair mark, or both; only the first can establish provenance or an ECHO ACK claim. */

Global variables and constants:
┌ maps of cached blocks and direct-publication marks; sets authorized, requested, pendingReq, retained, and answered; direct lane frontiers;
└ per-tuple ECHO claim counts and pending exact and integer ACK claims

upon the local application creates the next valid block  $b_i^k$  do
┌ store  $b_i^k$ ; process it as a direct publication from  $p_i$ ; broadcast  $\langle \text{PUBLISH}, b_i^k \rangle$ 

upon a direct  $\langle \text{PUBLISH}, b \rangle$  from  $p_a$  whose encoded author is  $a$  do
┌ if  $b = (\text{sid}, a, k, h', x)$  is canonical,  $|x| \leq \ell_{\max}$ , and  $\text{BLOCKOK}(b)$  then
└    $h \leftarrow \mathcal{H}(b)$ ; store  $b$  and mark  $(a, k, h)$  directly published by  $p_a$  /* may upgrade repaired bytes */

function  $\text{DIRECTPUB}_i(a, k, h)$ :
┌ return  $p_i$  holds the valid lane prefix through  $h$  and every block in it carries a direct-publication mark from  $p_a$ 

upon  $\text{DIRECTPUB}_i(a, k, h)$  first becomes true do
┌ retain the valid lane prefix through  $h$ ; advance the direct frontier of  $a$  to  $(a, k, h)$  if it extends that frontier

function  $\text{ACKCLAIMS}_i(B)$ :
┌ let  $R(B) = (r_1, \dots, r_m)$  be the canonical lane-reference vector; initialize an  $m$ -bit exact vector and an empty sparse integer map; foreach
└    $r_j = (a, k, h)$  do
└   if  $\text{DIRECTPUB}_i(a, k, h)$  then
└     set exact bit  $j$ 
└   else
└     if the predecessor walk from  $h$  verifies a greatest directly published ancestor  $(a, k - d, h')$  with  $0 < d < k$  then
└       add  $(j, d)$  to the sparse map
└ return the exact vector and sparse map

upon the first accepted authenticated proposal ECHO from  $p_s$  with claim  $A_s(B)$  and a formed canonical body  $B$  do
┌ foreach exact bit  $j$  of  $A_s(B)$  do
└   count  $p_s$  for  $r_j$ ; record the pending claim  $(s, j)$  with claimed height  $k$ ;  $\text{AUTHORIZE}(r_j)$ 

┌ foreach sparse entry  $(j, d)$  of  $A_s(B)$  do
└   record the pending claim  $(s, j, d)$  with claimed height  $k - d$ ;  $\text{AUTHORIZE}(r_j)$ 

upon a pending claim by  $p_s$  on anchor  $r_j = (a, k, h)$  with claimed height  $k^*$ , and a verified ancestor  $(a, k', h')$  of  $r_j$  with  $k' \leq k^*$  do
┌ count  $p_s$  for  $(a, k', h')$  unless already counted for that tuple /* state rule; also fires when the chain is already cached */

procedure  $\text{AUTHORIZE}(a, k, h)$ 
┌ add  $(a, k, h)$  to authorized if absent
┌ if a cached  $b = (\text{sid}, a, k, h', x)$  satisfies  $\mathcal{H}(b) = h$ ,  $\text{BLOCKOK}(b)$ , the size bound, and either  $k > 1$  or  $h'$  is the session genesis hash then
└   if  $k > 1$  then
└      $\text{AUTHORIZE}(a, k - 1, h')$ 
└ else
└   foreach  $p_j \in \Pi$  with  $(j, h) \notin \text{requested}$  do
└     add  $(j, h)$  to requested; send  $\langle \text{REQUEST}, h \rangle$  to  $p_j$ 

upon  $\langle \text{SERVE}, h, b \rangle$  arrives for a requested hash  $h$  do
┌ if  $b = (\text{sid}, a, k, h', x)$  is canonical,  $\mathcal{H}(b) = h$ ,  $|x| \leq \ell_{\max}$ , and  $\text{BLOCKOK}(b)$  then
└   cache  $b$  as repaired data, without a direct-publication mark /* coordinate-independent cache */
/* a failed hash/body check changes no state */

upon a cached  $b = (\text{sid}, a, k, h', x)$  matches an authorized  $(a, k, \mathcal{H}(b))$  and either  $k > 1$  or  $h'$  is the session genesis hash do
┌  $\text{AUTHORIZE}(a, k, \mathcal{H}(b))$  /* fires after either PUBLISH or SERVE; continues the walk */

upon an authorized predecessor walk verifies a valid lane prefix through genesis do
┌ retain every block in that lane prefix; invoke  $\text{TRYSERVE}$  on their hashes

upon a direct  $\langle \text{REQUEST}, h \rangle$  from  $p_j$  do
┌ add  $(j, h)$  to pendingReq if  $(j, h) \notin \text{answered}$ ;  $\text{TRYSERVE}(h)$ 

upon a cached block with hash  $h$  becomes retained do
┌  $\text{TRYSERVE}(h)$ 

procedure  $\text{TRYSERVE}(h)$ 
┌ if a retained block  $b$  satisfies  $\mathcal{H}(b) = h$  then
└   foreach  $(j, h) \in \text{pendingReq} \setminus \text{answered}$  do
└     add  $(j, h)$  to answered; remove it from pendingReq; send  $\langle \text{SERVE}, h, b \rangle$  to  $p_j$ 

function  $\text{AUTHOROK}_i(a, k, h)$ :
┌ return  $\text{DIRECTPUB}_i(a, k, h)$  or at least  $f + 1$  counted first-hand ECHO claims for  $(a, k, h)$ 

```

- (iii) if it is $(n-f)$ -available at one correct party, then it eventually becomes $(f+1)$ -available at every correct party.
- (iv) if it is exact- $n-f$ -available at a correct party by time $\beta \geq \text{GST}$, then it is $(f+1)$ -available at every correct party by $\beta + \delta$.

PROOF. For (i), if the checking party holds the lane prefix, the retention rule applies; otherwise it counted $f+1$ acknowledgments, including one from a correct party that holds and retains it. For (ii), requests go to every party and a correct holder answers. Starting at h , each verified block authorizes its predecessor, so induction on height retrieves the lane prefix through genesis; collision resistance makes each requested hash unambiguous. For (iii), at least $n - 2f \geq f+1$ counted acknowledgments are from correct parties, and their claim-bearing ECHOs eventually reach every correct party. A counted claim authorizes its anchor's predecessor walk at every receiver; the correct claim author retains that chain, so fair repair eventually derives the same exact tuples everywhere. For (iv), at least $n - 2f \geq f+1$ of the exact claims counted by time β came from correct ECHO authors. Their formed claim-bearing ECHOs were already broadcast and arrive everywhere by $\beta + \delta$ under the in-flight convention; exact positions require no anchor repair. $\square$

LEMMA B.3 (AUTHOR WITNESS). *If a correct party satisfies $\text{AUTHOROK}_j(a, k, h)$, then some correct party permanently holds the valid lane prefix ending at h and received every block in that lane prefix through PUBLISH messages directly from p_a . In particular, every block in the lane prefix satisfies BLOCKOK and p_a published every such block to a correct party.*

PROOF. The claim is immediate in the DIRECTPUB case. Otherwise, among the $f+1$ acknowledgers counted by p_j at least one is correct. A correct party contributes that logical author-bound acknowledgment in its ECHO only after its own DIRECTPUB predicate holds, and the retention rule is permanent. $\square$

A count of $n-f$ acknowledgments is therefore local evidence that at least $n - 2f \geq f+1$ correct parties hold the directly published valid lane prefix and, by Lemma B.2(iii), that both the $(f+1)$ -availability and AUTHOROK conditions used for a core entry below eventually hold at every correct party.

B.2 Interface

The direct interface has one sender input, two idempotent environment events, one completion event that makes the core irrevocable, and an optional direct-result event:

- $\text{PROPOSE}(v, C, T)$ — invoked once by Proposer(v); a correct caller supplies a value satisfying $\text{FORMED}_v(C, T)$. In Vantage, a correct proposer places in C only references (a, k, h) for which $\text{DIRECTPUB}_i(a, k, h)$ holds and the reference is exact- $n-f$ -available there, and in T only references for which $\text{DIRECTPUB}_i(a, k, h)$ holds. Concretely, it lists in C , for each author, the newest such exact- $n-f$ -available reference. For a non-quarantined author, T contains the least-height generally $n-f$ -available but not exact- $n-f$ -available directly published prefix strictly above C , when one exists, and otherwise the newest directly published lane tip whose valid lane prefix strictly contains the author's C entry — an author with no C entry is anchored at genesis, so any directly published tip qualifies — at most one C and one T entry per author. Inclusion is thus deterministic in the proposer's first-hand state. Here *newest* means greatest verified lane height, with the lexicographically smallest hash breaking an equal-height tie. If an author forked its lane, the C entry pins the kept branch below itself (the same height/hash order resolves locally visible forks; a forking author is Byzantine), and a directly published lane tip on any other branch is never included. Authors are processed in a canonical order, skipping any hash already listed under an earlier index, so all entries are globally distinct and no hash appears in both manifests. A correct Vantage proposer also keeps one local quarantine witness and two scalar retry counters per author. Ordinary inclusion and an initial READY-mix do not fill this slot. When

a READY quorum first completes a proposal while neither READY grade has quorum, the party stores each tip reference that is not exact- $n-f$ -available in its author's empty slot. It omits a from ordinary later T manifests until the witness becomes exact- $n-f$ -available or a containing prefix becomes its current C candidate, then clears the slot. Number this party's own proposer opportunities. A new witness is due at the next opportunity; after an actual retry, its successive gaps are 1, 2, 4, . . . opportunities. In one proposal the party selects at most one due quarantined author with a directly published prefix above the current core, choosing least due opportunity and then author order. If the author has a generally $n-f$ -available but not exact- $n-f$ -available prefix above the core, it puts the least-height such prefix in T as a stable confirmation target; otherwise it puts the newest directly published tip there. The same confirmation preference applies to ordinary, non-quarantined T selection. A due author without such a directly published tip is skipped without obstructing another eligible author or advancing its retry counter. Re-completion does not replace a nonempty slot or reset its backoff. This caller-side discipline is not part of FORMED and cannot constrain a Byzantine proposer. It never activates on a homogeneous completing quorum. Proposal ECHOs eventually clear any slot whose witness a correct author actually published. Even a fabricated witness cannot censor a correct author's lane: later due probes use that author's real directly published prefixes, and a general claim quorum freezes the least unconfirmed coordinate until exact-position ECHOs confirm it, without resetting the witness or its backoff. Over-inclusion elsewhere is harmless since sequencing outputs every data block once (Section 6). Each manifest thus has at most n entries. Vantage instantiates FORMED by requiring that each of C and T list at most one entry per author, all hashes across C and T are distinct, and every reference has a canonical positive height and hash. Correct parties never echo a malformed proposal (R2), so the per-author entry bound — and with it the n -entry bound — holds for every echoed proposal, Byzantine ones included. These are syntactic checks: when bytes or acknowledgments are available, $\text{AUTHOROK}_i(a, k, h)$ can hold only for a valid lane prefix whose encoded author and height are exactly the reference coordinate (a, k) .

- $\text{ACTIVATE}(v)$ — permits the fixed proposal's positive echo gate to fire. In Vantage this local event occurs when authenticated receipt advances the contiguous responsive proposal frontier through v ; formal entry activates separately through $\text{ENTER}(v)$.
- $\text{ENTER}(v)$ — records the local entry time, activates the fixed proposal if present, and arms the echo and ready fallback deadlines. Vantage generates this local event with the WISH pacemaker of Appendix B.3. Neither environment event is a counted statement or transferable evidence.
- $\text{COMPLETE}(v) \rightarrow B_v$ — emitted when the first proposal-ready quorum is counted; it fixes $B_v = (C, T)$ and makes C an irrevocable prefix of the view's eventual non-skip output. A completed but unsealed view is open-tip(C, T).
- $\text{DIRECT-SEAL}(v) \rightarrow X$ — the optional direct output, where X is $\text{commit-full}(C, T)$ or $\text{commit-core}(C)$. It fires on the first homogeneous ready quorum of the corresponding grade. The mixed grade is nonterminal and skip is not a Direct AGB output.

Direct AGB never reads resolver state. Vantage connects its DIRECT-SEAL output to a caller-owned $\text{TRY-SEAL}(v, X)$ arbiter. Every direct result, Vantage fast or skip result, and resolver outcome is submitted to it. While empty, the arbiter accepts the first submission, stores X , and emits the terminal $\text{SEAL}(v) \rightarrow X$ event; afterward it retains that outcome and ignores later submissions. Compatibility ensures that two submissions, if both occur, name the same outcome. This output multiplexer belongs to the composition, not to Direct AGB. The lifecycle vocabulary of Section 2 applies throughout: the arbiter is what turns a direct, fast, skip, or resolver result into the terminal *seal* event.

B.3 Vantage Pacemaker and Activation

The following service is Vantage’s concrete implementation of Direct AGB’s ENTER and ACTIVATE environment events. It is not used by the asynchronous safety proof of Theorem B.7.

The pacemaker uses first-hand $\langle \text{WISH}, x \rangle$ messages, following the bounded-space high-watermark construction of Bravo, Chockler, and Gotsman [10]. Party p_i stores, for every author p_j , the largest directly received wish $\omega_i[j]$. Let ω_i^+ and ω_i^Q be the $(f+1)$ st and $(n-f)$ th largest entries, respectively. When ω_i^+ exceeds p_i ’s own largest wish, it broadcasts a wish for ω_i^+ ; independently, whenever ω_i^Q increases it raises its entry target to ω_i^Q . Wishes are high-watermarks, and the two updates are independent: a wish for x supports every view at most x , and target advancement never waits for $\omega_i^Q = \omega_i^+$. On a target x , the party records entry to every missing view through x immediately and in order.

Initially $\omega_i[j] = 0$ for every j , the largest entered view is 0, and the own-wish high-watermark is 0. Genesis enters view 1, after which p_i sets $\omega_i[i] = 2$, records own wish 2, and broadcasts $\langle \text{WISH}, 2 \rangle$ (self-delivery is immediate). Upon receiving $\langle \text{WISH}, x \rangle$ first-hand from p_j , including as a piggybacked high-watermark, p_i sets $\omega_i[j] \leftarrow \max\{\omega_i[j], x\}$ and recomputes both order statistics. It first performs any enabled $f+1$ amplification, updating $\omega_i[i]$ and its own-wish high-watermark when it broadcasts and recomputing both order statistics after that update. It then records entry through any newly supported quorum target. Stale wishes cause no transition.

Write $E_i(u)$ for p_i ’s unique echo-stage response and $R_i(u)$ for its initial ready-stage response in view u . The local trigger for wishing follows Starfish’s two-response progress condition [33]: before emitting whichever of $E_i(v-1)$ and $R_i(v-2)$ completes the pair, p_i raises its own wish to at least $v+1$. Every response piggybacks the sender’s current own-wish high-watermark outside its counted response identity. An optional READY-mix refinement carries the same field but does not run this progress trigger a second time. Amplification may send a wish standalone. A wish only schedules views; it is never an echo, a ready, an availability acknowledgment, an origin bit, or a resolution justification.

Let $e_i(v)$ be the time p_i enters view v . Entry is strictly increasing locally. Recording entry to v arms both fallback schedulers for v ; positive responses may already have fired through the responsive path, and pending responses may finish after later entries. R2 and R3 close them by $e_i(v) + \theta_E$ and $e_i(v) + \theta_R$, respectively, in post-GST drift-free time, unless their positive gates fire earlier. Thus entry never waits for an older response window.

In parallel, p_i maintains a *responsive proposal frontier* a_i , initially 0. Formal entry to view v raises a_i to at least $v-1$. For each view, p_i fixes the first proposal received directly from its designated proposer and ignores later versions. When the fixed proposal for a_i+1 is well formed, it advances a_i by one; buffered consecutive fixed proposals are processed in increasing view order. A proposal that advances the frontier may activate its positive response path before formal entry, but it never causes formal entry and is not evidence for any quorum. Thus WISH remains the only recovery and synchronization mechanism, while a contiguous stream of first-hand proposals supplies a one-hop responsive path. As in the rest of the unbounded-execution model, parties retain future per-view messages; the non-Zeno convention makes the retained set finite at every finite time.

The designated proposer also uses WISH evidence directly: R1 invokes $\text{PROPOSE}(v, \cdot)$ already when ω_i^+ first supports v at $p_i = \text{Proposer}(v)$, one amplification hop before the entry quorum threshold ω_i^Q . The early proposal is passive — receivers fix it and authorize repair of its references, but no response fires before activation — and it cannot outrun genuine progress. For $v \geq 3$, by induction over raise events, the earliest correct wish supporting v is an own raise of the two-response trigger, by whose local step the sender has emitted its echo-stage response for a view at least $v-2$; an amplification raise to $\omega_i^+ \geq v$ counts a wish of at least one correct author with a strictly earlier send and inherits the same provenance. Genesis wishes support only views up to 2. Since $\omega_i^+ \geq v$ counts $f+1$ authors, at least one correct,

the trigger certifies that progress. Its payoff appears in the fallback analysis: the wish quorum behind the first correct entry to v contains $f+1$ correct wishes supporting v , which reach $\text{Proposer}(v)$ within one delay of that entry, so the proposer invokes within δ of the first entry rather than within the 2δ entry spread (Theorem B.8). A smaller entry spread would require a smaller entry threshold: the $f+1$ correct wishes guaranteed behind the first entry reach every correct party within one delay, but completing a $n-f$ quorum from them costs the amplification hop, since the up to f Byzantine wishes inside the first quorum reach only their chosen recipient. An entry threshold that may contain fewer than $f+1$ correct wishes would instead let the adversary drive a single correct party's deadlines arbitrarily far ahead, spending its one-shot refusals. Signed and certified pacemakers escape this dilemma by relaying transferable evidence; the proposer-side trigger recovers the lost delay exactly where the liveness analysis needs it, because proposing early is harmless where entering early is not.

LEMMA B.4 (PACEMAKER PROGRESS AND SYNCHRONIZATION).

- (a) *Every correct party eventually enters every view and emits one echo-stage and one initial ready-stage response for every view; a provisional READY-mix may additionally refine once.*
- (b) *There is a finite view V such that, for every $v \geq V$, if the first correct entry to v occurs at time s_v , then every correct party enters v by $s_v + 2\delta$.*
- (c) *If every party is correct, the same cutoff can be chosen so that every correct party enters each $v \geq V$ by $s_v + \delta$.*

PROOF. For (a), all correct parties enter view 1 from genesis; responses for nonpositive views are fixed, and every correct party initially broadcasts $\langle \text{WISH}, 2 \rangle$. Suppose all correct parties eventually enter v . The deadlines for views $v-1$ and $v-2$ force every correct party to emit both responses, thereby broadcasting a wish of at least $v+1$. Every correct party therefore receives a wish quorum for $v+1$ and enters every missing view through $v+1$. Induction proves entry to all views; the stage deadlines then prove emission of every response.

For (b), assume the standard non-Zeno execution convention and choose V larger than every view entered, proposed, responded to, or wished for by a correct party before GST; this is a finite prefix. Fix $v \geq V$ and its first correct entry time $s = s_v \geq \text{GST}$. Let $x \geq v$ be the target whose traversal caused that entry. The entering party counted a wish quorum supporting x , including at least $f+1$ correct authors. Their wishes reach every correct party by $s + \delta$, causing every correct party to broadcast a wish supporting x if it has not already done so. These wishes arrive everywhere by $s + 2\delta$, making every ω_i^Q support x . Sequential traversal therefore enters v by that time, even if the target is higher.

For (c), every author in the wish quorum counted at time s_v is correct. Those same wishes reach every correct party by $s_v + \delta$, directly giving each of them a wish quorum for the same target. No amplification hop is needed. $\square$

The synchronizer also bounds how long a view can hold every correct party.

LEMMA B.5 (EVENTUAL WORST-CASE VIEW DURATION). *Let v be a view whose t_1 — the earliest time at which every correct party has entered view v or higher — satisfies $t_1 \geq \text{GST}$. Then every correct party enters a view greater than v by $t_1 + \theta_R + \delta = t_1 + 4\Delta + \delta$. Vantage's eventual worst-case view duration, in the sense of [45], is therefore $4\Delta + \delta$.*

PROOF. Fix a correct party p_i . By t_1 it has recorded entry, in order, to every view through v , so $e_i(v-2) \leq e_i(v-1) \leq e_i(v) \leq t_1$; the entries themselves may precede GST. The fallback rules close its echo-stage response for $v-1$ by $\max\{e_i(v-1), \text{GST}\} + \theta_E$ and its initial ready-stage response for $v-2$ by $\max\{e_i(v-2), \text{GST}\} + \theta_R$: a timer still armed at GST has at most its full duration remaining once clocks are drift-free. Hence the progress pair for $v-1$ completes by $t_1 + \theta_R$; for $v=2$ the initial ready-stage member of the pair is a fixed genesis response, so the pair completes with

the echo-stage emission alone, and for $v = 1$ both members are fixed and the genesis wish already supports 2. Before emitting the completing response, p_i raises its own wish to at least $v + 1$ unless its own wish already supports $v + 1$; in either case p_i has broadcast a wish supporting $v + 1$ by $t_1 + \theta_R$. The correct parties' wishes supporting $v + 1$ — at least $n - f$ of them — reach every correct party by $t_1 + \theta_R + \delta$ under the in-flight convention, make ω^Q support $v + 1$ there without an amplification hop, and sequential traversal enters every view through $v + 1$. $\square$

B.4 Protocol

All statements below are broadcast to all parties over authenticated channels. A party sends at most one echo-stage statement and one *initial* ready-stage statement for a view. Echo-stage statements are either proposal echoes or ECHO-SKIP and are immutable. The initial ready-stage statement is proposal-ready or NO-READY; NO-READY and homogeneous proposal-ready are final, whereas an initial READY-mix may be followed by at most one same-proposal homogeneous refinement. A party counts a statement only after receiving it directly from its author. It counts the first ready-stage statement from an author once toward the stage-response and proposal-ready censuses. If that first statement is READY-mix for B , it additionally accepts one later READY- g for the same B , $g \in \{0, 1\}$, into the grade- g tally without counting the author again; every other later or conflicting statement is ignored. If the homogeneous statement is the first countable version from that author — whether due to network reordering or because digest-named statements awaited body validation — it is the final initial statement and a later READY-mix is ignored. Where READY-mix was counted first, it remains historical non-grade-1 evidence for resolver candidate selection even after refinement. The sender's own responses are delivered and counted immediately under the self-delivery convention of Section 4.

In the Vantage wrapper, responses for views $v' \leq 0$ are fixed genesis responses treated as received from every party. Genesis enters every correct party into view 1, raises its responsive proposal frontier to 0, and makes it broadcast $\langle \text{WISH}, 2 \rangle$. Entry, frontier state, and WISH messages are not evidence for a broadcast instance; safety holds for arbitrary entry and timeout timings.

- R1. Propose.** On its first sender input $\text{PROPOSE}(v, C, T)$, party $p_i = \text{Proposer}(v)$ sends $\langle \text{PROPOSE}, v, C, T \rangle$ once. Vantage invokes this input when its responsive frontier first satisfies $a_i \geq v - 1$ or its wish statistic first satisfies $\omega_i^+ \geq v$ (Appendix B.3), and constructs (C, T) as in Appendix B.2. In particular, formal entry to v raises that frontier to $v - 1$, so a correct Vantage proposer invokes the input no later than its own entry; receipt of the preceding direct proposal or early wish evidence may make it invoke earlier.
- R2. Echo stage.** Proposal messages may arrive before or after a party enters the proposal's view. The party fixes the first view- v proposal received directly from $\text{Proposer}(v)$ and ignores every later version. It checks $\text{FORMED}_v(C, T)$; a malformed fixed proposal is not echoable. Otherwise the party stores $B_v = (C, T)$ and invokes the value domain's authorization and repair hooks for C and T . In Vantage, a malformed proposal also cannot advance the responsive frontier, and authorization requests from all parties every missing block of each named lane prefix. A stored proposal becomes *active* after the local environment invokes $\text{ACTIVATE}(v)$ or $\text{ENTER}(v)$. In Vantage, processing the fixed proposal invokes $\text{ACTIVATE}(v)$ exactly when it advances the responsive frontier to v ; a buffered proposal can therefore activate when the missing contiguous prefix arrives. From activation onward, while the echo-stage response is pending, the party continuously evaluates the positive gate: $\text{COREOK}_i(C)$ and $\text{TIPOK}_i(C, T)$ both hold. In Vantage, the tip predicate means AUTHOROK_i holds for every entry of T and the *tip-anchoring check* holds: for every author index present in both manifests, the party holds the tip entry's full lane prefix and that lane prefix strictly contains the author's C entry. This is a pure local hash walk, so

counted acknowledgments alone never substitute for a paired tip. When the gate first holds, the party broadcasts $\langle \text{ECHO}, v, C, T, 1 \rangle$, even if formal entry has not yet occurred.

Formal entry to v additionally arms the fallback scheduler. If the response is still pending and a well-formed proposal is fixed, let t be the later of entry and its direct receipt. The fallback window is $[t, \min\{t + \Delta, e_i(v) + \theta_E\}]$. If no grade-1 echo has fired by its end, the party broadcasts $\langle \text{ECHO}, v, C, T, 0 \rangle$ if $\text{COREOK}_i(C)$ holds, and $\langle \text{ECHO-SKIP}, v \rangle$ otherwise. If no echoable proposal becomes active by the absolute fallback deadline $e_i(v) + \theta_E$, the party instead broadcasts $\langle \text{ECHO-SKIP}, v \rangle$. A well-formed proposal delivered after that deadline can no longer be echoed; it is still fixed and its references are authorized for repair, and in Vantage the responsive frontier processes it. No later echo-stage statement for v is allowed; the pre-entry and post-entry routes share this single immutable response state.

R3. Ready stage. Entry to view v arms its ready-stage refusal deadline. Independently of entry, while the initial response is pending the party continuously evaluates the first-hand echoes it has counted. As soon as a quorum of proposal echoes names a common view proposal $B_v = (C, T)$, it broadcasts the proposal-ready statement $\langle \text{READY}, v, C, T, g \rangle$, where, over all echoes counted at emission, the grade g is 1 if it has counted $n-f$ grade-1 proposal echoes for B_v , 0 if it has counted $n-f$ grade-0 proposal echoes for B_v , and the mixed grade mix otherwise. Grades 0 and 1 finalize the sender's ready stage. A mixed grade is provisional: while fewer than all n echo-stage responses have been counted and the ready deadline has not fired, the sender continues evaluating echoes for the same B_v . If one grade reaches $n-f$, it broadcasts one $\text{READY-}g$ refinement for B_v and finalizes. Counting all n echo-stage responses without such a quorum, or reaching $e_i(v) + \theta_R$, instead finalizes the mixed response without another statement; deadline ties process delivered messages first. The refinement is not a second initial response and does not invoke the WISH progress hook. The Vantage wrapper does not quarantine on this provisional response. If formal entry has occurred and no initial proposal-ready has fired by $e_i(v) + \theta_R$, it broadcasts the refusal $\langle \text{NO-READY}, v \rangle$. Wishes and entry notifications never satisfy the echo gate or count toward a ready quorum.

R4. Completion and direct result. Upon counting a ready quorum for a common $B_v = (C, T)$, disregarding grades and counting each author once, record $\text{COMPLETE}(v) \rightarrow B_v$. On the first homogeneous ready quorum, emit $\text{DIRECT-SEAL}(v) \rightarrow \text{commit-full}(C, T)$ for grade 1 or $\text{DIRECT-SEAL}(v) \rightarrow \text{commit-core}(C)$ for grade 0. A completed view with no homogeneous ready quorum remains open-tip(C, T) at the Direct AGB boundary. Vantage may close it through the resolver adapter of Section 6. In the same local transition that first completes a proposal, if neither grade has a ready quorum, the Vantage wrapper applies the quarantine update of Appendix B.2 to its non-exact-quorum tip entries. If a homogeneous quorum completes the proposal, completion and direct seal occur atomically and no quarantine is created. Ready counting continues after completion, so a late-arriving ready that closes a homogeneous quorum produces a direct result. Vantage submits every such result to its caller-owned TRY-SEAL arbiter; a compatible result arriving after a terminal Vantage seal is ignored by that arbiter. Neither completion nor the direct result waits for formal entry. When a homogeneous quorum is the first completing quorum, completion and the corresponding DIRECT-SEAL event occur in the same local step.

Vantage adds the following caller-side rule without changing Direct AGB. For a proposal $B = (C, T)$, a *matching response* is a grade-1 proposal echo for exactly B . Immediately before a correct party sends a matching response, it records an optimistic lock $L_i(v, B)$. The lock is *active* exactly while the party has counted fewer than $f+1$ first-hand echo-stage responses for v that are not matching responses for B . Thus a lock may be born inactive, and once inactive it

never becomes active again. Upon counting matching responses from all n parties, the wrapper emits

$$\text{FAST-SEAL}(v) \rightarrow \text{commit-full}(C, T)$$

once and submits that result to TRY-SEAL. It does not fire COMPLETE or DIRECT-SEAL, expose a core separately, or create a resolution witness; R3 and R4 continue normally. The resolver-side use of L_i is defined in Section 6.

The interface and R1–R4 above define Direct AGB. Each caller initializes $\text{quarantine}_i[a] = \perp$, its two retry counters to zero, and $\text{ownProposalTurn}_i = 0$ once per session; this caller state persists across views. Algorithm 3 gives Vantage’s concrete realization in event form; its frontier, WISH, manifest construction, and TRY-SEAL lines are caller-side wrappers rather than additional Direct AGB requirements. Algorithm 4 separately gives the Vantage-only fast- and skip-seal and lock-release hooks so they are not mistaken for Direct AGB rules.

Grades, including a permitted homogeneous refinement, are outside the counted view-proposal identity: mixed grades can never prevent completion. A residual mixed grade leaves only the tip open; a refinement may instead close a later homogeneous quorum. Grade 0 is positive support for a terminal core-only outcome, not merely the absence of grade 1. A ready carries no transferable echo certificate; its support is checked only by its sender.

LEMMA B.6 (RESPONSIVE PROPOSAL PACING). *Suppose every party is correct and v is past the cutoff of Lemma B.4(c). Let β_v be the time B_v is sent. Then*

$$\beta_{v+1} \leq \max_{u \leq v} \beta_u + \delta,$$

so the least not-yet-proposed view always follows within δ of the latest send. Call B_v prefix-extending when every lower view was sent by β_v . Suppose B_v is prefix-extending, the AUTHOROK and tip-anchoring checks for B_v pass everywhere by $\beta_v + \delta$. Then every correct party obtains the ordinary Direct AGB full result by $\beta_v + 3\delta$, and every correct party also emits and submits the Vantage fast result $\text{commit-full}(C_v, T_v)$ by $\beta_v + 2\delta$. The send times need not be monotone in the view number: the early wish trigger of R1 may send a proposal before an older one, and receivers buffer it until its contiguous prefix arrives or formal entry activates it.

PROOF. Because v is past the cutoff, $\beta_v \geq \text{GST}$. Let $z = \max_{u \leq v} \beta_u$. In an all-correct execution every view $u \leq v$ is proposed, and by z every such B_u has been sent; the in-flight convention delivers all of them everywhere by $z + \delta$, and in-order processing of buffered fixed proposals from genesis raises every correct frontier to v by $z + \delta$ — a proposal delivered after its echo deadline is still fixed and processed by the frontier (R2). Hence Proposer($v + 1$) sends B_{v+1} by $z + \delta$ unless it already did, proving the first claim.

For a prefix-extending B_v , the same delivery argument with $z = \beta_v$ activates B_v at every correct party by $\beta_v + \delta$, so under the stated checks every positive echo gate holds by $\beta_v + \delta$. No fallback response can preempt it: fallback timers exist only after formal entry, and the first correct formal entry satisfies $s_v \geq \beta_v - \delta$, since the all-correct spread of Lemma B.4(c) makes the proposer enter and send by $s_v + \delta$. Party p_i ’s echo fallback window starts at $t \geq$ its direct receipt of $B_v \geq \beta_v$ and therefore ends at $\min\{t + \Delta, e_i(v) + \theta_E\} \geq \beta_v + \delta$, using $e_i(v) + \theta_E \geq \beta_v + 3\Delta - \delta$ from $e_i(v) \geq s_v$; the absolute ECHO-SKIP deadline is the same bound, and the NO-READY deadline $e_i(v) + \theta_R \geq \beta_v + 4\Delta - \delta$ exceeds the ready gate time $\beta_v + 2\delta$ below. Messages delivered by a deadline are processed before its timeout branch, so every echo is grade 1 and fires by $\beta_v + \delta$. The echoes arrive everywhere by $\beta_v + 2\delta$; these are all n matching responses for B_v , so the Vantage wrapper emits its fast full result then. Independently, positive proposal-ready fires even before formal entry, and those statements arrive everywhere by $\beta_v + 3\delta$, producing the ordinary Direct AGB full result. All proposal echoes in this all-correct execution have grade 1, so no completed-open quarantine is created. $\square$

Algorithm 3: Direct Availability-Graded Broadcast as instantiated by Vantage for view v at party p_i (R1–R4;
 $Q = n - f$)

Global variables and constants:
 $fixed_v \leftarrow \perp$; $echo_v \leftarrow$ pending; $ready_v \leftarrow$ pending /* later provisional(B) or final */
 $completed_v, directed_v \leftarrow \perp$
 $active_v, activated_v, proposed_v \leftarrow$ false /* $active_v$: activated with a well-formed fixed proposal */
 $e_i(v), \rho_i(v) \leftarrow \perp$; $\theta_E = 3\Delta$; $\theta_R = 4\Delta$; caller state (Appendices B.2 and B.3): responsive frontier a_i , wish statistic ω_i^+ , $ownProposalTurn_i$, and per-author quarantine slots with $(probeAttempt, probeDue)$

upon $p_i = \text{Proposer}(v)$, not $proposed_v$, and either $a_i \geq v - 1$ or $\omega_i^+ \geq v$ **do**
 $proposed_v \leftarrow$ true; $ownProposalTurn_i \leftarrow ownProposalTurn_i + 1$; clear quarantine slots that are exact- Q -available or core-contained;
 $(C, T) \leftarrow \text{FORMMANIFESTS}_i(ownProposalTurn_i)$ /* exact- Q core; stable tips; at most one due probe */
for the selected probe, if any: $probeAttempt \leftarrow probeAttempt + 1$ and $probeDue \leftarrow ownProposalTurn_i + 2^{probeAttempt-1}$
broadcast $(\text{PROPOSE}, v, C, T)$ /* R1 */

upon local $\text{ENTER}(v)$ or $\text{ACTIVATE}(v)$ from the Vantage pacemaker/frontier wrapper **do**
if this is $\text{ENTER}(v)$ and $e_i(v) = \perp$ **then**
 $e_i(v) \leftarrow$ now; $activated_v \leftarrow$ true; raise a_i to at least $v - 1$; arm deadlines $e_i(v) + \theta_E$ and $e_i(v) + \theta_R$
if this is $\text{ACTIVATE}(v)$ **then**
 $activated_v \leftarrow$ true
if $activated_v$ and $fixed_v = (C, T)$ **then**
 $active_v \leftarrow$ true

upon $fixed_{a_i+1}$ is a well-formed proposal **do**
 $a_i \leftarrow a_i + 1$; invoke $\text{ACTIVATE}(a_i)$ /* responsive frontier; repeats through buffered proposals */

upon the first direct $(\text{PROPOSE}, v, C, T)$ from $\text{Proposer}(v)$ while $fixed_v = \perp$ **do**
 $\rho_i(v) \leftarrow$ now
if (C, T) is malformed **then**
 $fixed_v \leftarrow$ reject /* later proposer versions remain ignored */
else
 $fixed_v \leftarrow (C, T)$; store B_v ; invoke AUTHORIZE for each lane prefix named by C or T /* typed repair; never provenance */
if $activated_v$ **then** $active_v \leftarrow$ true

upon $echo_v$ is pending and $active_v$, evaluated whenever local state changes **do**
let $(C, T) \leftarrow fixed_v$
if $\text{COREOK}_i(C)$ and $\text{TIPOK}_i(C, T)$ **then**
 $echo_v \leftarrow$ sent /* one immutable echo-stage response */
 $\text{RESPOND}(\langle \text{ECHO}, v, C, T, 1 \rangle)$ /* R2 positive path */

upon $e_i(v) \neq \perp$, $fixed_v = (C, T)$, $echo_v$ is pending, and the fallback deadline $d_i^E(v) = \min\{\max\{e_i(v), \rho_i(v)\} + \Delta, e_i(v) + \theta_E\}$ passes **do**
 $echo_v \leftarrow$ sent; **if** $\text{COREOK}_i(C)$ **then**
 $\text{RESPOND}(\langle \text{ECHO}, v, C, T, 0 \rangle)$
else
 $\text{RESPOND}(\langle \text{ECHO-SKIP}, v \rangle)$

upon $e_i(v) \neq \perp$, the deadline $e_i(v) + \theta_E$ passes, $echo_v$ is pending, and no active well-formed fixed proposal exists **do**
 $echo_v \leftarrow$ sent; $\text{RESPOND}(\langle \text{ECHO-SKIP}, v \rangle)$ /* no active echoable proposal */

upon the counted echo set changes and $ready_v$ is pending **do**
if some $B = (C, T)$ has Q proposal echoes **then**
 $g \leftarrow 1$ if Q counted echoes for B have grade 1; 0 if Q have grade 0; otherwise mix
 $ready_v \leftarrow$ provisional(B) if $g = \text{mix}$, else final; $\text{RESPOND}(\langle \text{READY}, v, C, T, g \rangle)$ /* R3; no entry or own-echo guard */

upon the counted echo set changes and $ready_v = \text{provisional}(B)$ **do**
if Q counted echoes for B have one grade $g \in \{0, 1\}$ **then**
 $ready_v \leftarrow$ final; **broadcast** $(\text{READY}, v, C, T, g)$ with the own wish high-watermark piggybacked /* same-value homogeneous refinement; no second WISH hook */
if $ready_v$ is still provisional and all n echo-stage responses are counted **then** $ready_v \leftarrow$ final

upon $e_i(v) \neq \perp$ and the deadline $e_i(v) + \theta_R$ passes **do**
if $ready_v = \text{provisional}(B)$ **then**
 $ready_v \leftarrow$ final /* finalize MIX locally; no second statement */
if $ready_v = \text{pending}$ **then**
 $ready_v \leftarrow$ final; $\text{RESPOND}(\langle \text{NO-READY}, v \rangle)$

upon the counted ready set changes **do**
count each author once; a same- B refinement after READY-mix changes only that author's grade tally
foreach $B = (C, T)$ named by counted proposal-ready statements **do**
if $completed_v = \perp$ and at least Q name B **then**
 $completed_v \leftarrow B$; store B ; authorize lane prefixes named by C ; **if** fewer than Q grade-1 and fewer than Q grade-0 READYs name B **then**
put each non-exact- Q tip in its empty quarantine slot and set its $(probeAttempt, probeDue) \leftarrow (0, ownProposalTurn_i + 1)$
fire $\text{COMPLETE}(v) \rightarrow B$ /* R4; makes the core irrevocable; the composition runs the cursor */
if $directed_v = \perp$ and at least Q grade-1 readies name B **then**
 $directed_v \leftarrow \text{commit-full}(C, T)$; fire $\text{DIRECT-SEAL}(v) \rightarrow \text{commit-full}(C, T)$ and submit it to the caller's TRY-SEAL
else if $directed_v = \perp$ and at least Q grade-0 readies name B **then**
 $directed_v \leftarrow \text{commit-core}(C)$; fire $\text{DIRECT-SEAL}(v) \rightarrow \text{commit-core}(C)$ and submit it to the caller's TRY-SEAL

function $\text{RESPOND}(m)$:
attach $\text{ACKCLAIMS}_i(B)$ outside the identity of any proposal ECHO ; apply Appendix B.3's wish hook and piggyback the own wish high-watermark outside m 's identity; **broadcast** m

Algorithm 4: Vantage's fast- and skip-seal wrappers and resolution stances at p_i

```
/* Hooks into Direct AGB;  $Q = n - f$ ; count only the first direct response per author */
Global variables and constants:
   $fastLock_v \leftarrow \perp$ ;  $fastReleased_v, fasted_v \leftarrow \text{false}$ ; for every target  $u$ : the persistent stance  $z_i(u) \leftarrow \text{free}$  and
   $skipSent_u, skipSubmitted_u \leftarrow \text{false}$ 

upon about to emit grade-1 ECHO for  $B = (C, T)$  in view  $v$  do
   $fastLock_v \leftarrow (C, T)$ 
   $fastReleased_v \leftarrow \text{true}$  iff already  $f + 1$  counted echo-stage responses are not matching responses for  $B$ 
  emit the Direct AGB ECHO /* the lock is recorded first */

upon the counted view- $v$  echo set changes do
  if  $fastLock_v = (C, T)$  and  $f + 1$  counted responses are not matching responses for  $(C, T)$  then
     $fastReleased_v \leftarrow \text{true}$  /* the lock is active iff  $fastLock_v \neq \perp$  and not  $fastReleased_v$  */
  if not  $fasted_v$  and all  $n$  responses are matching responses for some  $(C, T)$  then
     $fasted_v \leftarrow \text{true}$ ; fire FAST-SEAL( $v$ )  $\rightarrow$  commit-full( $C, T$ ); submit it to TRY-SEAL /* no Direct AGB completion */

upon  $R_i(u) = \text{NO-READY}$ , at least  $Q$  counted target-view responses are ECHO-SKIP,  $z_i(u) = \text{free}$ , the known terminal outcome for  $u$  is absent or skip,
and not  $skipSent_u$  do
   $z_i(u) \leftarrow \text{skip-voted}$ ;  $skipSent_u \leftarrow \text{true}$ 
  broadcast  $\langle \text{SKIP-VOTE}, u \rangle$  /* persist the stance first; no  $f + 1$  amplification */

upon at least  $Q$  direct  $\langle \text{SKIP-VOTE}, u \rangle$  statements are counted and not  $skipSubmitted_u$  do
   $skipSubmitted_u \leftarrow \text{true}$ ; fire SKIP-SEAL( $u$ )  $\rightarrow$  skip; submit it to TRY-SEAL /* no Direct AGB completion */
```

B.5 Guarantees

Every quorum below is first-hand and counts distinct authors. Manifests travel by value; replacing them by hashes is an optimization outside these proofs. Every decisive threshold is a quorum or $f+1$, and WISH never contributes to one. R2 treats the eligibility hooks as opaque local predicates; only their witness contract and the composition results depend on their content.

Every correct proposal-ready sender counted a proposal-echo quorum for the named proposal; a correct grade- g sender, $g \in \{0, 1\}$, counted a grade- g echo quorum. Hence any two correct proposal-readies name the same proposal, and correct grade-1 and grade-0 proposal-readies cannot coexist: their grounding echo quorums would intersect in a correct immutable echoer.

The direct safety theorem below treats the eligibility hooks opaquely. It concerns only completions, the initial-response and refinement discipline, and direct R4 seals. Cross-party validity and agreement of resolver-supplied terminal seals are separate composition properties proved in Lemmas D.1 and D.10 and theorem C.2.

THEOREM B.7 (SAFETY). *In every execution, regardless of timing:*

- (a) (Local uniqueness and sender integrity) *Each correct party completes v at most once and emits at most one DIRECT-SEAL result. Every completion names a proposal received directly from Proposer(v) by a correct echoer. If Proposer(v) is correct and proposes $B_v = (C, T)$, every completion names that proposal and every direct result for view v names the payload (C, T) .*
- (b) (Value agreement) *All correct proposal-ready statements, proposal-ready quorums, and completions for v derive from one view proposal $B_v = (C, T)$.*
- (c) (Grade compatibility) *A grade-1 proposal-ready quorum and a grade-0 proposal-ready quorum cannot both exist. Hence correct parties never emit conflicting direct outcomes commit-full and commit-core; either may coexist with the same open-tip(C, T). Parties that have not counted a completion quorum may still remain unknown.*
- (d) (Eligibility witness) *Completion implies Witnessed(C); a grade-1 proposal-ready quorum also implies Witnessed(T). In Vantage these properties mean that every named valid lane prefix is author-backed and retrievable.*

(e) (Ready-refusal exclusion) *A proposal-ready quorum and a no-ready quorum for view v cannot both exist.*

PROOF. (a) A correct party acts only on its first completion and first direct result. Every triggering proposal-ready quorum contains a correct sender, whose grounding echo quorum contains a correct echo. That echo was sent only for a proposal received directly from $\text{Proposer}(v)$, proving sender integrity. Against a correct proposer it names the sender's unique B_v .

(b) The grounding fact gives one proposal for all correct proposal-readies. Every proposal-ready quorum contains a correct sender, so all such quorums and the completions they trigger use that proposal.

(c) Quorums of grade-1 and grade-0 proposal-readies would contain correct senders of both grades, which the grounding fact excludes.

(d) A completing quorum contains a correct ready sender, whose grounding echo quorum contains a correct party that passed COREOK ; the eligibility contract gives $\text{Witnessed}(C)$. For a grade-1 ready quorum, choose a correct grade-1 sender and apply the same argument to its grade-1 echo quorum and TIPOK . The concrete Vantage consequence follows from Lemmas B.2 and B.3.

(e) The two quorums count $n-f$ distinct initial ready-stage authors each, so they share at least one correct party. That party's initial choice cannot be both proposal-ready and NO-READY ; a READY-mix refinement does not add another author to either quorum. $\square$

Graded Complete-Adopt relation. Reading a correct party's initial proposal-ready response as $\text{ADOPT}(v, B, g)$, $g \in \{0, \text{mix}, 1\}$, with a possible monotone update from mix to a homogeneous grade, and reading its NO-READY response as $\text{NO-ADOPT}(v)$ gives a graded analogue of BBCA's Complete-Adopt interface [26], directly from Theorem B.7 and the grounding argument above: all correct adopters name one proposal, and grade-1 and grade-0 adopters cannot coexist; a completion implies at least $f+1$ correct adopters of its proposal, while a quorum of NO-ADOPT responses excludes every completion; and each correct adopter's grounding echo quorum establishes $\text{Witnessed}(C)$ through the eligibility contract, a grade-1 adopter's likewise $\text{Witnessed}(T)$. These are names for already emitted responses, not a probe operation, and no totality is added: without the Vantage resolver, some correct parties may never complete or obtain a direct result for a Byzantine-proposer instance.

THEOREM B.8 (CONDITIONAL TERMINATION). *Let $s \geq \text{GST}$ be the first correct $\text{ENTER}(v)$ event, assume every correct party enters v by $s + 2\delta$, and assume the view proposal is broadcast no earlier than GST . Suppose $\text{Proposer}(v)$ is correct and invokes $\text{PROPOSE}(v, C, T)$ with $\text{FORMED}_v(C, T)$ no later than time $s + \delta$. Assume that at every correct party $\text{COREOK}_i(C)$ holds at some instant no later than its echo-window closure; persistence then keeps it true through the fallback action. Then every correct party fires $\text{COMPLETE}(v)$ by $s + 4\delta + \Delta \leq s + 5\Delta$, returning B_v .*

PROOF. By premise, all correct parties formally enter by $s + 2\delta$ and the sender invokes its well-formed input by $s + \delta$. No correct receiver rejects it, so the proposal is received everywhere by $s + 2\delta$ and, since formal entry activates any fixed proposal, active everywhere by $s + 2\delta$. For each correct party p_i , both $e_i(v) \leq s + 2\delta$ and its direct proposal receipt occur by $s + 2\delta$. Thus the fallback-window start t is at most $s + 2\delta$, while $e_i(v) + \theta_E \geq s + 3\Delta \geq t + \Delta$; its window therefore ends at $t + \Delta$, at least Δ after direct receipt. The assumed core check passes within every echo window, so every correct party sends a proposal echo by $s + 2\delta + \Delta$. This is no later than the earliest absolute echo deadline $s + 3\Delta$; R2's absolute ECHO-SKIP branch requires that no echoable proposal be active by that deadline, so under the core-check premise the window branch emits a proposal echo instead.

The echoes arrive everywhere by $s + 3\delta + \Delta \leq s + 4\Delta$, no later than the earliest ready deadline $s + 4\Delta$; messages delivered by a deadline are processed before its timeout branch. Every correct party therefore sends proposal-ready, and those statements arrive everywhere by $s + 4\delta + \Delta$, proving completion. $\square$

For every view beyond the cutoff of Lemma B.4(b), Vantage's WISH service supplies the entry-spread premise of Theorem B.8, and the cutoff's definition excludes a pre-GST correct proposal for that view. The R1 construction supplies a formed value, and its early wish trigger supplies the invocation premise: the wish quorum counted at the first correct entry contains $f+1$ correct wishes supporting v , sent by s and delivered to Proposer(v) by $s + \delta$ under the in-flight convention, so ω^+ supports v there and the proposer invokes by $s + \delta$ unless it already has. For the Vantage eligibility hooks, every correct-proposer core entry is exact- $n-f$ -available before the send time $\beta \geq \text{GST}$ and therefore becomes $(f+1)$ -available everywhere by $\beta + \delta$ by Lemma B.2(iv). A quorum assembled using integer claims has the same eventual property by Lemma B.2(iii), but another receiver may first have to retrieve the anchoring chain; it therefore cannot promote a correct proposer's core until exact-position support is also present. The theorem otherwise depends only on Direct AGB's environment events and message rules.

For a view with a correct proposer beyond this cutoff, none of these premises requires TipOK. If a correct party cannot verify the tip, R2 emits a grade-0 proposal echo once its fallback window closes, while R3 and R4 count proposal echoes and readies regardless of grade. Thus every correct party still completes by $s + 4\delta + \Delta$, regardless of which correct parties fail which tip checks. Completion fixes the witnessed core, and once the output cursor reaches the view and obtains its lane prefixes, it emits $\text{Core}_D(C_v)$ without waiting for the tip; only the tip may remain open (Theorem B.7 and section 6).

COROLLARY B.9 (TRANSACTION LATENCY). *Suppose every party is correct, each party places arriving transactions in a new valid data block and publishes it at once. Suppose the execution is in a non-Zeno responsive suffix beginning at a view r past the cutoff of Lemma B.4(c): every view below r has already been proposed, and sealed and fully output at every correct party, by β_r . Let β_v be the send time of B_v . A transaction arriving at time $t \geq \beta_r$ is sequenced within*

$$\begin{aligned} L_{\text{tip}} &\leq \delta + \delta + 2\delta = 4\delta, \\ &\text{and, ignoring tips,} \\ L_{\text{core}} &\leq \delta + \delta + 2\delta + \delta + 2\delta = 7\delta. \end{aligned}$$

If the next prefix-extending proposal opportunity is sent at time $t + \delta$ after processing the new block, the tip path is aligned and sequences it by $t + 3\delta$. If an anchoring proposal is sent at $t + \delta$ after processing the new block and a later prefix-extending proposal is sent at $t + 3\delta$ after processing its ECHO acknowledgment quorum, the core path is aligned and sequences it by $t + 5\delta$ through fast seal, or by $t + 6\delta$ through the Direct AGB READY path. Over any finite union of complete responsive-opportunity intervals, if the eligibility time is wall-clock-uniform and independent of the fixed proposal schedule, the expected wait for the next prefix-extending opportunity is at most $\delta/2$. Under this additional convention,

$$\mathbb{E}[L_{\text{tip}}] \leq \delta + \frac{\delta}{2} + 2\delta = 3.5\delta, \quad \mathbb{E}[L_{\text{core}}] \leq \delta + \frac{\delta}{2} + 2\delta + \delta + 2\delta = 6.5\delta.$$

PROOF. We first factor out the proposal wait. Fix either path and let $x \geq \beta_r$ be a time by which every proposer satisfies DIRECTPUB for a correct-author reference covering the block; on the core path, require in addition that every proposer has counted $n-f$ acknowledgments for that reference. Every view is eventually proposed by Lemma B.4(a) and R1, while non-Zeno execution permits only finitely many proposal sends by x . Hence there is a least $k \geq r$ with $\beta_k > x$. Since $x \geq \beta_r$, we have $k > r$. The suffix premise bounds $\beta_u \leq \beta_r \leq x$ for $u < r$, minimality bounds $\beta_u \leq x$ for

$r \leq u < k$, and Lemma B.6 gives

$$x < \beta_k \leq \max_{u < k} \beta_u + \delta \leq x + \delta.$$

The inclusion rule makes B_k cover the block. On the core path its newest exact- n - f -available core does so. On the tip path, either the core already covers it or the newest directly published lane tip does. Indeed, in this all-correct suffix every claim at its named position is exact: once a general quorum is reached for a named tip, that same tip is exact-quorum available and becomes the core candidate, so no stable-confirmation target above the core displaces the newest tip. The author's lane is unforked.

Every proposal in the all-correct suffix passes its AUTHOROK and tip-anchoring checks everywhere within one delay of its send: every named lane prefix was directly published by its correct author before the send and reaches every party within one further delay. B_k is prefix-extending by the send bounds above, so Lemma B.6 fast-seals it commit-full within 2δ of its send; thus B_k seals by $x + 3\delta$. Every lower view was sent by x — below r by the suffix premise, in $[r, k)$ by those bounds — so, by the delivery argument of that lemma, every proposal B_u with $u < k$ is fixed and, through in-order frontier processing, active at every correct party by $x + \delta$, without waiting for formal entry. For a suffix view $u \in [r, k)$ the checks pass by $\beta_u + \delta \leq x + \delta$, so every correct party's grade-1 gate holds by $x + \delta$. No fallback response preempts it: a fallback window exists only after entry, starts at $t = \max\{e_i(u), \rho_i(u)\}$, and ends no earlier than the gate time $\max\{t, \beta_u + \delta\}$ on both arms. Indeed, $t + \Delta$ dominates both t and $\beta_u + \Delta$ from $\rho_i(u) \geq \beta_u$; and $e_i(u) + \theta_E \geq \beta_u + 3\Delta - \delta$ dominates $\beta_u + \delta$ as well as $t \leq \max\{e_i(u), \beta_u + \delta\}$, using $e_i(u) \geq s_u \geq \beta_u - \delta$ — valid for every suffix view, since a correct proposer invokes no later than its own entry and the all-correct entry spread past the cutoff is δ — and $\rho_i(u) \leq \beta_u + \delta$. Hence all n matching responses for each suffix view in $[r, k)$ are sent by $x + \delta$, counted by $x + 2\delta$, and fast-seal it. The drained earlier prefix therefore makes the block sequenced by $x + 3\delta$.

For the tip path, direct publication makes the block eligible at $x = t + \delta$, giving 4δ . If a prefix-extending proposal is sent exactly at that time after processing the block, it covers the block and all lower views have already been proposed, so sequencing occurs by $t + 3\delta$.

For the core path, let $x_0 = t + \delta$, when every party has directly received the correct-author block. The first prefix-extending proposal after x_0 is sent by $x_0 + \delta$. It names a tip covering the block, and every correct party's ECHO carries the exact-position acknowledgment. Those ECHOs are counted everywhere within 2δ of that proposal's send, so the block is a core candidate everywhere by $t + 4\delta$. Applying the preceding proposal wait and fast-seal argument once more gives $t + 7\delta$. In the aligned case, the anchoring proposal is sent at $t + \delta$, its exact claims are counted by $t + 3\delta$, and a core proposal sent then fast-seals by $t + 5\delta$. The same send-to-seal theorem gives $t + 6\delta$ when the core proposal instead finishes through the three-delay READY path.

For the expected bounds, let S_j be the chronological proposal-send events that fill the least not-yet-proposed numerical view and hence extend the fully proposed prefix. Already-sent higher-view proposals may enlarge the resulting jump, but are not fresh opportunities. If the prefix reaches view q at S_j , then the responsive-pacing rule gives $\beta_{q+1} \leq \max_{u < q+1} \beta_u + \delta \leq S_j + \delta$; hence the next fresh event satisfies $g_j = S_{j+1} - S_j \leq \delta$. Conditional on the fixed schedule, a wall-clock-uniform eligibility time over any finite union of complete intervals $[S_j, S_{j+1})$ therefore gives

$$\mathbf{E}[W] = \frac{\sum_j g_j^2}{2 \sum_j g_j} \leq \frac{\delta}{2}.$$

For the tip path, adding publication and the 2δ fast-sealing time gives 3.5δ . For the core path, the wait to the first anchoring proposal has the same $\delta/2$ expectation; its ECHO census costs 2δ , the wait to the subsequent core proposal is at most δ , and fast sealing costs 2δ , giving 6.5δ . Any limiting time average, when it exists, inherits the same inequality. $\square$

Communication. Per instance, Direct AGB exchanges $O(n^2)$ messages: one proposal broadcast (n messages), one echo-stage and one initial ready-stage broadcast per party (n^2 messages each), and at most one homogeneous READY refinement per party after READY-mix (at most another n^2 messages). Thus the explicit worst-case count is $3n^2 + O(n)$, while the all-grade-1 common path has no refinement and retains $2n^2 + O(n)$. ECHO-SKIP and NO-READY replace initial statements and never add any. In Vantage, responses piggyback the sender’s current WISH; threshold amplification adds at most one own WISH broadcast per raised view, within the same $O(n^2)$ per-view message bound. Fast sealing only reads the existing ECHO set and adds no message; its lock is one payload identifier plus the already maintained first-response counts. Completed-open quarantine adds one reference and two scalar counters per author and no message; its probes reuse ordinary proposals. The grounded skip shortcut adds at most one short SKIP-VOTE broadcast per party on an eligible failed view, another $O(n^2)$ messages in that view and none on the all-correct favorable path. Its per-target stance is one persistent trit. Thus Vantage’s complete per-view statement layer remains $O(n^2)$ messages. In Vantage’s proved by-value instantiation, manifests are carried by value and well-formedness bounds each at n entries — for Byzantine proposals too, since correct parties never echo a malformed proposal — so every statement is $O(n\lambda)$ bits and a view costs $O(n^3\lambda)$ bits, including the proposal-relative ACK fields.¹ The default implementation names a single proposal by hash in later statements and puts an $O(n)$ -bit availability bitmap on each ECHO. Its aligned common case is therefore $O(n^2\lambda + n^3)$ bits per view; sparse (position, distance) exceptions cost $O(n^3(\log n + \log \nu))$ bits. Appendix F.3 gives the accounting. Digest-naming is a wire optimization; the ECHO acknowledgment semantics are part of the protocol. Data dissemination is accounted separately: a data block of $\ell_b \leq \ell_{\max}$ payload bits occupies $\ell_b + O(\lambda)$ bits on the wire and costs $O(n(\ell_b + \lambda))$ for its author’s typed publication; acknowledgments add no standalone message and were counted in the ECHO layer above. Thus blocks of size $\ell_b = \Omega(n\lambda)$ cost $O(n)$ bits of communication per payload bit — the cost of n -fold replication itself. Repair is off the common path and absent with correct authors: a repairing party spends $O(n\lambda)$ on requests per distinct authorized missing block, including ancestors discovered while walking a named root, and, under the deduplicated typed-repair response rule, may receive up to $O(n(\ell_b + \lambda))$ per block — $O(n^2(\ell_b + \lambda))$ across all parties in the worst case. Counted availability claims also authorize roots, but a view counts one echo-stage response per sender, each naming at most $2n$ references, which bounds the roots a Byzantine sender can authorize per view. Repairing a lane prefix sums these per-block costs over its missing ancestors. Coded dissemination would reduce this and is orthogonal.

The resolution plane of Section 6.1 is accounted separately. For one target and resolver view, WISH synchronization, suggestions and proofs, fresh ECHOs and witnesses, the IT-HS phases, and DONE cost $O(n^2)$ messages. A full/core value contains two $O(n\lambda)$ manifests; because witnesses and DONE attach that value, the proved by-value cost is $O(n^3\lambda)$ bits, excluding lane-prefix repair. Skip values are shorter. Bounded value-body requests to counted phase authors add $O(n^2)$ messages and $O(n^3\lambda)$ bits per resolver view in the worst case, preserving both bounds. Different targets run concurrently, so k overlapping active targets multiply both bounds by k . The plane is absent from the logical dependency chain of direct completion and sealing, not from network and processor use while resolution is active.

C RESOLUTION AND OUTPUT: FULL SPECIFICATION AND PROOFS

Resolver adapter. Resolution is composition state outside Direct AGB. For each target data-plane view u , the adapter may receive a direct-resolver decision: a bounded value R (below) whose submitted outcome X is commit-full(C, T),

¹Scalar fields — view numbers, heights, lengths — cost $O(\log \nu)$ bits, where ν bounds those quantities in the execution prefix. The displayed bounds use the deployment regime $\log \nu = O(\lambda)$; otherwise each $O(\lambda)$ scalar term reads $O(\lambda + \log \nu)$. This accounting convention does not bound the unbounded-execution liveness model.

commit-core(C), or skip. A non-skip value retains its exact backing payload (C, T); the core outcome omits T from output but keeps it in the value's identity. The adapter authorizes the named lane roots, submits X to the caller-owned TRY-SEAL($u, \cdot$) arbiter, and fires SEAL(u) $\rightarrow X$ if no compatible terminal result was already stored. Direct AGB, fast-seal, grounded skip, and resolver submissions race only at this arbiter. The first terminal result is persistent and later compatible submissions are idempotent.

A local terminal seal does not remove a party from resolver participation. If such a party receives a peer resolver WISH for u , it allocates the target-local instance, relays its own WISH, and may fresh-ECHO only a resolver value whose submitted outcome equals that seal. This rule is needed when a direct or skip seal was selectively delivered: the parties still missing it may need every correct participant. A party without either target-local evidence or a local terminal seal instead requires WISHes from $f + 1$ distinct members before it allocates an instance, so one Byzantine member cannot activate arbitrary target phases. Upon each direct WISH, a correct party also sends that requester its current own WISH watermark and, once per requester and coordinate, every applicable own retained SUGGEST, PROOF, PROPOSE, phase statement, and witness through the requested watermark plus one, together with its own retained DONE, if any. Thus traffic sent before a late activation is not lost even though unknown-target phase traffic is otherwise discarded.

Values and local candidates. For target u , write

$$R_u^{\text{full}}(C, T), \quad R_u^{\text{core}}(C, T), \quad R_u^{\text{skip}}$$

for the three bounded resolver values. Their submitted outcomes are, respectively, commit-full(C, T), commit-core(C), and skip. The second value keeps T so that full and core values derived from the same target proposal have an unambiguous identity. The identifier $\mathcal{H}(R)$ abbreviates a domain-separated digest that binds the session, target, outcome kind, and complete value.

Let $h_i = \max\{a_i + 1, \omega_i^+\}$ be p_i 's local data-plane proposal horizon. Once $h_i \geq u + 3$, each party repeatedly derives a monotone candidate set from its first-hand target records; a party that has terminally sealed u does not start an instance from them but joins on a peer WISH. Every candidate requires $Q = n - f$ initial ready-stage responses. In addition:

- (C1) $R_u^{\text{full}}(C, T)$ requires $f + 1$ matching grade-1 target ECHOs for (C, T);
- (C2) $R_u^{\text{core}}(C, T)$ requires Q ready-stage responses whose first counted form is not grade 1, and $f + 1$ matching target ECHOs of either grade for (C, T);
- (C3) R_u^{skip} requires Q NO-READY responses.

Only the first authenticated target ECHO and initial READY from each sender are counted. A READY-mix counted first remains historical non-grade-1 candidate evidence after a permitted same-proposal homogeneous refinement. The sender's own provisional MIX nevertheless cannot pass fresh acceptance until it closes. Because $f + 1$ matching ECHOs for distinct payloads use disjoint first-response authors, a local candidate set contains at most $2\lfloor n/(f + 1) \rfloor + 1$ values. Its canonical order is payload order, full before core for the same payload, and skip last.

Fresh acceptance is deliberately stronger than candidacy. Immediately before sending a resolver ECHO for a zero-key proposal, p_i checks:

- (F1) its own immutable echo-stage response exists and its own initial READY state is closed;
- (F2) full is rejected by an own grade-0 READY or a different ready payload; core is rejected by an own grade-1 READY or a different ready payload; skip requires the own response NO-READY;

- (F3) full has locally available C, T and passes the paired-tip ancestry check; core has locally available C ;
- (F4) the value is compatible with any local terminal seal and active optimistic lock; and
- (F5) before a non-skip ECHO, the persistent stance changes atomically from free to non-skip, or is already non-skip; a skip-voted stance rejects it.

A fresh full ECHO carries origin bit 1 only if its sender previously emitted the exact matching grade-1 target ECHO. A fresh core ECHO carries origin bit 1 after a matching target ECHO of either grade. Skip carries no origin bit. A fresh ECHO quorum for a non-skip value is usable only with at least $f + 1$ origin-1 ECHOs. Every resolver ECHO carries the exact proposal key q and is counted at coordinate (u, r, q, x) . Thus $q = 0$ ECHOs that executed these fresh checks never combine with $q > 0$ ECHOs for a carried Backed value, even when both proposals name the same digest x .

Stable backing. After a usable quorum of resolver ECHOs for $(u, r, 0, x)$, a correct party broadcasts one witness $\langle \text{RESOLUTION-WITNESS}, u, r, x, R \rangle$, attaching the bounded value with $\mathcal{H}(R) = x$. A party that has not sent that witness relays it after counting $f + 1$ matching witnesses. It sets $\text{BACKED}_i(u, x)$ after counting Q matching witnesses. A positive-key view-change suggestion also attaches R ; a receiver accepts the suggestion only after checking its digest, well-formedness, IT-HS key proof, and $\text{BACKED}_i(u, x)$.

LEMMA C.1 (STABLE BACKING). *If a correct party sets $\text{BACKED}(u, x)$, then a usable fresh resolver ECHO quorum for one value R with $\mathcal{H}(R) = x$ occurred, at least one correct party permanently holds R , and every correct party that participates in the instance eventually sets the same Backed predicate and obtains R . No correct witness can support two values at one (u, r) .*

PROOF. Consider the causally first correct witness for (u, r, x) . It cannot be a relay: among the $f + 1$ witnesses required for relay, one is correct and would precede it. It therefore followed the usable fresh ECHO quorum and attached the value it retained. Correct witnesses for two different values at the same (u, r) would each descend from a fresh ECHO quorum; those quorums intersect in a correct one-ECHO sender, a contradiction. A correct party that counts Q witnesses counts at least $Q - f = n - 2f \geq f + 1$ correct senders. Reliable delivery brings those witnesses and their attached value to every correct participant, causing each to relay. If a witness preceded a party's activation, that party's WISH makes each correct witness sender replay its own witness directly; the same $f + 1$ relay then starts at the late party. The at least $n - f = Q$ correct parties therefore supply a witness quorum everywhere. Per-coordinate first-send and per-requester replay rules give the final claim. $\square$

C.1 Resolved AGB Contract

Direct AGB intentionally leaves residual mixed-grade and silent instances unsealed. A resolver is a composition service with the following per-target contract. Every non-skip result also supplies the exact backing payload (C, T) .

- RS1. Single choice.** Resolver decisions at correct parties for target u name one exact value and outcome.
- RS2. Future-closed direct compatibility.** A full result for (C, T) excludes every grade-0 READY quorum and every READY quorum for a different payload. A core result backed by (C, T) excludes every grade-1 READY quorum and every READY quorum for a different payload. A skip result excludes every READY quorum.

RS3. Output validity.

A core result satisfies $\text{Witnessed}(C)$; a full result satisfies both $\text{Witnessed}(C)$ and $\text{Witnessed}(T)$.

For termination we use one additional property:

RL1. Per-target totality. For every target u , one outcome X_u eventually reaches every correct party that lacks a terminal seal; every party already terminally sealed at u holds that same compatible outcome.

This service has agreement power; the definition does not claim that AGB implements it.

THEOREM C.2 (RESOLVED AGB COMPOSITION). *Direct AGB composed with any resolver satisfying RS1–RS3 has at most one terminal outcome at each correct party, and no two correct parties terminally seal different outcomes for a target u . If completion exists, every terminal outcome is non-skip, is backed by the completion’s unique payload, and contains its core. Every non-skip outcome has the witness promised by RS3 or by the direct path. If RL1 also holds, every correct party eventually terminally seals u .*

PROOF. The caller-owned arbiter gives local uniqueness. Two direct results agree by Theorem B.7(b)–(c). Two resolver results agree by RS1, and a direct result agrees with a resolver result by RS2 and AGB value agreement. A completion has a READY quorum, so RS2 excludes skip and binds every non-skip resolution to its payload. Its core is fixed by Theorem B.7(b),(d), and the value-domain expansion contract makes $\text{Core}_D(C)$ a prefix of $\text{Full}_D(C, T)$ for every prior-output set D . Direct outcomes obtain their witness from Theorem B.7(d); resolver outcomes obtain it from RS3. RL1 supplies the common outcome to every party that did not already seal it. $\square$

Ordered output. The cursor processes data-plane views in increasing order. At a locally completed but open cursor view, it retrieves and verifies the core prefixes, emits the canonical core expansion once, and stops before the tip. A full seal then retrieves and emits the canonical tip expansion before advancing; a core seal advances without the tip; a skip seal emits nothing. A resolver decision received before local completion supplies the same manifests. Late compatible completion or sealing is idempotent. Later AGB and resolver instances continue while the cursor is stopped, but no payload from a later view crosses that position.

Call a sequence of instances *input-fair* when every admissible item from a correct source eventually occurs in the core of some completed instance.

COROLLARY C.3 (ATOMIC BROADCAST FROM RESOLVED AGB). *Run one Direct AGB instance per increasing data-plane view, compose each with a resolver satisfying RS1–RS3 and RL1, and assume input fairness and fair repair. Under the cursor above, deterministic view-order expansion, and first-occurrence deduplication, correct output sequences are prefix-comparable, have one common limit, and eventually contain every finite prefix of that limit. Every output item is emitted at most once, satisfies external validity, is witnessed as required by Direct AGB or RS3, and is retrieved before local output. Every admissible correct-source item is eventually output. Applying a deterministic transition function implements state-machine replication.*

PROOF. By Theorem C.2, all correct parties use one outcome per view, and a completed open core is a prefix of either compatible non-skip outcome. The cursor therefore exposes only prefixes of one deterministic view-order expansion. RL1 advances every cursor through every finite set of views; RS3, Direct AGB availability, and fair repair provide the referenced prefixes before output. Deduplication gives at-most-once output, and input fairness gives correct-input inclusion. Equal input prefixes and a deterministic transition function give equal state prefixes. $\square$

D CONCURRENT TARGET-LOCAL RESOLVER: CONSTRUCTION AND PROOFS

Each unresolved target u owns an independent sequence of resolver views $r = 1, 2, \dots$. Resolver view 1 starts with the committee member immediately after $\text{Proposer}(u)$; subsequent views rotate through the committee. The wire format

binds each message's type and applicable target, resolver view, and value digest; a resolver ECHO additionally binds the proposal key q . The authenticated connection binds its direct sender. For each target and resolver view, parties count only the first statement from each sender of each message type. A quorum additionally requires equality of the full typed coordinate; in particular, ECHOs with different proposal keys never combine. Parties count phase traffic at most one resolver view above their own WISH watermark and act on a view's proposal, ECHO, KEY, and LOCK quorums only while that view is current, a quorum counted before entry firing on entry; witness generation and relay, Backed, WISH, and DONE relay and decision are view-independent.

WISH uses the same high-watermark pattern as the data pacemaker, independently for each target: $f + 1$ WISHes amplify the own watermark and Q enter all missing views through that watermark. A locally justified initial WISH is retransmitted every 5Δ until entry. On entering view r , a party sends the primary its KEY3/KEY2 suggestion, broadcasts its KEY1 proof, and arms both a 5Δ no-proposal timer and an 11Δ full-view timer. Either timer raises the own WISH to $r + 1$ when its condition still applies. Each new WISH also elicits the responder's current own WISH and at most one direct replay per coordinate of every applicable own retained view-scoped send through $r + 1$ and of any own DONE. This instantiates IT-HS's send-upon-join discipline under our non-FIFO channels.

The resolver then uses the information-theoretic HotStuff view-change core of Abraham et al. [3]. KEY1 supplies lock-opening proofs, KEY2 supplies ACCEPTKEY proofs, and KEY3 is the candidate carried across views. A positive suggestion attaches its value body. The primary waits for Q accepted suggestions and re-proposes a maximum positive key, breaking ties by digest. If every accepted key is zero, it takes the next value of its candidate set cyclically under a persistent local cursor and advances that cursor; with no candidate it proposes nothing and the view times out. The cursor moves on the party's own primary turns, rather than being indexed by the global resolver view, so a common divisor between committee size and candidate count cannot skip a candidate forever.

A zero-key proposal is fresh and invokes F1–F5 immediately before ECHO. A positive-key proposal must be Backed. The ECHO binds that exact key, and only ECHOs matching the receiver's stored (u, r, q, x) proposal count together. In either case, the IT-HS lock accepts the locked value or a proposal whose key and $f + 1$ lock-opening proofs satisfy OPENLOCK. A syntactically valid designated-primary proposal is stored even while these guards are false: its arrival stops the shorter no-proposal timer, while the full-view timer remains armed and the proposal is rechecked when proofs, backing, or target data change or its view becomes current. After Q matching ECHOs for one proposal coordinate, a carried value advances directly; a fresh value first executes the witness step and advances only after it is Backed. Q matching KEY1, KEY2, KEY3, and LOCK statements advance the successive phases. When a new KEY1 value differs from x_1 , only then does *prev1* take the old KEY1 view; KEY2 updates *prev2* by the same conditional rule. Q LOCKs emit DONE. DONE has an $f + 1$ relay and a Q decision threshold. The complete executable rules are in Algorithms 5 and 6.

Every digest quorum is inert until the party holds a well-formed value body with the named digest. A missing body is requested only from authors of the counted statements; a reply is accepted only for a pending target/digest and creates no protocol statement. This repair rule handles non-FIFO body/phase delivery and does not weaken the first-hand thresholds.

LEMMA D.1 (FRESH EXTERNAL VALIDITY). *If a usable fresh resolver ECHO quorum exists for target u , its value has the following properties.*

- (i) *A full value backed by (C, T) excludes every grade-0 target READY quorum and every target READY quorum for a different payload; C, T are retrievable and author-backed.*

Algorithm 5: Target-local resolver activation and stable external validity at party p_i for target u ($Q = n - f$)

```

/* For each  $(u, r)$ , count only the first authenticated statement from a sender of each message type. Quorums match the full typed
coordinate: a resolver ECHO uses  $(u, r, q, x)$ , including the exact proposal key  $q$ , and a later phase uses  $(u, r, x)$ .  $\mathcal{H}(X)$  abbreviates
the domain-separated digest of a bounded value  $X \in \{R_u^{\text{full}}(C, T), R_u^{\text{core}}(C, T), R_u^{\text{skip}}\}$ , whose submitted outcomes are commit-full( $C, T$ ),
commit-core( $C$ ), and skip; the core value keeps  $T$  only in its identity. */

Global variables and constants:
┌ monotone candidate set  $C_i[u]$  and persistent next-candidate pointer  $next_i[u]$ ; resolver WISH high-watermarks  $wish_i[u, j]$  and own watermark  $ownWish_i[u]$ ;
└  $BACKED_i(u, x) \leftarrow \text{false}$ ; persistent AGB stance  $z_i(u) \in \{\text{free, non-skip, skip-voted}\}$ 

upon the data-plane proposal horizon reaches  $u + 3$ , or target evidence changes thereafter do
┌ add every locally justified value to  $C_i[u]$  in canonical order: every value needs  $Q$  initial ready-stage responses for  $u$ ;  $R_u^{\text{full}}(C, T)$  also needs  $f + 1$  matching
  grade-1 ECHOs;  $R_u^{\text{core}}(C, T)$  needs  $Q$  ready-stage responses whose first counted form was not grade 1 and  $f + 1$  matching ECHOs of either grade;  $R_u^{\text{skip}}$  needs
   $Q$  NO-READY responses; if  $C_i[u] \neq \emptyset$  and no instance, decision, or terminal seal for  $u$  exists then
  ┌ allocate the instance and invoke  $\text{RAISEWISH}(u, 1)$ 

procedure  $\text{RAISEWISH}(u, r)$ 
┌ if an instance exists, no decision exists, and  $r > ownWish_i[u]$  then
  ┌ persist  $ownWish_i[u] \leftarrow r$ ; broadcast  $(\text{RESOLUTION-WISH}, u, r)$ ; if no resolver view has been entered then arm a  $5\Delta$  retransmission timer

upon a direct  $(\text{RESOLUTION-WISH}, u, r)$  from  $p_j$  do
┌ if the resolver already decided  $(x, X)$  then
  ┌ send the retained  $(\text{DONE}, u, x, X)$  to  $p_j$  once
┌ else
  ┌ record  $wish_i[u, j] \leftarrow \max\{wish_i[u, j], r\}$  /* one watermark per sender, kept before allocation */
  ┌ if no instance exists then
    ┌ if a retained local terminal seal exists then
      ┌ allocate the instance and invoke  $\text{RAISEWISH}(u, 1)$ 
    ┌ else
      ┌ if  $f + 1$  senders have watermarks then allocate the instance
  ┌ if an instance exists then
    ┌ if  $f + 1$  WISH high-watermarks support  $w > ownWish_i[u]$  then
      ┌ invoke  $\text{RAISEWISH}(u, w)$ 
    ┌ enter every missing view through the largest  $w$  supported by  $Q$  WISHes; send  $p_j$  the current own WISH and, once per requester and coordinate, each
      applicable own SUGGEST, PROOF, PROPOSE, phase statement, and witness through  $r + 1$ , plus the own DONE, if any

upon the initial-WISH timer expires before any resolver view is entered do
┌ rebroadcast the own WISH and re-arm the  $5\Delta$  timer

function  $\text{FRESHOK}(u, X)$ :
┌ require a well-formed value for  $u$ , the own immutable echo-stage response, and a closed own initial READY state
┌ for  $R_u^{\text{full}}(C, T)$ , reject an own grade-0 READY or a different own READY payload; require locally available  $C, T$  and the tip-anchoring check
┌ for  $R_u^{\text{core}}(C, T)$ , reject an own grade-1 READY or a different own READY payload; require locally available  $C$ 
┌ for  $R_u^{\text{skip}}$ , require the own response NO-READY
┌ require compatibility with any terminal seal and active optimistic lock
┌ before a non-skip acceptance, atomically claim  $z_i(u) : \text{free} \rightarrow \text{non-skip}$  or keep non-skip; reject skip-voted
┌ return origin bit 1 for a full value after an exact matching grade-1 target ECHO, and for a core value after a matching target ECHO of either grade; return bit 0 for
  another non-skip value and no bit for skip

upon  $Q$  resolver ECHOs for the fresh coordinate  $(u, r, 0, x)$ , a held value  $X$  with  $\mathcal{H}(X) = x$  (requested from counted senders if missing), and, when  $X$  is non-skip, at least
   $f + 1$  origin-1 ECHOs among them; in any resolver view do
┌ retain  $X$  and broadcast  $(\text{RESOLUTION-WITNESS}, u, r, x, X)$ 

upon  $f + 1$  matching witnesses for  $(u, r, x)$ , before sending one do
┌ retain the attached verifying  $X$  and broadcast the own witness

upon  $Q$  matching witnesses for  $(u, r, x)$  do
┌ latch  $BACKED_i(u, x) \leftarrow \text{true}$  and retry the current phase

```

(ii) A core value backed by (C, T) excludes every grade-1 target READY quorum and every target READY quorum for a different payload; C is retrievable and author-backed.

(iii) A skip value excludes every target READY quorum.

These claims remain true when the value is carried into later resolver views.

PROOF. Let E be the fresh resolver ECHO quorum. Any target READY quorum intersects E in at least $n - 2f \geq f + 1$ parties and therefore in a correct party. That party evaluated its own closed immutable target history in F1–F2. For full, it excludes own grade 0, a different payload, and any later refinement; for core it analogously excludes own grade 1 and a different payload; for skip it requires its own NO-READY. The corresponding conflicting READY quorum is therefore impossible.

Algorithm 6: One target's WISH/IT-HS views and decision at party p_i

```

/* SUGGEST, PROOF, PROPOSE, ECHO, KEY, and LOCK statements are counted through one view above  $ownWish_i[u]$ . The ECHO send and the ECHO,
KEY, and LOCK quorum rules act only while their view is current; a quorum already counted at entry fires on entry. Witness generation
and relay, Backed, WISH, and DONE relay and decision are view-independent. */
Global variables and constants:
   $(key1, x_1, prev1)$ ,  $(key2, x_2, prev2)$ ,  $(key3, x_3)$ , and  $(lock, x_L)$ , initially zero; one PROPOSE as primary, one sent ECHO, and one statement of each later phase
  per resolver view; at most one witness per  $(u, r)$  and one DONE per target
upon entering target-local resolver view  $r$  do
  | send the primary  $\langle \text{SUGGEST}, u, r, key3, x_3, key2, x_2, prev2, X_3 \rangle$ , where  $X_3 = \perp$  iff  $key3 = 0$  and otherwise  $\mathcal{H}(X_3) = x_3$ ; broadcast
  |  $\langle \text{PROOF}, u, r, key1, x_1, prev1 \rangle$ ; arm a  $5\Delta$  no-proposal timer and an  $11\Delta$  full-view timer
function ACCEPTKEY( $q, x$ ):
  | return  $q = 0$ , or  $f + 1$  counted suggestions with KEY2 history  $(k_2, y_2, p_2)$  satisfy  $p_2 < k_2 < r$  and either  $q \leq p_2$  or  $(q \leq k_2 \wedge x = y_2)$ 
function OPENLOCK():
  | return  $f + 1$  counted proofs with KEY1 history  $(k_1, y_1, p_1)$  satisfy  $p_1 < k_1 < r$  and either  $lock \leq p_1$  or  $(lock \leq k_1 \wedge y_1 \neq x_L)$ 
/* Accept a zero-key suggestion directly. Accept a positive suggestion only when  $0 < q < r$ , ACCEPTKEY( $q, x$ ), BACKED $_i(u, x)$ , and its
attached  $X$  verifies with  $\mathcal{H}(X) = x$ . */
upon  $p_i$  is the primary of  $(u, r)$  and has  $Q$  accepted suggestions do
  | choose one with maximum positive key view, breaking ties by digest; if that key is positive then
  | | broadcast  $\langle \text{PROPOSE}, u, r, q, x, X \rangle$  with its attached  $(q, x, X)$ 
  | else if  $C_i[u] \neq \emptyset$  then
  | | take  $X$  at position  $next_i[u] \bmod |C_i[u]|$  of canonical  $C_i[u]$  and advance the persistent pointer; broadcast  $\langle \text{PROPOSE}, u, r, 0, \mathcal{H}(X), X \rangle$ 
  | else
  | | propose nothing in this view /* the view times out */
upon the first designated-primary proposal  $(u, r, q, x, X)$  with  $q < r$  and  $\mathcal{H}(X) = x$  do
  | store it, stopping only the no-proposal timer
upon  $r$  is current, a proposal  $(u, r, q, x, X)$  is stored, no ECHO was sent in  $r$ , and either  $lock = 0$ ,  $x = x_L$ , or  $(r > q \geq lock \wedge OPENLOCK())$ ; evaluated whenever proofs,
backing, target data, or the current view change do
  | if  $q = 0$  then
  | | require FRESHOK( $u, X$ ) and attach its origin bit
  | else
  | | require BACKED $_i(u, x)$ ; attach no origin bit
  | broadcast  $\langle \text{ECHO}, u, r, q, x, origin \rangle$ 
/* Act on a digest quorum only while holding a verifying  $X$ ; if it is missing, request it from counted senders and accept only the
pending digest. */
upon  $Q$  ECHOs matching the stored proposal coordinate  $(u, r, q, x)$  do
  | if  $q = 0$ , wait until the witness rule of Algorithm 5 makes  $x$  Backed; if  $x \neq x_1$  then
  | |  $prev1 \leftarrow key1$  and  $x_1 \leftarrow x$ 
  |  $key1 \leftarrow r$ ; broadcast  $\langle \text{KEY1}, u, r, x \rangle$ 
upon  $Q$  matching KEY1s for  $(u, r, x)$  do
  | if  $x \neq x_2$  then
  | |  $prev2 \leftarrow key2$  and  $x_2 \leftarrow x$ 
  |  $key2 \leftarrow r$ ; broadcast  $\langle \text{KEY2}, u, r, x \rangle$ 
upon  $Q$  matching KEY2s for  $(u, r, x)$  do
  |  $(key3, x_3) \leftarrow (r, x)$ ; broadcast  $\langle \text{KEY3}, u, r, x \rangle$ 
upon  $Q$  matching KEY3s for  $(u, r, x)$  do
  |  $(lock, x_L) \leftarrow (r, x)$ ; broadcast  $\langle \text{LOCK}, u, r, x \rangle$ 
upon  $Q$  matching LOCKs for  $(u, r, x)$  and a verifying  $X$  is held do
  | broadcast  $\langle \text{DONE}, u, x, X \rangle$ 
upon  $f + 1$  matching DONEs for  $(u, x)$  before sending one do
  | retain a verifying attached  $X$  and broadcast the own DONE
upon  $Q$  matching DONEs for  $(u, x)$  do
  | decide  $X$ , retain the decision, and submit its outcome to TRY-SEAL( $u, \cdot$ )
upon the no-proposal timer expires with no stored designated-primary proposal, or the full-view timer expires before a decision do
  | invoke RAISEWISH( $u, r + 1$ )

```

Every correct ECHOer checked the required local prefixes, so at least one correct permanent holder exists. A usable non-skip quorum also contains $f + 1$ origin-1 ECHOs, at least one from a correct party. In the full case that party's matching grade-1 target ECHO checked AUTHOROK for CUT and tip ancestry; in the core case its matching target ECHO checked AUTHOROK for C. The author-witness and availability lemmas give publication provenance and retrievability. Carrying changes no target record or ECHO fact, and Lemma C.1 binds the later value to this fresh quorum. $\square$

LEMMA D.2 (FAST-SEAL COMPATIBILITY). *For a proposal $B = (C, T)$, any two correct fast seals name the same payload. A correct FAST-SEAL(v) $\rightarrow$ commit-full(C, T) is compatible with every Direct AGB result and every resolver decision for v , and C, T are witnessed.*

PROOF. A fast observer counted the unique response of every author as a matching response for B , a grade-1 ECHO for exactly B . Hence all correct parties emitted that response and installed the optimistic lock for B . Two all- n censuses cannot name different proposals. Quorum intersection excludes a grade-0 or different-payload READY quorum, so direct results are compatible, and the fast observer's checks witness C, T .

Suppose an incompatible fresh resolver ECHO quorum existed. Its intersection with the correct parties contains a correct sender. That sender's optimistic lock could accept the incompatible value only after $f + 1$ nonmatching target responses released it. At least one such response is correct and also occurs in the all- n census, contradicting that every response there is a matching response for B . By Lemma C.1, every carried or decided resolver value descends from a fresh quorum, so it is compatible as well. $\square$

LEMMA D.3 (GROUNDED SKIP COMPATIBILITY). *For every target u :*

- (i) *if a correct party broadcasts $\text{SKIP-VOTE}(u)$, no correct party ever broadcasts a proposal-READY for u ;*
- (ii) *a quorum of skip votes and a usable fresh non-skip resolver ECHO quorum cannot both exist; and*
- (iii) *every correct $\text{SKIP-SEAL}(u) \rightarrow \text{skip}$ is compatible with every direct, fast, or resolver submission.*

PROOF. Let $b \leq f$ be the actual number of Byzantine parties. A correct skip voter counted Q target ECHO-SKIP responses, including at least $n - f - b$ correct echo-skippers. A correct proposal-READY sender would have a grounding ECHO quorum with at least $n - f - b$ correct proposal echoers. The two correct sets are disjoint, yet

$$2(n - f - b) > n - b$$

because the margin is $n - 2f - b \geq f + 1 - b > 0$. Thus no correct proposal-READY, completion, or direct result exists; an ECHO-SKIP also excludes the all- n fast census.

A skip-vote quorum and a fresh non-skip resolver ECHO quorum intersect in at least $n - 2f \geq f + 1$ parties, including a correct one. Before the vote that party persistently claimed skip-voted; before the ECHO it instead claimed non-skip. The claims are exclusive in either temporal order. A carried resolver value has a Backed fresh quorum by Lemma C.1, so the same argument applies to it and proves compatibility. $\square$

LEMMA D.4 (ONE VALUE PER RESOLVER VIEW). *Fix a target u and resolver view r . If two correct parties send any statements among KEY1 , KEY2 , KEY3 , and LOCK in view r , then the statements name the same value.*

PROOF. A correct KEY1 sender counted Q ECHOs for one exact proposal coordinate (u, r, q, x) . Two such ECHO quorums intersect in at least $2Q - n = n - 2f \geq f + 1$ parties and hence in a correct party. A correct party sends at most one ECHO in a resolver view, so two correct KEY1 statements in that view have the same value. Induct on the remaining phases. A correct sender of the next phase counted Q matching statements of the preceding phase; any two such quorums again intersect in a correct one-shot sender, and every such statement agrees with the already unique value. This gives the claim through LOCK. $\square$

LEMMA D.5 (PERSISTENT LOCK SAFETY). *Suppose a correct party sends DONE for x from a quorum of LOCK statements in resolver view r . No correct party sends KEY1 for $y \neq x$ in a later view. Consequently, all correct DONE statements, including relays, name one value, and no two correct parties decide differently.*

PROOF. The LOCK quorum contains at least $Q - f = n - 2f \geq f + 1$ correct senders. Fix a set L of $f + 1$ of them; each member of L held lock (r, x) when it sent its LOCK statement.

Assume for contradiction that $s > r$ is the smallest view in which a correct party sends KEY1 for some $y \neq x$, and choose the causally first such KEY1 in view s . Its sender counted an ECHO quorum E for y ; every ECHO in E therefore precedes that chosen KEY1. Because $|E| = Q$ and $|L| = f + 1$, $E \cap L$ is nonempty; let p_h be a party in the intersection. Since p_h sent its LOCK while r was current and its ECHO after entering $s > r$, before that ECHO it still has an x -valued lock at some view at least r . Indeed, changing that lock to a different value in an intermediate view would require a correct KEY1 for that value in the same view by Lemma D.4, contradicting the minimality of s . It cannot first change the lock within view s before this ECHO either: a correct LOCK for the new value causally follows a correct KEY1 for that value, which would precede the chosen KEY1.

Thus p_h can ECHO y only by counting $f + 1$ proofs that satisfy OPENLOCK; one proof sender p_j is correct. Write its proof as (k_1, z, p_1) , where $p_1 < k_1 < s$. If p_h 's lock view is at most k_1 and $z \neq x$, then p_j sent a non- x KEY1 at view $k_1 \geq r$. Otherwise the accepted proof has the lock view at most p_1 . By the conditional previous-key update, p_j sent KEY1 at both p_1 and k_1 with different values, so one of them is non- x . In either case a correct non- x KEY1 occurs in a view between r and s : equality with r contradicts Lemma D.4, and a larger view contradicts the minimality of s .

It follows that phase-originated correct DONEs in different views agree: take the smaller view and apply the first part, while equal views use Lemma D.4. A causally first correct DONE for any other value cannot be a relay, because its $f + 1$ triggering DONEs include an earlier correct sender. Hence relays introduce no new value. Finally, every Q -DONE decision quorum contains a correct sender, so all correct decisions have that unique value. $\square$

LEMMA D.6 (PERSISTENT KEY SUPPORT). *If a correct party sets KEY3 to (k, x) with $k > 0$, then at least $f + 1$ correct parties' suggestions in every later view support ACCEPTKEY(k, x). Once stable backing and the attached values have reached the correct participants, a correct primary eventually accepts every correct suggestion. Moreover, if some correct party has a lock at view $\ell > 0$, every set of Q accepted suggestions in a later view contains a positive KEY3 view at least ℓ .*

PROOF. Acting only in the current view keeps a party's KEY1, KEY2, KEY3, and lock views nondecreasing. Setting KEY3 to (k, x) requires Q KEY2 statements for (k, x) , including at least $f + 1$ from correct parties. Immediately afterward, each such sender has either $k_2 \geq k$ with $x_2 = x$, or already has $p_2 \geq k$. A later KEY2 with the same value preserves the first condition. At the first change of value, the conditional update sets p_2 to the previous $k_2 \geq k$, after which the second condition is permanent. Their future suggestions therefore continue to support ACCEPTKEY(k, x).

A zero-key correct suggestion is accepted directly. A positive correct suggestion carries its retained value; the $f + 1$ correct supporters above eventually reach the primary, and stable backing eventually holds there, so the positive suggestion is accepted as well.

Finally, a correct lock at ℓ followed a quorum of KEY3 statements in view ℓ . At least $n - 2f$ of those senders are correct and thereafter report KEY3 views at least ℓ . Any set of $Q = n - f$ senders intersects this set because its complement has at most $2f < Q$. Hence a Q -suggestion set contains a positive key at least ℓ . $\square$

LEMMA D.7 (PERSISTENT LOCK-OPENING SUPPORT). *In resolver view r , let a correct primary select a maximum positive accepted suggestion (q, x) . For every correct lock (ℓ, z) , $q \geq \ell$; if $x \neq z$, at least $f + 1$ correct proofs eventually satisfy OPENLOCK for that lock.*

PROOF. The inequality $q \geq \ell$ follows from the final part of Lemma D.6. Because (q, x) was accepted, the primary counted $f + 1$ supporting KEY2-history tuples. At least one belongs to a correct party p_j . Its tuple satisfies either $p_2 \geq q$, or $k_2 \geq q$ with $x_2 = x$. Since $q \geq \ell$ and $x \neq z$, p_j therefore has either $p_2 \geq \ell$, or $k_2 \geq \ell$ with $x_2 \neq z$.

In the second case, when p_j set (k_2, x_2) it counted a KEY1 quorum, including $f + 1$ correct KEY1 senders at view $k_2 \geq \ell$ with value different from z . In the first case, the conditional previous-key update records two KEY2 views $p_2 < k_2$ with different values. At least one of those values differs from z ; the corresponding KEY1 quorum again supplies $f + 1$ correct senders at a view at least ℓ with a non- z value.

For each of these correct KEY1 senders, the condition needed by OPENLOCK persists. Repeating the same value leaves $k_1 \geq \ell$ and $x_1 \neq z$; changing value sets p_1 to the old $k_1 \geq \ell$, after which $p_1 \geq \ell$ remains true. These parties broadcast their proof tuples on entering r , and send-upon-join replays them if needed. Thus every correct party with lock (ℓ, z) eventually counts the required $f + 1$ proofs. $\square$

LEMMA D.8 (ONE TIMELY RESOLVER VIEW). *Suppose the first correct entry to target-local view r occurs at $t \geq \text{GST}$, all correct parties enter by $t + 2\delta$, the primary is correct, and stable backing and attached values have reached all correct participants. Suppose either that the primary selects a positive key, or that all accepted keys are zero and its fresh candidate is already acceptable with its required data at every correct party, with at least $f + 1$ correct parties attaching origin bit 1 when that candidate is non-skip. Then all correct parties receive the proposal before their 5Δ no-proposal deadlines. A fresh proposal decides by $t + 11\delta$; a carried proposal decides by $t + 10\delta$.*

PROOF. Suggestions and proofs reach the correct primary by $t + 3\delta$. Lemma D.6 makes the Q correct suggestions acceptable. If their maximum key is positive, it is at least every correct lock and Lemma D.7 makes the proposal acceptable at every correct party. If their maximum is zero, the final part of Lemma D.6 implies that no correct lock is positive, and the premise supplies fresh acceptance and, for a non-skip value, the required origin support. The proposal reaches all correct parties by $t + 4\delta$, and their ECHOs arrive everywhere by $t + 5\delta$.

For a fresh proposal, the resulting witnesses arrive by $t + 6\delta$ and make the value Backed. KEY1, KEY2, KEY3, LOCK, and DONE then arrive by, respectively, $t + 7\delta, t + 8\delta, t + 9\delta, t + 10\delta, t + 11\delta$. A carried value is already Backed and omits the witness delay, so the same five arrivals end at $t + 10\delta$. At least $n - f = Q$ correct parties participate. The 5Δ proposal deadline therefore has one delay of slack even when $\delta = \Delta$, and the fresh full-view deadline is met at equality only in that limiting case. A timeout raises WISH without abandoning the current view, so a deadline tie does not suppress the proposal or decision. $\square$

THEOREM D.9 (DIRECT-RESOLVER PROPERTIES). *For each target u :*

- (D1) *no two correct parties decide different resolver values;*
- (D2) *every decided non-skip value is author-backed and satisfies the future-closed compatibility of Lemma D.1;*
- (D3) *if one correct party decides R , every correct party eventually either decides R or already holds its compatible terminal seal; and*
- (D4) *under partial synchrony, every target not already terminally sealed at all correct parties eventually obtains one common resolver decision or the compatible terminal seal at every correct party.*

PROOF. For D1, Lemmas D.4 and D.5 prove agreement directly for the stated message and update rules. The target coordinate separates concurrent instances, and the value digest binds every nonzero value to the session. Every value entering KEY1 is Backed: a fresh proposal waits for its witness quorum, and a carried proposal checks the stable predicate before ECHO. Thus no phase advances a value outside the resolver's external-validity domain. D2 follows from Lemmas C.1 and D.1.

We first establish a candidate-convergence fact used by both D3 and D4. By the Direct AGB response deadlines, every correct party eventually broadcasts its target ECHO-stage and closed initial ready-stage response. Let $b \leq f$ be

the actual Byzantine count. For each payload P , let G_P be the correct parties whose target response is a grade-1 ECHO for P . Call P large when $|G_P| \geq n - f - b$; at most one payload is large because $2(n - f - b) > n - b$.

If a large $P = (C, T)$ exists, the universally acceptable value is $R_u^{\text{full}}(C, T)$. A correct grade-0 or different-payload READY would require a grounding ECHO quorum whose at least $n - f - b$ correct members are disjoint from G_P , contradicting the same inequality. Active locks for other payloads eventually count at least $f + 1$ correct nonmatching responses and release. A correct skip voter would likewise require a disjoint set of at least $n - f - b$ correct echo-skippers, so no correct stance is skip-voted. The at least $n - f - b \geq f + 1$ correct members of G_P supply the candidate evidence, origin bits, data, and provenance.

Otherwise every nonempty G_P is small and every optimistic lock eventually releases after the other correct target responses arrive. No correct grade-1 READY exists: its grounding quorum would contain at least $n - f - b$ correct grade-1 ECHOers and make that payload large. The closed correct ready histories therefore fall into two exhaustive cases: some correct response is a proposal-READY, or all correct responses are NO-READY. AGB value agreement makes all correct proposal-READY responses name the same payload. In the first case, its grounding quorum provides $f + 1$ correct matching ECHOs, all correct READYS are non-grade-1, and the universal value is core. By Lemma D.3(i), that correct proposal-READY also excludes every correct skip vote. In the second, the at least Q correct NO-READY responses make skip universal. Fair repair eventually supplies every required prefix, so in every case one fixed value R^* is in every unresolved correct party's candidate set and passes F1–F5. By Lemmas D.1 to D.3, any correct party already sealed holds the same submitted outcome and can help vote for R^* .

For D3, every unresolved correct party eventually derives the candidate R^* above and therefore starts WISH even if it received none of the earlier resolver traffic. Q DONEs at one correct party contain at least $n - 2f \geq f + 1$ correct senders. Their reliable broadcasts cause every already active correct party to relay DONE. A party activated later broadcasts a WISH; each of those correct senders replies with its own retained DONE, so the $f + 1$ relay starts there as well. The at least Q correct relays then decide everywhere. A party with a local terminal seal already satisfies the alternative in D3; on a peer WISH it can also activate and vote only for the value whose submitted outcome equals that seal.

It remains to prove D4. If a correct resolver decision already exists, D3 gives the claim, so assume none exists. The candidate sets are finite and eventually stabilize. The first local candidate broadcasts WISH. Every correct party that already holds the compatible terminal seal responds to that first peer WISH by broadcasting its own initial WISH; every unresolved correct party eventually derives R^* and broadcasts its own. Thus the Q correct parties supply a view-1 WISH quorum without Byzantine participation. Reliable delivery, WISH amplification, timers, and round-robin leaders then make correct parties enter arbitrarily high common resolver views. The send-upon-join replies return both a responder's own watermark and any earlier phase traffic, so pre-activation delivery alone cannot leave a correct party below an entry or phase quorum. Applied per target, the high-watermark argument of Lemma B.4(b) gives an eventual 2δ correct-entry spread. Explicitly, the Q WISHes seen by the first correct entrant contain at least $f + 1$ correct senders. Their WISHes reach every correct party within one delay and cause amplification; the resulting Q correct WISHes arrive within one more delay.

Choose a sufficiently late common view after candidate sets, Backed facts, and retained value bodies have stabilized. By Lemma D.6, a correct primary accepts Q suggestions. If one is positive, it selects a maximum positive key; the same lemma and Lemma D.7 make that carried proposal acceptable through every correct lock. If all accepted keys are zero, no correct lock is positive. Each unresolved correct primary advances its persistent cursor on its own turns through the finite stabilized candidate set. At least one correct party is unresolved in the case under consideration, and R^* belongs to every such party's set, so one of its primary turns selects R^* . If an intervening fresh value instead reaches

KEY3, it is Backed and the positive-key case applies thereafter. Static round-robin rotation supplies infinitely many correct-primary turns. Therefore a view eventually satisfies Lemma D.8 and decides, proving D4. $\square$

LEMMA D.10 (VANTAGE RESOLVER CONTRACT). *The target-local resolver satisfies RS1–RS3 and RL1 of Appendix C.1.*

PROOF. RS1 and RL1 are D1, D3, and D4 of Theorem D.9. RS2 and RS3 follow from D2 and Lemma D.1. Fast and skip compatibility are given separately by Lemmas D.2 and D.3. $\square$

THEOREM D.11 (PREFIX AND SEAL AGREEMENT). *Each correct party seals a view at most once, and no two correct parties seal different outcomes. If completion exists, every terminal outcome is $\text{commit-full}(C, T)$ or $\text{commit-core}(C)$ for its unique payload and contains its fixed core. Before sealing, a correct completion may expose $\text{open-tip}(C, T)$; for every prior-output set D , $\text{Core}_D(C)$ is a prefix of either compatible non-skip expansion.*

PROOF. Apply Theorem C.2 using Lemma D.10. Lemmas D.2 and D.3 make the caller-side shortcuts compatible with direct and resolver submissions. The persistent arbiter gives local uniqueness, and Theorem B.7 gives the completed-open core prefix. $\square$

THEOREM D.12 (SEALING LIVENESS). *With the data pacemaker and target-local resolver above, every correct party eventually seals every data-plane view. A completed open view is eventually sealed as full or core everywhere.*

PROOF. Fix a target u . If compatible local direct, fast, or grounded skip routes seal it everywhere, the claim holds. Otherwise Theorem D.9(D4) supplies a common resolver value to every party that lacks the compatible terminal seal, and the adapter submits it to the arbiter. Theorem D.11 makes all outcomes equal. A completed view has a READY quorum; RS2 excludes skip, so its result is full or core. $\square$

COROLLARY D.13 (CRASH-ONLY SILENT-VIEW BOUND). *Let $s \geq \text{GST}$ be the first correct entry to data-plane view u , and suppose every correct party enters by $s + 2\delta$. Suppose the proposer and every other faulty party crashed before sending any view- u or later protocol message, so no view- u proposal or proposal ECHO exists. Then every correct party submits $\text{SKIP-SEAL}(u) \rightarrow \text{skip}$ by*

$$s + 4\Delta + 3\delta.$$

If all correct entries are aligned at time e , the bound is $e + 4\Delta + \delta$.

PROOF. Every correct party emits ECHO-SKIP by its absolute ECHO deadline, at latest $s + 2\delta + 3\Delta$. These at least Q correct responses arrive everywhere by $s + 3\delta + 3\Delta$. Every correct party emits NO-READY by at latest $s + 2\delta + 4\Delta$, no earlier than the former time because $\delta \leq \Delta$. By then it therefore holds both its own NO-READY and Q ECHO-SKIP responses.

No correct party can justify a non-skip resolver value because no target proposal ECHO exists, and the faulty parties send no resolver message. Every stance remains free until the vote; an independently learned compatible skip does not suppress it. All correct parties broadcast SKIP-VOTE(u) by $s + 2\delta + 4\Delta$, and those votes arrive everywhere one actual delay later. Aligned entry removes the 2δ entry spread. Each later crashed view has its own deadlines; the corollary does not collapse a burst into one timeout. $\square$

E END-TO-END CORRECTNESS PROOF

THEOREM E.1 (END-TO-END CORRECTNESS). *Every correct party eventually seals every data-plane view with exactly one common terminal outcome. At every time, the output sequences of any two correct parties are prefix-comparable; they have*

the same limit sequence, and every finite prefix is eventually output by every correct party. Every output block occurs at most once, satisfies BLOCKOK, was published with its valid lane prefix by its encoded author to at least one correct party, has a permanent correct holder, and is obtained by each correct party before that party locally outputs it. Moreover, every valid data block published by a correct author is eventually output.

PROOF. Theorems D.11 and D.12 give exactly one eventual common seal per view. For the ordered-output claims, Direct AGB supplies the local properties, Lemma D.10 supplies RS1–RS3, Theorem D.12 supplies per-target totality, and fair repair is Lemma B.2(ii). The only extension beyond Corollary C.3 is the pair of caller-side local routes: Lemma D.2 proves that a fast seal names the same witnessed full outcome as every Direct AGB or resolver submission, while Lemma D.3 proves compatibility of a skip seal. Hence the corollary’s cursor argument applies unchanged. It remains to establish input fairness.

Let a correct author a publish a valid data block b at height k_b . Every correct party eventually receives the full lane prefix through b in the author’s PUBLISH messages. Fix a correct party p_i . By Lemma B.4(a), R1, round-robin assignment, and non-Zeno execution, p_i has infinitely many data-plane proposer turns; resolution runs in separate target-local instances and never consumes a data-plane proposal. If a is quarantined, its exponential probe schedule is also infinite: among the finite author set, earliest-due selection is fair because every selected author’s next due opportunity moves strictly forward. These claims remain true when the stored witness was fabricated, since eligibility tests the proposer’s real directly published lane state.

Consider such turns after the pacemaker cutoff. If p_i already has an exact- $n-f$ core prefix covering b , a subsequent proposal carries it in C . Otherwise, if a stable confirmation target exists above the current core, the next ordinary inclusion or fair due probe for a names the least-height such target, not a fresher head. It is a fixed coordinate of the correct author’s unforked lane, so every correct party eventually has DIRECTPUB for it. A sufficiently late proposal naming it therefore receives exact-position claims from all correct parties and makes it exact- $n-f$ -available. Either that proposal full-seals the prefix, or a subsequent proposal carries the new candidate in C and advances the core beyond its previous coordinate.

If no stable target exists, take an ordinary inclusion or due probe after every correct party has received b . Unless the core already covers b , R1 names a real directly published tip covering b . Each correct ECHO claim for that tip acknowledges a direct prefix covering b — exactly at the named position or through its verified integer-derived prefix. Consequently every coordinate above the current core through b becomes generally $n-f$ -available. If the proposal full-seals, it already outputs b ; otherwise the least such coordinate becomes the stable confirmation target of the preceding paragraph. Each confirmation either outputs the target or makes it the exact- $n-f$ candidate carried by a subsequent core proposal; that proposal advances the core. Induction over the finite coordinates through k_b therefore eventually covers b . Continuous publication cannot move the least unconfirmed target, and no periodic availability message is used.

Choose a final full-sealing confirmation, or the subsequent proposer turn that carries the confirmed prefix in C , after the cutoff. In the latter case Theorem B.8 makes the proposal complete everywhere even if some unrelated tip spoils its grades; completion makes its core irrevocable and the resolver eventually seals the view. The output cursor therefore emits the confirmed prefix. This proves input fairness. All remaining premises of the cursor argument in Corollary C.3 now hold, and it gives the remaining claims. $\square$

COROLLARY E.2 (STATE-MACHINE REPLICATION). *Fix an initial state and a deterministic transition function defined on BLOCKOK blocks. Applying output blocks in order gives correct parties identical states after every common output prefix, and every finite state prefix is eventually reached by every correct party.*

F IMPLEMENTATION AND EVALUATION DETAILS

F.1 Implementation and measured encoding

Our Rust implementation uses one deterministic protocol task and separate networking and storage. It performs no public-key or signature operation. Validators retain 32-byte identifiers internally but use canonical one-byte wire indices; the transport binds every connection to a committee identity and authenticates each frame with a pairwise MAC, on by default (Section 7.1).

The same binary runs both Autobahn modes and two Simple-IT variants on a shared TCP, batching, worker, storage, client, and metric stack [16, 20, 45]. Simple-IT follows its paper, adds a safe notarization bound, and fetches missing proposals; Bluestreak and Sailfish++ run on the separate artifact [31, 32]. Timer-calibrated in-tree runs set $\Delta = 200$ ms: the largest injected RTT is 309 ms, hence the largest one-way delay is 154.5 ms; we round this to 160 ms and reserve 40 ms for processing and scheduling. Vantage uses $\theta_E = 3\Delta = 600$ ms and $\theta_R = 4\Delta = 800$ ms. The fault-free Q1 and Q2 runs do not consume a fallback decision. The target-local resolver used in Q4 and Q5 has a $5\Delta = 1$ s no-proposal timer and an $11\Delta = 2.2$ s full-view timer. Both optimistic all-to-all and certified-tip Autobahn use the paper’s $10\Delta = 2$ s consensus timeout; all-to-all routing and tip selection do not alter it. Simple-IT uses $8\Delta = 1.6$ s with Opt-RBC and $5\Delta = 1$ s with Bracha RBC. Autobahn’s separate 500 ms unanimous-fast-path wait is an implementation tuning, not a view timeout. The upstream artifact implements linear Autobahn; we add its paper’s all-to-all exchange for the optimistic variant. Other shared settings are a 100 ms header delay, 64 KiB/5 ms network flushing, and 20 ms worker batching; Vantage and Simple-IT use BLAKE3 and round-robin leaders.

F.2 Measurement methodology

Network and workload. Validators use private addresses in one availability zone. Netem assigns validator i row $i \bmod 10$ of Table 2, placing half the RTT on each direction without jitter [31]. Variants at one committee size reuse instances and rules; sizes use new instances, so cross-size trends include AWS placement variation. Each validator offers R/n open-loop load in 50 ms bursts. Transactions are timestamped, random, effectively incompressible 512-byte values [31, 32].

Workers store and disseminate batches before passing digests to the primary. Vantage puts them in unsigned lane blocks, broadcasts headers every 100 ms, and fetches missing batches; the interval approximates the uncertified-DAG round duration.

Metrics. After all validators advance, runs warm up for 30 seconds and measure for 120. Latency spans submission through local sequencing and byte retrieval [31, 45]; throughput is sequenced transactions per window. We report the median validator’s p50/p99 latency, total primary-plus-worker CPU, and outbound framed bytes per sequenced byte. Cloud points run as up to three campaign repetitions on fresh fleets; every reported metric is the median across the available repetitions of its per-run value. The $n=100$ committee-scaling cell joins from the throughput campaign’s 100 tx/s point, which every repetition measures.

Baseline fleet. We measured both artifact baselines on c5d.2xlarge and m5d.2xlarge (equal 8 vCPUs; 16 vs 32 GiB RAM; 200 vs 300 GB local NVMe) and report each from its better fleet. Bluestreak runs on m5d.2xlarge: on c5d.2xlarge its throughput collapses well before the highest accepted point it reaches on m5d.2xlarge, with six validators killed for exhausting memory. The limit is memory rather than CPU. Sailfish++ stays on the shared c5d.2xlarge fleet. Bluestreak therefore runs on a stronger instance than VANTAGE; no baseline runs on a weaker one.

Table 2. Injected round-trip latency matrix (ms). Row and column order is the committee-index assignment order.

	USE1	CAC1	EUS2	SAE1	APSE2	USW1	EUW1	EUN1	APS1	APNE1
USE1	1	14	104	112	198	65	68	110	201	146
CAC1	14	1	106	122	196	78	67	103	189	142
EUS2	104	106	1	215	281	163	29	50	143	238
SAE1	112	122	215	1	309	175	176	220	299	254
APSE2	198	196	281	309	1	137	254	268	150	101
USW1	65	78	163	175	137	1	127	172	226	108
EUW1	68	67	29	176	254	127	1	38	125	199
EUN1	110	103	50	220	268	172	38	1	148	245
APS1	201	189	143	299	150	226	125	148	1	140
APNE1	146	142	238	254	101	108	199	245	140	1

Table 3. End-to-end latency as p50/p99 in milliseconds, for the median validator, taken as the median across the available campaign repetitions. These are the cells the corresponding figures plot, with the tail added. Columns are VANTAGE, optimistic all-to-all Autobahn (A2A), seamless Autobahn, the two Simple-IT variants, Bluestreak, and Sailfish++. A dash marks a size or offered-load point the variant does not reach, or a variant the relay stress does not cover. Overload endpoints are excluded, since a variant past overload reports a cumulative percentile over pre-collapse traffic.

	VANTAGE	A2A	Seamless	IT-Opt	IT-Bracha	Bluestreak	Sailfish++
<i>Committee scaling at 100 tx/s (Figure 5)</i>							
$n=10$	446/599	474/677	706/992	642/802	774/970	463/627	682/1957
$n=20$	446/601	484/677	684/950	666/833	782/985	466/628	704/2268
$n=50$	442/603	490/694	692/969	669/864	777/985	473/633	773/1518
$n=100$	452/613	560/776	795/1097	699/868	778/985	480/640	786/2091
<i>Throughput ladder at $n = 100$, accepted rungs (Figure 6)</i>							
100 tx/s	452/613	560/776	795/1097	699/868	778/985	480/640	786/2091
10k	456/616	566/788	800/1112	692/878	780/983	481/641	788/1992
150k	475/640	604/832	838/1150	704/897	794/1001	512/786	840/2910
200k	483/646	616/850	852/1163	708/904	800/1010	522/870	—
225k	487/653	629/860	864/1178	712/908	804/1018	542/1283	—
250k	492/654	636/868	877/1198	718/916	810/1028	541/1465	—
275k	509/698	1130/3306	1442/3553	742/992	825/1064	—	—
<i>Leader-relay stress at $n = 20$, $f = 6$ (Figure 7)</i>							
100 tx/s	450/1395	809/1154	—	674/861	—	—	—
1k	452/1354	860/1144	—	668/863	—	—	—
10k	454/1392	1706/2592	—	672/868	—	—	—
20k	460/1426	1836/2694	—	679/872	—	—	—
100k	458/1317	1886/3562	—	674/867	—	—	—
200k	462/1226	2336/5660	—	672/859	—	—	—

F.3 Representation optimizations

The protocol places lane-frontier acknowledgments in the authenticated envelope of each proposal ECHO, outside its immutable Direct AGB identity. Since $\mathcal{H}(B_v)$ binds an ordered reference vector, a bitmap marks exact directly published positions. A sender behind a named tip may instead give a sparse pair containing the position and positive integer distance to its greatest directly published ancestor. The pair carries no digest: a receiver credits it only after the named tip’s verified predecessor chain derives the exact ancestor, and otherwise keeps it pending. Once verified, either form acknowledges the sender’s whole directly received prefix through that coordinate, so one claim can backfill older

lane coordinates. Every resulting (a, k, h) remains attributed to and counted first-hand from the envelope sender; the envelope is neither a certificate nor a forwarded-MAC vector. Batch-recovery ECHOs remain by value and use the same claims over their core and tip vectors. Only a quorum of exact-position claims promotes a correct proposer’s core; integer-derived counts remain valid for `AUTHOROK` and eventual availability but are not assumed to become common within one delay. A general quorum therefore freezes the least-height unconfirmed direct prefix as an exact-position confirmation target; it neither promotes the core nor clears completed-open quarantine by itself.

The implementation retains periodic digest-bearing availability watermarks behind the benchmark switch that disables ECHO claims. This is an A/B oracle, not the default protocol or a liveness dependency.

The proposal travels by value; later single-proposal ECHOs and READYs name $\mathcal{H}(B_v)$, a grade, and any origin bit. Before a digest quorum acts, the receiver must hold the canonical body and execute the unchanged by-value rule. Statement authors retain and serve it, but served bytes create no provenance or vote. Transport batches to 64 KiB or 5 ms. Bounded recovery and state reclamation in the artifact are outside the protocol and theorem statements.

With λ bounding a digest, identifier, and reference, the proposal and digest statements cost $O(n^2\lambda)$ bits over all deliveries; the $O(n)$ -bit ECHO bitmaps add $O(n^3)$. Thus the aligned single-proposal encoding uses $O(n^2\lambda + n^3)$ control bits and $O(n^2)$ logical messages. Sparse integer exceptions cost $O(n^3(\log n + \log v))$ bits, where v bounds heights and distances; by-value recovery retains the proved $O(n^3\lambda)$ statement bound under the paper’s scalar-size convention.

F.4 Experiment-specific records

Committee scaling. The seven protocol variants at $n \in \{10, 20, 50\}$ use one on-demand c5d.2xlarge validator per instance (8 vCPUs, 16 GiB RAM, 200 GB local NVMe) in eu-west-1a, the matrix above, 100 aggregate tx/s, five-second Prometheus scrapes, and the common warmup/window. All 21 first attempts complete at approximately the offered load, with no reported netem drop or panic. The $n = 100$ row instead joins each variant’s 100 tx/s point from the throughput campaign and therefore inherits that figure’s fleets (Appendices F.2 and F.4); this is why Bluestreak’s $n = 100$ cell is m5d.2xlarge while its smaller sizes are c5d.2xlarge.

Fresh-reference cadence. For the $n = 100$, 100 tx/s points, we subtract each validator’s progress counter at the measurement-window boundaries, divide by that run’s actual window, and take the median validator. The available runs give about 14 Vantage proposal views/s, about 10 Bluestreak DAG rounds/s, about four Sailfish++ rounds/s, and about eight slots/s for both Autobahn modes. Autobahn has no direct slot gauge, so its rate is derived from received ConsensusRequest counts: one request per optimistic slot and two per seamless slot. The generator records every run’s median-validator delta in `carrier-cadence.json`. These counters measure the next opportunity to reference fresh data, not equivalent protocol work or capacity.

View anatomy and network-delay control. The $n = 20$ mechanism study uses the target-local resolver implementation with no fault-mode switch enabled; the resolver therefore remains inactive. All validators run in one release binary on the 32-vCPU, 124-GiB native-Linux host used for Q5, with matrix delays injected in process rather than through the Docker netem path, the optimized encoding, and 100 tx/s. Each of three 60-second runs submits 6,000 transactions and materializes all of them after drain. A reported percentile is first taken across validators and then the median across runs.

Fast-seal waits for all 20 matching ECHOs, while Direct AGB may finish from $n - f = 14$ READYs. Under heterogeneous delays, the Direct AGB result sometimes precedes the slowest ECHO, so fast-seal supplies about 70% of local seals; at a uniform 143 ms RTT every observed seal uses fast-seal. All seals are full. Uniformization changes coverage from 175 to

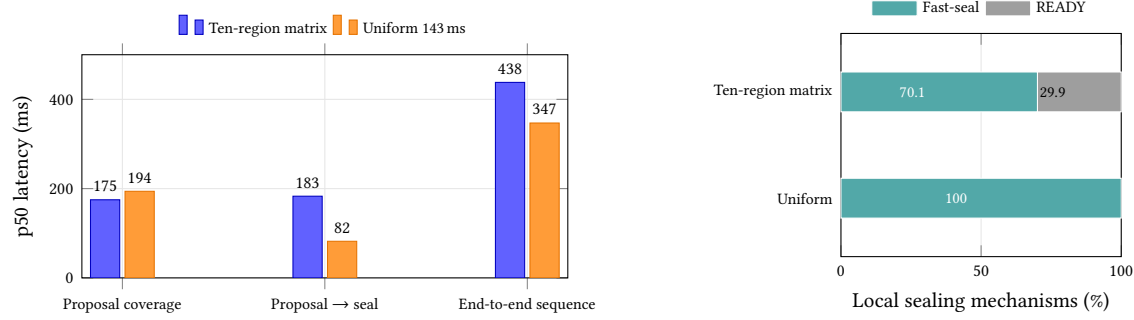


Fig. 11. Matched network-delay control at $n = 20$. Left: p50 stage and end-to-end latency. Right: local sealing mechanisms summed over validators. The uniform value is the 143 ms median off-diagonal RTT after assigning the ten-region matrix to 20 validators.

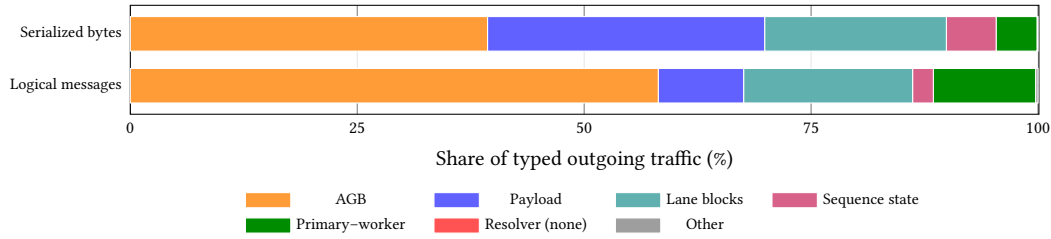


Fig. 12. Outgoing typed bytes and logical unicast messages by module in a representative matrix run of the $n = 20$ mechanism study. The inactive resolver sends nothing.

194 ms, proposal-to-seal from 183 to 82 ms, and reduces end-to-end sequencing latency from 438 to 347 ms. Figure 12 gives the outgoing traffic shares by module in the representative matrix run.

Under the matrix, the run-median p50/p99 is about 444/596 ms from submission to materialization. From block publication, author-observed first proposal coverage is about 175/320 ms, exact- Q availability 298/404 ms, and ordering 371/466 ms. Proposal receipt to local full seal is about 183/275 ms. Under uniform RTT, submission-to-materialization is about 353/434 ms and every seal uses fast full. The remaining local p50 intervals are about 11 ms from submission to batch seal, 51 ms from the first batch digest to author-block publication, and 6 ms from ordering to materialization. The 51 ms is the expected half of the 100 ms header delay, and the 32 ms by which author-observed coverage exceeds the 143 ms median RTT is the expected half of the roughly 70 ms proposal spacing.

The representative matrix run sends about 1.2 million typed logical messages, 200 MB of typed-message bodies, and 230 MB physically over 60 seconds. AGB contributes about 40% of typed bytes and 60% of messages; payload plus lane blocks contribute about half of the bytes and 30% of messages. Sequence state and primary-worker traffic account for nearly all of the remainder. The target-local resolver contributes no bytes or messages. Optional tracing retains at most 4,096 block timestamps per validator.

For a hardware-consistency check, we retain the same six repetitions on the original Apple M4 Pro. Its matrix p50/p99 is about 445/599 ms, its uniform p50/p99 359/445 ms, and it again emits no resolver traffic. Q4 reports the native-Linux measurements and uses the workstation set only as a consistency check; the two machines are not pooled.

Throughput sweep. The $n = 100$ sweep offers 100, 10,000, 150,000, 200,000, 225,000, 250,000, 275,000, and 300,000 tx/s on c5d.2xlarge validators (8 vCPUs, 16 GiB RAM, 200 GB NVMe) in eu-west-1a; the Sailfish++ campaign adds a 170,000 tx/s point.

The campaign strictly validates all validators at 100 and 10k and treats higher loads as exploratory. Acceptance means at least 95% median throughput, with the first lower point retained as overload. Accepted points end with 100 responsive validators and no reported netem drops. VANTAGE, optimistic all-to-all Autobahn, seamless Autobahn, and both Simple-IT variants pass through 275k and overload at 300k; Bluestreak overloads at 275k and Sailfish++ at 170k. These are deployment endpoints, not capacity bounds.

Leader-relay stress. The $n = 20, f = 6$ campaign offers 100, 1,000, 10,000, 20,000, 100,000, and 200,000 total tx/s. Six Byzantine publishers receive the same per-author load as every correct publisher. Each creates one batch per Δ , sends it directly to a fixed five-party correct-holder group, exposes the lane header, and refuses repair. The publisher retains its own copy and correct holders serve normally, so each batch has six direct holders but remains below the $f + 1 = 7$ PoA threshold. The six holder groups jointly cover all 14 correct consensus leaders while preserving a complete prefix at every holder. In the Autobahn variant, Byzantine consensus leaders propose their certified cut rather than self-inflicting a timeout. Across the three variants, the only emulated faulty behavior is partial data publication and repair refusal.

Each protocol uses a fresh fleet of 20 on-demand c5d.2xlarge validators plus one control instance in eu-west-1a. Protocol traffic uses private addresses and the matrix of Table 2 through netem; observed probes span approximately 1–309 ms. The common 30-second warmup, 120-second window, 512-byte transactions, calibrated timers, and positional Vantage encoding apply. Useful throughput excludes transactions from the six Byzantine publishers; their Autobahn commitment is measured separately. All points except the 200k optimistic Autobahn retry report 20/20 healthy validators, zero netem drops, and zero panics. The retry ends at 19/20 healthy; the overloaded host does not expose its final netem counter. A first 200k attempt, retained only as a diagnostic, stalls two validators and is not plotted.

Direct-resolver recovery. Q5 runs Docker on a separate native-Linux host with 32 virtual CPUs and 124 GiB RAM. Both profiles use $\Delta = 200$ ms, the ten-region netem matrix, a 100,000-packet netem limit, 512-byte transactions, compact committee IDs, authenticated channels, and one-second Prometheus reads. These local mechanism runs do not share a fleet or measurement window with the cloud campaigns.

The transient profile uses $n = 10, f = 3$ for 130 seconds. The counted 1,000 tx/s load is assigned only to the seven non-victims. At second 20, the other three otherwise-correct containers receive SIGKILL simultaneously; they restart with their stores after a configured 30-second blackout, about 31 seconds in the observed timeline. Each process exports an exact latency histogram cleared every second, and Prometheus reads every second. The figure plots the non-victim median at each read, not a repetition aggregate: throughput is a trailing five-second rate and latency is the latest-one-second p50. All ten validators are live at the end. This profile has one run.

The mixed-open profile uses $n = 10, f = 3$ for 240 seconds and three fresh committees. From seconds 20–110, each Byzantine publisher proposes only its own partially disseminated current tip, ignoring its local quarantine, and suppresses its outgoing AGB, skip, WISH, resolver-phase, witness, DONE, and resolver-serve messages. Outside that interval it resumes ordinary behavior immediately. Correct publishers offer the counted 1,000 tx/s; the faulty publishers add 600 uncounted tx/s. Resolver completion is read from protocol events at every correct validator; the plotted backlog is the committee median of completed-open injected targets not yet sealed locally. The arrival/seal comparison uses the common steady interior of each attack, excluding its boundary transients. All reported Q5 runs end with every correct validator live, no panic, and the injected backlog drained.

Prometheus evaluates both mixed-open metric series every second and excludes the three Byzantine validators. Throughput is a trailing-five-second rate; latency is the median of the correct validators’ latest-one-second materialization p50. The figure first computes those per-run committee medians and then shows their three-run median and min-max band. Empty one-second latency windows remain missing rather than repeating a stale gauge.

The plotted mixed-open repetitions use the complete resolver path, including admission, candidate cycling, retained-terminal participation, and send-upon-join replay. Resolver unit/integration tests and an $n = 31$ smoke run provide additional functional coverage.

Resolver instances activate once the proposal frontier reaches $u + 3$. In this profile a Byzantine primary sends no resolver proposal, so its view ends at the $5\Delta = 1$ s no-proposal timeout. A target can encounter up to two such primaries before a correct-primary execution. The separate $11\Delta = 2.2$ s deadline applies after a proposal appears and need not expire in a successful correct-primary view. The traces report target completion and output release, but do not divide the observed delay among these stages.

Inactive-path check. A separate one-run-per-arm, fault-free $n = 30$ diagnostic at 1,000 tx/s checks that merely enabling the target-local resolver does not perturb the inactive path. Both builds sustain the offered load at about 0.44 s p50, 0.60 s p99, and 0.21 median validator CPU cores, with no observable inactive-path regression.

Artifact scope. The implementation and harness repositories named in Section 7.1 contain the implementation, experiment configurations, generated figure inputs, and per-run records; the figure data of this paper are under recorded/paper-fixedbuild-20260822 in the harness repository. Q1 and Q2 terminate through homogeneous AGB and consume no fallback decision; Q3 stresses partial-publication admission. Q4 keeps the target-local resolver inactive, whereas Q5 exercises it under crash and mixed-open faults. The matched $n = 30$ check above compares the inactive path before and after enabling the target-local resolver.

Reproduction. In the artifact’s wan-bench repository, run `wanbench campaign --config CONFIG --execute;` its paper configuration index names the committee, throughput, baseline, and leader-relay campaigns. Regenerate the figure inputs with `python3 vantage/gen_paper_dat.py`. For the local controls, build the artifact’s vantage-bft repository with `cargo build --release -p node --features pipeline-tracing` and run `./target/release/node local-benchmark`; the repository contains the Q4 flags for the matrix and uniform-RTT arms. Repeat each arm three times with a fresh data directory.

For Q5, run the scripts in `vantage-bft/docker-bench`. The sustained profile uses `./direct_resolver_q5.sh`; the transient profile uses `./transient_crash_q5.sh`. The recorded configurations supply the durations and repetition count. The latter fixes $n = 10$, $f = 3$, keeps the counted load off validators 0–2, schedules their 30-second blackout. The plotting and validation commands are `direct_resolver_q5.py` and `transient_crash_q5.py` in the same directory; each accepts the archived run root and an output directory.

G EXTENDED RELATED WORK

Section 3 gives a normalized quantitative comparison with five baselines. Here we place those protocols and other related work in the broader design space, organized by where a protocol spends strong agreement and whether its evidence is transferable.

Signature-free BFT. Agreement without digital signatures is classical: Castro’s MAC-vector Algorithm BFT reaches a 3δ good case, with cubic authenticator communication from recipient-specific MAC vectors [12, 13]. Those vectors provide

forwarded per-recipient evidence and hence lie outside our channel-only model. Mostéfaoui, Moumen, and Raynal gave signature-free asynchronous binary consensus with $O(n^2)$ messages [27]. Information-Theoretic HotStuff brought the rotating leader-and-view discipline to authenticated channels without signatures or a PKI [3]; TetraBFT reduces the good case to 5δ [44]; Forget-IT attains the good-case optimum of 3δ [1]. Simple-IT makes the line practical with reliable notification and an optimistic broadcast path under a stronger honesty threshold [45]; speculative pipelining can narrow leader-proposal spacing at the cost of requiring runs of correct leaders. Its shared-mempool variant is compared directly in Section 3. Vantage instead instantiates fresh IT-HS agreement independently for each unresolved target. A one-type first-hand witness relay supplies the stable external-validity predicate that the standalone agreement protocol assumes, and typed WISH messages synchronize each target’s resolver views. No instance runs before target-local evidence or $f + 1$ peer WISHes activate it. This dependency can gate output after an open tip but remains outside direct completion and homogeneous sealing. A different route to the same goal buys authentication from the infrastructure: SwitchBFT drops protocol signatures by relying on packet source authentication in a trusted datacenter network, and programs a switch to enforce agreement decisions and check that safety is not violated, matching the speed of an in-switch crash-fault-tolerant protocol [46]. Its trust therefore rests on the network — a fabric in one administrative domain with a trusted switch on the path — where VANTAGE assumes only pairwise authenticated channels between validators and no trusted network element, which is what lets it run over the wide-area committee of Section 7.

Certified-data designs. Narwhal separates availability-certified dissemination from ordering [16]; DAG-Rider [21], Bullshark [39], Shoal and Shoal++ [5, 38], and Sailfish [36] order DAGs in which every vertex is reliably broadcast or availability-certified, so ordering adds little communication but every data unit completes a strong-broadcast or certification round before it can be referenced; most practical instantiations also use signatures. DAG-Rider’s RBC-DAG is signature-free given an ideal perfect coin, although its standard practical coin realization uses threshold cryptography for liveness. Sailfish++ makes the Sailfish line signature-free by running optimistic signature-free RBC for every vertex [37]; Section 3 details the resulting nonleader classes and the contrast with Vantage’s graded tip. On the signed side, Autobahn combines PoA-certified per-author lanes with a PBFT-style core and optionally admits an uncertified current tip [20]; Section 3 compares that optimization and its seamlessness tradeoff directly with VANTAGE’s graded tip. Dumbo-NG separates continuously growing signed data streams from asynchronous agreement [19], and Raptr integrates prefix certification of uncertified batches into leader consensus [40]. Simplex [14] and DispersedSimplex [35] are the signed leader-dissemination baselines against which the signature-free leader line measures itself.

Concurrent work, Multimmit, independently addresses missing data in a signed multi-chain protocol [24]. Its votes report a supported prefix per producer chain and may extend the leader’s proposed tips, so one missing producer need not block the other chains, a localization they prove rather than approximate. Its per-chain report is a finer split than Vantage’s single grade bit. Its extension votes omit a step that Vantage retains: a Vantage tip entry must reach the proposer before the proposal. In its good case a block disseminated at t is ordered by $t + 3\delta$ in expectation and $t + 2\delta$ at best, measured from dissemination as ours is. The cost is cryptographic and in resilience: $n \geq 5f + 1$, a PKI, and ordinary plus aggregate signatures, together with two threshold schemes that their optimization section shows can be replaced by the aggregate scheme alone. Aggregation is what keeps their certificates succinct, and the standardized post-quantum signatures have no comparable aggregate [29, 30], so a post-quantum instantiation would keep the structure and lose the succinctness. Vantage studies signature-free one-delay admission with $n \geq 3f + 1$. Because it is concurrent work with no reported evaluation and a different fault threshold, it is not included in Table 1.

Other concurrent work takes proposal pipelining beyond the one-round cadence of DAGs: Cadence runs independent signed slots at an arbitrary configured interval τ , with concurrent proposers disseminating coded encrypted proposals against τ -spaced deadlines, and signed include/exclude evidence plus multivalued Byzantine agreement (MVBA) resolving a failed fast path [7]. Cadence can configure τ independently of δ , a stronger pacing result than our all-correct at-most- δ fresh-opportunity gap. Its favorable per-slot finality is $\Delta + 2\delta$ from the formal slot start $D - \Delta$, where D is the slot deadline, although in-order append may wait for earlier slots. Its endpoint and mechanisms — aggregate signatures, per-slot threshold encryption, and proposer-local input rather than a distinct data-block author — place it outside the comparison experiment of Table 1. VANTAGE instead studies how far pipelined multi-author data can go with hashes and non-transferable first-hand counts alone.

Banyan paired the $n = 3f+2p-1$ two-delay fast path with rotating leaders [41]; concurrent FinWhale brings it to Mysticeti, using evidence blocks to reconcile fast and slow decisions across local DAG views [23]. At $p = 1$, it keeps $n = 3f+1$ and survives one non-voter, unlike Vantage’s all- n fast-seal shortcut, but its blocks remain signed. The two systems share the 2δ carrier-relative endpoint; their aligned totals and FinWhale’s round bound are in Table 1.

Recent hybrid and sparse DAGs. Recent work reduces certified-DAG metadata without removing the data-side dissemination gate. Angelfish replaces some nonleader RBC vertices with one-delay best-effort votes, but those votes carry no transactions; every transaction-carrying vertex remains reliably broadcast, and the protocol uses BLS certificates [43]. Clownfish sparsifies certified-DAG edges in RBC- and signed consistent-broadcast variants and explicitly leaves a signature-free adaptation to future work [42]. DAGs for the Masses uses verifiable sampling to reduce Bullshark’s parent set, while every vertex remains reliably broadcast and references carry signed availability evidence [4]. Morpheus is a closer signed endpoint: its Autobahn-style mode lets a leader hash-reference a freshly received author block, but voters proceed only after obtaining that block’s signed 0-QC [25]. Hashgraph’s constant-degree gossip DAG likewise uses signed two-parent events [8]; the broader certified and uncertified DAG design space is surveyed in [34].

Uncertified data with signatures. Cordial Miners introduced leader-based ordering of a best-effort block DAG [22]; Mysticeti improves latency by pipelining leaders, committing a signed uncertified DAG with a three-round leader path [6]. Its idealized aligned cross-author block path is 4δ ; the familiar 4.5δ starts at transaction arrival and adds the average wait for the author’s next block. The Starfish paper presents the pacemaker under which this line is provably live, and its Mysticeti-L variant improves consensus bandwidth with multisignatures. Its explicit all-correct post-GST construction for Mysticeti approaches $5\frac{1}{3}\delta$ average block latency; adding the $\delta/3$ admission induced by that synchronized arrival schedule gives $5\frac{2}{3}\delta$ average transaction latency, a different experiment. Starfish itself further decouples availability through coded dissemination [31]. Bluestreak reduces nonleader metadata without multisignatures [32], and concurrent FinWhale adds a two-round fast path to this family [23]. In these protocols, signatures keep relayed blocks and votes attributable. Vantage instead distinguishes direct author publication from repair and counts author-receipt acknowledgments first-hand.

Starfish-S is the closest mechanism lineage: its optimistic, standard, and pending grades use later committed anchors [33]. Vantage replaces its signed DAG evidence with first-hand response histories and locally validated entries.

Graded and abortable broadcast. Feldman–Micali gradecast grades confidence in one value for Byzantine agreement [18]; AGB instead grades optional-suffix inclusion after fixing a common core and omits faulty-proposer totality. BBKA gives an indivisible value Complete–Adopt, and BBKA-Chain recovers a backbone with transferable adopt/no-adopt evidence [26]. AGB’s READY/NO-READY and MIX refinement imply a graded counterpart, but Vantage resolves

open and general silent views through target-local agreement, using its grounded shortcut only for clean crash silence. Optimistic RBC retains reliable-broadcast totality [9, 37]. The BBChain comparison in Section 3 gives Vantage no bounded fallback. GradedDAG grades a different object in an asynchronous signed DAG [15].

Foundations. AGB uses Bracha’s echo/ready pattern [9]; Abraham, Ren, and Xiang classify unauthenticated broadcast latency [2]. Its pacemaker combines the bounded-space synchronizer of Bravo, Chockler, and Gotsman with Starfish’s two-response trigger [10, 33]; verifiable dispersal could compress data and repair independently [11]. The on-demand resolver adopts IT-HS’s three-key and lock-opening discipline for first-hand authenticated messages [3]. Separately, Vantage excludes overlap between grounded skip votes and ECHOs for fresh non-skip resolver values, without $f + 1$ skip-vote amplification. Its $f + 1$ relay applies to resolver witnesses only after a genuine fresh ECHO quorum.